FINSIGHT

Complete System Architecture, Implementation & Code Walkthrough

From UI to Database, Django Backend, FastAPI Analytics & Anomaly Detection

Scope Every claim verified against the source tree at `~/Desktop/FinSight`

Repository 3 services · 8 Django apps · 5 tables · 25 REST endpoints · 70 automated tests

Measured ~3,300 lines Python (backend) · ~1,360 (analytics) · ~4,760 TypeScript (frontend)

Convention Not in the code → labelled PLANNED · present but unused → PARTIAL · working → IMPLEMENTED

React 18 · TypeScript 5.6 · Vite 5 · Material UI 6 · Redux Toolkit 2 · RTK Query · Recharts · Django 4.2 · DRF 3.15 · SimpleJWT · drf-spectacular · PostgreSQL 16 · SQLite fallback · FastAPI · Pydantic 2 · pandas · NumPy · SciPy · scikit-learn · Redis 7 · httpx · Docker Compose · nginx · Gunicorn

Table of Contents

> **HOW TO READ THIS**
>
> Part numbers match the original brief; parts are grouped into chapters that follow a request through the system. Every implementation claim carries a file path in `this style` — if you cannot find a claim's file, the claim is wrong.

PART 1 · CHAPTER I

Project Overview

FinSight is a personal-finance management and spending-analytics platform. A user adds accounts and cards, records income and expenses against categories, and gets back a live dashboard, statistical analysis of their spending, and automatic detection of transactions that are unusual *for them*.

1.1 The plain-language version

You spend ₹400–₹1,200 on food most days. One evening an ₹8,000 restaurant bill lands. A bank statement shows that line among two hundred others; it does not tell you it is ten times your normal food spend. FinSight does, in a sentence: "*₹8,000 is about 10.4× higher than your average Food expense (₹768).*" That sentence is the product. The accounts, categories, balance keeping and statements exist to make it trustworthy.

1.2 Problem, user, and what makes it interesting

Problem	How FinSight addresses it
Money is recorded, not understood	Totals, averages, medians, per-category shares, month-over-month growth and savings rate computed on top of the ledger
Unusual spending hides in volume	Three detectors (robust Z-score, IQR, Isolation Forest) over the user's own history, each producing a severity and a plain-English reason
"Unusual" is relative	Statistics computed **per category**, never one global mean. ₹18,000 rent is normal; ₹8,000 food is not
Balances drift	Every create/update/delete adjusts the balance in one DB transaction with row locks, always reversing the previous impact first
Multi-account reality	Bank, cash, credit card, wallet, investment accounts coexist; statements per-account or consolidated

Target user: a salaried individual. The seed data is Indian and INR-denominated — `backend/common/management/commands/seed_demo.py` generates six months of Swiggy/BigBasket/Uber/IndiGo transactions against an ₹85,000 salary and ₹18,000 rent. Single user per account: no household sharing, no multi-tenancy, no accountant role.

> **WHY THIS PROJECT IS TECHNICALLY INTERESTING**
>
> 1. **A genuine service split along a real seam** — state versus computation — not two frameworks for their own sake.
> 2. **Money correctness is enforced**: `transaction.atomic()` + `select_for_update()` on every mutation, with 10 of the 43 Django tests existing purely to prove balances stay right.
> 3. **Graceful degradation is designed**: the dashboard computes locally and only *enriches* with FastAPI, so an analytics outage returns 200 with `analytics_available: false`. There is a test that kills the connection and asserts this.
> 4. **Statistics are grouped**, and the Z-score is MAD-based — robust to the very outliers it hunts.
> 5. **The detector is swappable** behind one response contract (`AnomalyItem`).

1.3 Feature inventory — what actually exists

Feature	Status	Where
Register / login / JWT refresh / logout / profile / change password	IMPLEMENTED	`backend/users/`
Accounts CRUD (6 types) with opening balance	IMPLEMENTED	`backend/accounts/`
Categories CRUD · 19 defaults auto-seeded · delete-with-reassignment	IMPLEMENTED	`backend/categories/`
Transactions CRUD with automatic balance keeping	IMPLEMENTED	`backend/transactions/`
Filtering (8 filters), search, ordering, pagination	IMPLEMENTED	`transactions/filters.py`
Monthly statements, opening/closing balance, JSON + CSV	IMPLEMENTED	`backend/statements/`
Dashboard: 4 stat cards, trend, breakdown, top expenses, recents, alerts	IMPLEMENTED	`analytics/services.py`
Summary / trends / category analytics / one-shot analyze + insights	IMPLEMENTED	FastAPI `app/services/`
Anomaly detection — 3 algorithms	IMPLEMENTED	`app/services/anomaly_service.py`

Anomaly persistence + review/ignore/reopen workflow	IMPLEMENTED	`backend/anomalies/`
Redis-backed analytics caching (900 s TTL)	IMPLEMENTED	`analytics/services.py`
Service-to-service bearer token (constant-time compare)	IMPLEMENTED	`app/security.py`
OpenAPI 3 + Swagger + Redoc, both services	IMPLEMENTED	drf-spectacular; FastAPI native
Django Admin for all five models · demo seeding command	IMPLEMENTED	each `admin.py`
Docker Compose: postgres, redis, fastapi, django, frontend/nginx	IMPLEMENTED	`docker-compose.yml`
70 automated tests (43 Django + 27 FastAPI)	IMPLEMENTED	`*/tests.py`
Server-side JWT revocation on logout	PARTIAL	`token_blacklist` app not installed — see Part 8
Notification preferences stored but nothing sends notifications	PARTIAL	No email backend or scheduler
Frontend unit/component tests	PLANNED	None; `make test-frontend` only type-checks and builds
Celery, CI/CD, cloud deployment, bank import, OCR, budgets, goals	PLANNED	Not present anywhere in the repo

1.4 Architecture at a glance

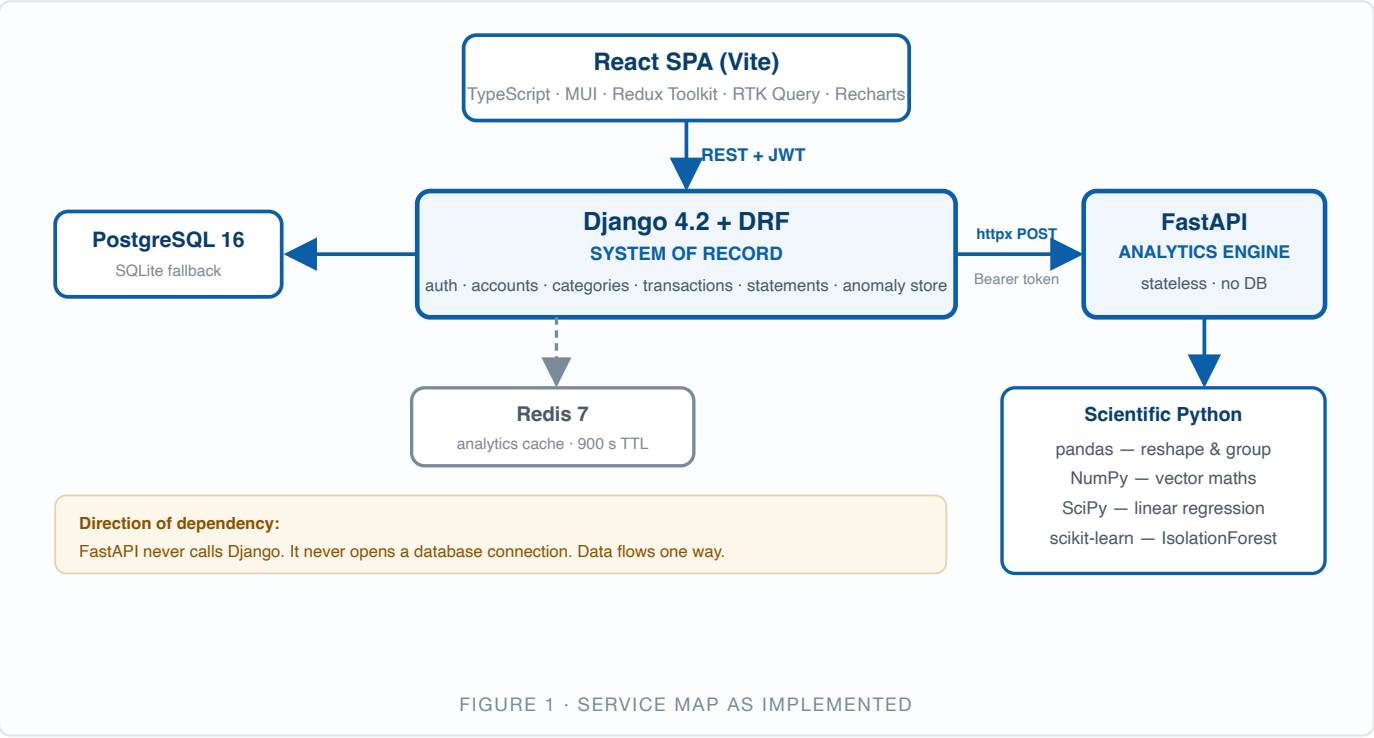


FIGURE 1 · SERVICE MAP AS IMPLEMENTED

PART 2 · CHAPTER I

Why These Technologies?

Every entry is installed and imported. Versions from `backend/requirements.txt`, `analytics-service/requirements.txt`, `frontend/package.json`.

2.1 Frontend

Tech	Role in FinSight	Why it fits	Alternatives & why not
React 18.3	The whole UI — 11 pages, 18 shared components, mounted in `src/main.tsx`	A dashboard is many small widgets over one dataset that must all update when a transaction is added. Component-level reactivity is the right model, and the ecosystem is deepest here	Vue/Svelte (thinner chart + grid ecosystem), Angular (over-opinionated at this size), Django templates (full page reload per filter)
TypeScript 5.6	All of `src/`; ~15 API shapes typed once in `types/index.ts`	A money app with `strict`, `noUnusedLocals`, `noUnusedParameters` on: a renamed	JSDoc (weaker cross-module inference); Zod would add runtime

		backend field is a compile error, not "₹NaN" on screen	validation the types don't give — a fair gap, noted in Part 33
Vite 5.4	Dev server :5173 with `/api → :8000` proxy; 4 manual vendor chunks	The proxy makes the API same-origin in dev, so CORS behaves like it does behind nginx in Docker	CRA (deprecated), Webpack (slower), Next.js (SSR is useless for a private authenticated dashboard)
Material UI 6.1	Every visual primitive; one theme in `theme.ts`	Fintech UI is table- and form-heavy. MUI ships accessible Table, TablePagination, Dialog, Snackbar — roughly the whole component budget	Tailwind + shadcn (rebuild the grid/dialog yourself), Ant Design (reads "enterprise admin")
Redux Toolkit 2.3 + RTK Query	2 hand-written slices (auth, ui) + ~28 endpoints across 6 files	Two different problems: auth tokens are true global state that must survive reload; everything else is *server cache*. RTK Query gives caching, dedup, loading flags and tag invalidation	TanStack Query + Zustand (arguably better fetching, but RTK Query ships with the store you already need); hand-written thunks (~84 reducers); Context alone (no cache, re-renders everything)
Recharts 2.13	3 charts: area trend, income-vs-expense, category donut	Declarative React components rather than an imperative canvas API	Chart.js (imperative, needs refs), D3 (far more power than 3 charts justify)
react-hook-form 7.53	Login, Register, Settings, Accounts, Categories, transaction dialog	Uncontrolled-first — typing an amount does not re-render the page	Formik (heavier, controlled by default)
notistack · dayjs · react-router-dom	Toasts (max 3, 4 s) · date formatting · nested layout routing	dayjs is 2 KB with a Moment-like API; router's nested routes mean the auth guard is written once	Moment (large, deprecated), date-fns (fine, less familiar API)

2.2 Django backend

Tech	Role	Why it fits	Alternatives & why not
Django 4.2.16 LTS	All financial state; 8 apps	~60% of this backend is work Django hands you: hashed-password user model, permissions, migrations, admin, security middleware, an ORM with real transaction control. Hand-writing that for a money app is how you ship a vulnerability	See Part 3 — this is *the* decision and gets its own chapter
DRF 3.15.2	Serializers, ViewSets, routers, permissions, filtering, pagination, throttling	One ModelViewSet + a router = 5 endpoints with pagination and filtering. `accounts/views.py` is 33 lines for the full CRUD API. Its global defaults are the security posture: a new view is locked down unless it opts out	Django Ninja (genuinely attractive, would unify validation style; DRF chosen for ecosystem maturity), GraphQL (solves over-fetching this app doesn't have)
SimpleJWT 5.3.1	30-min access, 7-day refresh, rotation on, signed with `JWT_SECRET_KEY`	The frontend is a separate origin (:5173 vs :8000). Sessions would need cross-site cookies and CSRF coordination; a bearer header sidesteps both and keeps the API stateless	Session cookies (cross-origin pain), opaque DB tokens (a lookup per request, no natural expiry), OAuth2 (overkill, no third-party clients)
django-filter 24.3	8 declarative filters in 33 lines	Query-param filtering without hand-parsing	Manual `request.GET` parsing per endpoint
drf-spectacular + sidecar	OpenAPI 3 at `/api/schema/`, Swagger at `/api/docs/`, Redoc at `/api/redoc/`	Sidecar self-hosts the UI assets, so docs work with no CDN and no external network call	drf-yasg (older OpenAPI 2), hand-written docs (drift immediately)
dj-database-url · psycopg[binary] 3.2	One `DATABASE_URL` → the whole DB config; psycopg *3*, not 2	Same image runs in every environment; psycopg3 is faster with better typing and a native async path	Six separate settings; psycopg2 (older, no async)
httpx 0.27	The client Django uses to call FastAPI	First-class timeout handling and an async path available for a future migration	`requests` (no async, weaker timeout ergonomics)
redis 5.0 · whitenoise 6.7 · gunicorn 23 · python-dotenv	Cache client · compressed static files from Gunicorn · 3 workers/60 s timeout · loads `backend/.env` then repo-root `.env`	WhiteNoise is what lets the admin and self-hosted Swagger assets work without a separate static server	A separate nginx static tier (unnecessary at this size)

2.3 Analytics service

Tech	Role	Why it fits	Alternatives & why not
FastAPI	6 endpoints in 90 lines — the schemas carry the weight	This service is a pure function: JSON in, JSON out, no DB, no session. Pydantic validation at the edge means a malformed payload is a 422 before pandas runs	Flask (no validation or schema generation), a second Django app (drags the ORM along), gRPC (protobuf tooling for a few calls per page), Lambda (cold-starting scikit-learn is unpleasant)
Pydantic 2 + pydantic-settings	`schemas/analytics.py` is the entire Django↔FastAPI contract: 4 enums, 4 requests, 8 responses. `config.py` reads every knob from env with typed defaults	The contract *is* the architecture. Because `amount` is `float = Field(gt=0)`, bad data can never reach the maths. Pydantic 2's Rust core validates 5,000 transactions cheaply — there is a test that does exactly that	dataclasses + manual checks, marshmallow, attrs — none generate OpenAPI, and manual validation on a security boundary is the code you least want to hand-maintain
pandas ≥2.1	One `to_dataframe()` per request; every service then works in `groupby` / `agg` terms	"Total per category, per month, with count and mean" is one line of pandas and fifteen of dict juggling. Category-grouped statistics — the core idea of the detector — is literally `df.groupby("category")`	Pure dicts (slower, noisier), Polars (faster but thinner sklearn interop), SQL aggregation (would require giving FastAPI a DB connection, destroying statelessness)
NumPy ≥1.26	All of `analytics/statistics.py`: mean, median, std, percentile, NaN/Inf guards, vectorised Z-score and MAD	Vectorised expressions read like the mathematical definition, which is what you want in code you must audit	Hand-rolled statistics — every one a place to introduce a subtle bug
SciPy ≥1.11	Exactly one function: `stats.linregress` in `linear_trend()`	Numerically stabler than a hand-rolled least-squares fit, and the discarded `rvalue` / `pvalue` are one line from enabling "and the trend is statistically significant"	Three lines of NumPy — a fair criticism of a large dependency; see Part 20
scikit-learn ≥1.4	`IsolationForest` (200 trees), `StandardScaler`, `OrdinalEncoder`	The reason the analytics service is Python at all. A Node equivalent would reimplement percentiles, MAD and an isolation forest by hand	PyOD (wider catalogue, unneeded), TensorFlow/PyTorch (an autoencoder on a few thousand rows would overfit at far greater cost)

2.4 Data & infrastructure

Tech	Role	Why it fits	Alternatives & why not
PostgreSQL 16	Five tables; used whenever `DATABASE_URL` is set (always in Docker)	Three properties this app depends on: real `NUMERIC(14,2)` for money, row locking via `SELECT … FOR UPDATE` (what `select_for_update()` compiles to), and enforced multi-column unique constraints	MySQL (workable; Postgres has stronger constraints and Django support), MongoDB (wrong shape — this data is relational and needs ACID on balance updates)
SQLite	Automatic when `DATABASE_URL` is empty	Clone → install → migrate → runserver, no infrastructure. Also how the 43 tests run fast	Caveat: it ignores `select_for_update()`, so the concurrency guarantee is only truly exercised on Postgres. Part 11
Redis 7	Django's `CACHES["default"]` when `REDIS_URL` is set	A dashboard load re-runs detection over 180 days; caching turns a repeat view into one Redis GET	LocMemCache *is* the fallback, but is per-worker — three Gunicorn workers would keep three inconsistent caches. Memcached lacks pattern-delete
Docker Compose · nginx 1.27	5 services, 1 bridge network, 2 volumes, health checks · serves the React bundle and proxies `/api/` to `django:8000`	`docker compose up --build` is the whole onboarding story. Because nginx proxies the API, the browser sees one origin in Docker, so CORS is irrelevant there	Manual multi-service setup (nobody will do it), Kubernetes (overkill for local dev)

PART 3 · CHAPTER I

Why Django *and* FastAPI?

The defining decision of the project, and the one an interviewer will press hardest on. The short answer: **the two frameworks own two genuinely different problems.** Django owns state that must be correct forever. FastAPI owns computation that must be fast and replaceable.

3.1 The split, stated once

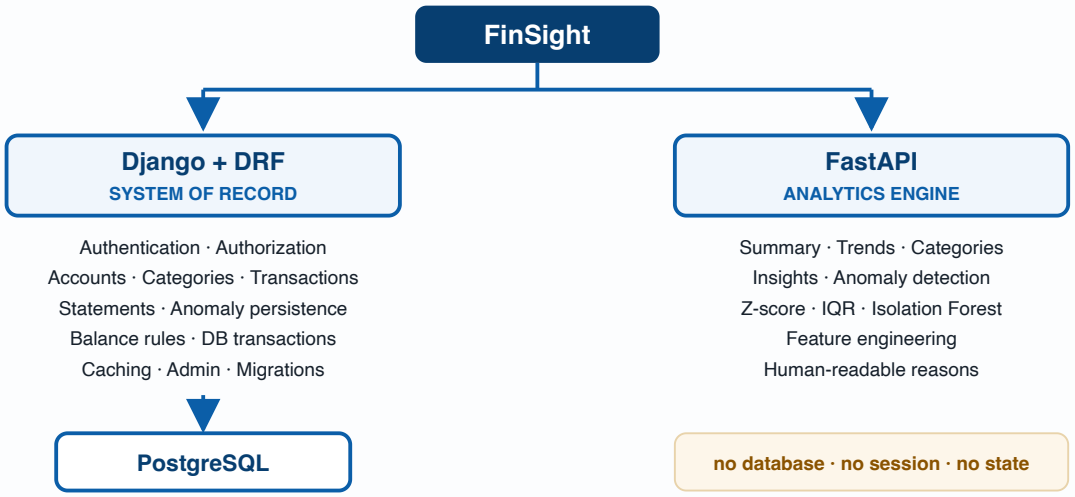


FIGURE 2 · RESPONSIBILITY SPLIT AS IMPLEMENTED

3.2 What Django does, and why it is right for it

Responsibility	Where	What Django gives you for free
Authentication	`users/` — custom email user, SimpleJWT	PBKDF2-SHA256 hashing with per-user salts, timing-attack mitigation, 4 password validators, `AbstractBaseUser`. Writing this yourself for a money app is how CVEs happen
Authorization	`common/permissions.py` + every `get_queryset()`	Two-layer isolation: the queryset is filtered by `request.user` *and* object permissions re-check ownership. A DRF permission class is 5 lines
Data model	5 models across 5 apps	Declarative fields → migrations → SQL. `UniqueConstraint`, `Index`, `on_delete` semantics, `DecimalField` → `NUMERIC(14,2)`
CRUD API	3 `ModelViewSet`s + routers	`accounts/views.py` is 33 lines and yields list/create/retrieve/update/destroy with pagination, filtering, search and ordering
Business rules	`transactions/services.py`, `statements/services.py`	`transaction.atomic()` and `select_for_update()` — real database transaction control, which is the entire reason balances stay correct
Anomaly persistence	`anomalies/`	The review workflow (OPEN/REVIEWED/IGNORED) is state, and state belongs here — not in the engine that computed it
Operations	`admin.py` × 5, `seed_demo`, migrations	A working back-office and a repeatable schema history, at effectively zero cost

The ORM, concretely

```
# backend/transactions/views.py — isolation + N+1 prevention in one expression
Transaction.objects.filter(account__user=self.request.user).select_related("account", "category")
```

`account__user` traverses a foreign key in the filter; `select_related` turns what would be 1 + 2N queries into a single JOIN. Both are one-liners, both are the difference between a correct fast API and a slow leaky one.

3.3 What FastAPI does, and why it is right for it

Property	Why it matters here
Pydantic at the edge	`schemas/analytics.py` is the contract. `amount: float = Field(gt=0)` means invalid data becomes a 422 *before* pandas runs. DRF serializers could do this too, but Pydantic v2's Rust core validates 5,000 transactions cheaply — there is a test that does exactly that
Native scientific stack	pandas, NumPy, SciPy and scikit-learn are Python-first. This is the reason the analytics service is Python at all
Stateless by construction	No DB driver in `requirements.txt`, no models, no sessions. Scaling is "add replicas"
Free OpenAPI	Swagger at `/docs` from the type hints — the contract documents itself
Small surface	6 endpoints in ~90 lines. Routes are HTTP plumbing; every route is `def`, not `async def`, so Starlette runs CPU-bound pandas work in a threadpool instead of blocking the event loop

3.4 Why not Django alone?

Perfectly possible — you would `pip install pandas scikit-learn` and add an `analytics` app. Here is what you would actually be buying:

Problem	Concrete consequence in this codebase
Coupled scaling	An Isolation Forest fit is CPU-bound and can take hundreds of milliseconds. In Gunicorn's sync worker model that worker serves nothing else meanwhile. Under load, *logins* queue behind ML. Today Django runs 3 light workers and FastAPI scales separately
Dependency weight	pandas + NumPy + SciPy + scikit-learn is ~200 MB. Every Django replica would carry it. The Django image currently does not
Blast radius	A scikit-learn upgrade that changes `score_samples` behaviour would force a redeploy of the service that holds the money
Blurred boundaries	With an ORM in scope, "just read the transactions directly in the detector" is irresistible. Within months the ML code has ORM calls inside it and is untestable without a database. The physical split makes that impossible
Slower experimentation	Swapping the detector today touches one file behind one contract, verified by 27 tests with no DB. In-process it means booting Django and fixtures
No graceful degradation	In-process analytics failure raises inside the request. The current design catches `AnalyticsUnavailable` and still returns a working dashboard — there is a test asserting this

> **THE HONEST COUNTER-ARGUMENT**
>
> For a single-user portfolio app, Django alone would be less code, less deployment, and no network hop. The split is justified by *where the app is heading* — heavier models, independent scaling, faster iteration on the detector — not by today's load. Say that plainly in an interview; claiming current necessity is not credible.

3.5 Why not FastAPI alone?

You would rebuild, by hand, the things Django ships:

Need	Django gives	FastAPI-only cost
ORM & migrations	Built in, `makemigrations`	SQLAlchemy + Alembic, wired by hand
Auth	`AbstractBaseUser`, hashing, validators	Hand-roll passlib + a user table — on a money app
Permissions	Declarative classes, global default	A dependency per route; one omission is a data leak
Admin	Free, 5 files of config	Build a back-office or use raw SQL
CRUD	`ModelViewSet` + router = 5 endpoints	Five handlers × three resources, by hand
Pagination / filtering / throttling	Global DRF settings	Hand-written per endpoint

Roughly 60% of the backend would become code you own and must secure. For a financial system of record that is a bad trade — and none of it would make the analytics any better.

3.6 Why the seam is where it is

> **THE TEST THAT PROVES THE BOUNDARY IS REAL**
>
> `test_dashboard_survives_analytics_outage` in `backend/analytics/tests.py` patches the HTTP client to raise `ConnectError`, then asserts the dashboard still returns **200** with correct local numbers and `analytics_available: false`.
>
> That test only makes sense because the split follows a real fault line: **the ledger is authoritative, analytics are derived.** Derived data can be temporarily missing without the product being broken. If the split had been drawn somewhere arbitrary — say, "categories live in FastAPI" — no such graceful path would exist.

Three rules keep the seam clean, all visible in the code:

1. **FastAPI never calls Django and never opens a database connection.** Dependency flows one way.
2. **The payload carries names, not primary keys** — `"category": "Food"`, not `17` (`analytics/services.py` → `serialize_for_engine`). The engine knows nothing of Django's schema.
3. **Django validates everything it gets back.** `detect_and_store` builds `valid_txn_ids` and drops any finding referencing an unknown id — a compromised engine cannot write an anomaly onto someone else's transaction.

PART 4 · CHAPTER I

Project Structure

Three deployable units at the repository root, each with its own dependency file, Dockerfile and `.env.example`.

```
FinSight/
├── backend/              Django + DRF        :8000   ~3,300 LOC Python
├── analytics-service/    FastAPI             :9000   ~1,360 LOC Python
├── frontend/             React + Vite        :5173   ~4,760 LOC TypeScript
├── docker-compose.yml     5 services, 1 network, 2 volumes
├── Makefile               install / migrate / seed / run / test / up
├── .env.example           every variable the stack reads
├── README.md              quick starts, demo credentials
└── docs/ARCHITECTURE.md   the design document this PDF expands on
```

4.1 backend/ — eight Django apps

```
backend/
├── config/               project package (not an app)
│   ├── settings.py       294 lines — env-driven; DATABASE_URL → Postgres else SQLite
│   ├── urls.py           /admin/ · /health/ · /api/schema|docs|redoc/ · /api/…
│   ├── wsgi.py asgi.py   Gunicorn / ASGI entry points
├── common/               shared infrastructure, no models of its own
│   ├── models.py         TimeStampedModel — abstract created_at/updated_at
│   ├── pagination.py     StandardResultsPagination (20, max 200)
│   ├── permissions.py    IsOwner · IsOwnerOrReadOnlyDefault
│   ├── responses.py      EnvelopeJSONRenderer + envelope_exception_handler
│   └── management/commands/seed_demo.py   ~171 lines, 6 months of demo data
├── users/                custom email User, managers, JWT views       (5 endpoints)
├── accounts/             Account model + ModelViewSet                 (5 endpoints)
├── categories/           Category + defaults + post_save signal       (6 endpoints)
├── transactions/         Transaction + services.py + filters.py       (5 endpoints)
├── statements/           no models — computed from transactions       (2 endpoints)
├── analytics/            no models — client.py + services.py + views  (6 endpoints)
├── anomalies/            Anomaly model + detect/review/ignore actions (7 endpoints)
├── requirements.txt      14 pinned packages
├── Dockerfile            python:3.12-slim · gunicorn 3 workers
└── db.sqlite3            local dev database (git-ignored)
```

File	Why it exists · what calls it
`config/settings.py`	Single source of configuration. Chooses the DB from `DATABASE_URL`, sets DRF's global defaults (JWT auth, `IsAuthenticated`, pagination, envelope renderer, throttle rates), JWT lifetimes, CORS, security headers, Redis-or-locmem cache, and structured logging. Read once at startup
`common/responses.py`	Every API response has the same four keys. Registered globally as `DEFAULT_RENDERER_CLASSES` and `EXCEPTION_HANDLER`, so no view ever formats a response itself

`common/permissions.py`	`IsOwner` handles both a direct `user` FK and the `account.user` chain used by transactions. `IsOwnerOrReadOnlyDefault` lets users read but not modify the 19 seeded default categories
`transactions/services.py`	The financial core: three functions, each `@db_transaction.atomic` with `select_for_update()`. Called from the serializer's `create` / `update`, the viewset's `perform_destroy`, and `seed_demo`
`analytics/client.py`	The only place httpx is used. Collapses timeout / transport error / 5xx / 4xx / bad-JSON into one `AnalyticsUnavailable`
`analytics/services.py`	Two jobs: local dashboard computation (pure Django ORM) and cached proxying to FastAPI. The seam where Celery would later slot in
`anomalies/services.py`	`detect_and_store` — calls the engine, validates ownership, upserts on `(user, transaction, method)` without clobbering review state, prunes stale OPEN findings
`categories/signals.py`	`post_save` on User → `Category.create_defaults_for(user)`. Why a new user has 19 categories before they do anything

4.2 analytics-service/ — layered by role

```
analytics-service/
├── app/
│   ├── main.py                FastAPI app · CORS · 2 exception handlers · /health
│   ├── config.py              pydantic-settings — thresholds, token, log level
│   ├── security.py            require_service_token (hmac.compare_digest)
│   ├── schemas/analytics.py   THE CONTRACT — 4 enums, 4 requests, 8 responses
│   ├── api/analytics.py       6 routes; router-level token dependency; timing logs
│   ├── services/              orchestration — one module per analytics concern
│   │   ├── spending_service.py   build_summary
│   │   ├── trend_service.py      build_trends   (daily / weekly / monthly)
│   │   ├── category_service.py   build_categories
│   │   ├── anomaly_service.py    detect_anomalies — the 3 detectors + merge
│   │   └── insight_service.py    build_insights — numbers → English
│   ├── analytics/statistics.py pure maths — describe, zscores, modified_zscores,
│   │                          iqr_bounds, linear_trend, pct_growth, severity_*, safe_float
│   ├── ml/isolation_forest.py 8 features → OrdinalEncoder → StandardScaler → 200 trees
│   └── utils/preprocessing.py to_dataframe · expenses_only · income_only · month_span
├── tests/                     27 tests, 5 files, no database required
├── requirements.txt           fastapi, uvicorn, pydantic, pandas, numpy, scipy, sklearn
└── Dockerfile                 uvicorn --workers 2
```

The layering is strict and worth naming: **routes never compute, services never do maths, `statistics.py` never touches pandas.** That is why `test_per_category_math` can pass three dictionaries and assert five numbers with no fixtures.

4.3 frontend/ — grouped by role

```
frontend/src/
├── main.tsx              Redux Provider · ThemeProvider · SnackbarProvider · BrowserRouter
├── App.tsx               all routes + RequireAuth / RedirectIfAuthed guards
├── theme.ts              one MUI theme — palette, radii, typography
├── types/index.ts        ~15 interfaces mirroring every API shape
├── store/
│   ├── index.ts          configureStore + api middleware
│   ├── authSlice.ts      access · refresh · user, persisted to localStorage
│   └── uiSlice.ts        sidebar open/closed
├── services/             the API layer — one file per resource
│   ├── api.ts            baseQuery: auth header, envelope unwrap, 401 refresh + mutex
│   ├── mutex.ts          hand-written 25-line mutex (no dependency)
│   ├── authApi · accountsApi · categoriesApi · transactionsApi
│   ├── statementsApi · analyticsApi · anomaliesApi
├── pages/                11 route components
├── components/
│   ├── Layout/           DashboardLayout · SidebarContent · navItems
│   ├── common/           StatCard · SectionCard · PageHeader · SeverityChip ·
│   │                     TypeChip · ConfirmDialog · states.tsx (Loading/Error/Empty)
│   ├── charts/           SpendingTrendChart · IncomeExpenseChart · CategoryDonut
│   ├── dashboard/        AnomalyAlerts · CategoryBreakdown · TopExpenses ·
│   │                     RecentTransactionsTable
│   └── transactions/     TransactionFormDialog · TransactionFilters
├── hooks/index.ts        typed useAppDispatch / useAppSelector / useIsAuthenticated
└── utils/                format.ts (currency, dates, errorMessage) · constants.ts
```

4.4 How a feature crosses all three

```
"Add a transaction"

  frontend  components/transactions/TransactionFormDialog.tsx   form + validation
            services/transactionsApi.ts                        POST + tag invalidation
            services/api.ts                                    auth header, envelope
      ↓
  backend   transactions/urls.py        router → viewset
            transactions/views.py       queryset scoping, permissions
            transactions/serializers.py validation, FK narrowing
            transactions/services.py    atomic + row lock + balance
            transactions/models.py      the table
      ↓
  database  INSERT transaction · UPDATE account.balance   (one transaction)
      ↓
  frontend  invalidatesTags → Dashboard/Analytics/Anomaly queries refetch
```

PART 5 · CHAPTER II

Frontend Architecture

5.1 Startup — `src/main.tsx`

```
ReactDOM.createRoot(document.getElementById("root")!).render(
  <React.StrictMode>
    <Provider store={store}>                        {/* ① Redux — auth, ui, RTK Query cache */}
      <ThemeProvider theme={theme}>                 {/* ② one MUI theme for the app          */}
        <CssBaseline />                             {/* ③ CSS reset                          */}
        <SnackbarProvider maxSnack={3} autoHideDuration={4000}
                          anchorOrigin={{ vertical: "bottom", horizontal: "right" }}>
          <BrowserRouter>                           {/* ④ history-based routing              */}
            <App />
          </BrowserRouter>
        </SnackbarProvider>
      </ThemeProvider>
    </Provider>
  </React.StrictMode>,
);
```

Provider order is deliberate: Redux outermost because everything below may read state; router innermost because only components need it. `maxSnack={3}` prevents a toast avalanche when a mutation invalidates several queries at once.

5.2 Routing and the auth guard — `src/App.tsx`

```
function RequireAuth() {
  const isAuthed = useIsAuthenticated();
  const location = useLocation();
  if (!isAuthed) return <Navigate to="/login" replace state={{ from: location }} />;
  return <DashboardLayout />;          // renders <Outlet /> for the matched child
}

function RedirectIfAuthed({ children }: { children: React.ReactNode }) {
  return useIsAuthenticated() ? <Navigate to="/" replace /> : <>{children}</>;
}

<Routes>
  <Route path="/login"    element={<RedirectIfAuthed><LoginPage /></RedirectIfAuthed>} />
  <Route path="/register" element={<RedirectIfAuthed><RegisterPage /></RedirectIfAuthed>} />
  <Route element={<RequireAuth />}>            {/* layout route — no path of its own */}
    <Route path="/"             element={<DashboardPage />} />
    <Route path="/transactions" element={<TransactionsPage />} />
    <Route path="/accounts"     element={<AccountsPage />} />
    <Route path="/categories"   element={<CategoriesPage />} />
    <Route path="/statements"   element={<StatementsPage />} />
    <Route path="/analytics"    element={<AnalyticsPage />} />
    <Route path="/anomalies"    element={<AnomaliesPage />} />
    <Route path="/settings"     element={<SettingsPage />} />
  </Route>
  <Route path="*" element={<NotFoundPage />} />
</Routes>
```

ONE GUARD, EIGHT PROTECTED ROUTES

Because `RequireAuth` is a *layout route*, adding a page means adding one line inside it — it cannot be forgotten. `state={{ from: location }}` records where the user was headed. This is a **UX** guard, not a security boundary: every endpoint independently requires a valid JWT, so bypassing it in devtools reveals only empty shells.

5.3 The API layer — `src/services/api.ts`

One `baseQuery` does four jobs for every request in the app.

```
const rawBaseQuery = fetchBaseQuery({
  baseUrl: import.meta.env.VITE_API_BASE_URL ?? "/api",
  prepareHeaders: (headers, { getState }) => {
    const token = (getState() as RootState).auth.access;      // ① attach the JWT
    if (token) headers.set("Authorization", `Bearer ${token}`);
    return headers;
  },
```

```
});

const unwrap = (result) => {                                    // ② strip the envelope
  const env = result.data as ApiEnvelope<unknown>;
  if (env && typeof env === "object" && "success" in env) {
    if (env.success) return { ...result, data: env.data };
    return { error: { status: 400, data: env } };
  }
  return result;
};

export const baseQueryWithReauth: BaseQueryFn = async (args, api, extra) => {
  await mutex.waitForUnlock();                                  // ③ queue during refresh
  let result = unwrap(await rawBaseQuery(args, api, extra));

  if (result.error?.status === 401) {                           // ④ transparent refresh
    if (!mutex.isLocked()) {
      const release = await mutex.acquire();
      try {
        const refresh = (api.getState() as RootState).auth.refresh;
        const refreshed = refresh
          ? await rawBaseQuery({ url: "/auth/refresh/", method: "POST", body: { refresh } }, api, extra)
          : null;
        const data = (refreshed?.data as ApiEnvelope<{ access: string }> | undefined)?.data;
        if (data?.access) {
          api.dispatch(tokenRefreshed(data.access));
          result = unwrap(await rawBaseQuery(args, api, extra));   // replay the original
        } else {
          api.dispatch(loggedOut());
        }
      } finally { release(); }
    } else {
      await mutex.waitForUnlock();                              // someone else refreshed
      result = unwrap(await rawBaseQuery(args, api, extra));
    }
  }
  return result;
};
```

WHY THE MUTEX EXISTS

The dashboard fires ~4 queries at once. If the access token has expired, all four 401 simultaneously. Without the lock, four refresh calls race — and since `ROTATE_REFRESH_TOKENS=True`, each invalidates the previous, so three of the four fail and the user is logged out at random. The mutex (`services/mutex.ts`, 25 hand-written lines, no dependency) makes exactly one request refresh while the others wait and then replay.

5.4 Endpoint definitions and the tag graph

```
// services/transactionsApi.ts
createTransaction: builder.mutation<Transaction, TransactionPayload>({
  query: (body) => ({ url: "/transactions/", method: "POST", body }),
  invalidatesTags: ["Transaction", "Account", "Dashboard", "Analytics", "Anomaly"],
}),
```

Tag	Provided by	Invalidated by
`Transaction`	list, detail	create / update / delete
`Account`	accounts list	account CRUD **and every transaction mutation** (balances changed)
`Category`	categories list	category CRUD
`Dashboard`	`/dashboard/`	any transaction or account mutation
`Analytics`	summary, trends, categories, analyze	any transaction mutation
`Anomaly`	anomalies list, stats	detect, review, ignore, reopen, transaction mutations
`Statement`	`/statements/`	transaction mutations
`User`	`/auth/me/`	profile update

This is the whole cache-coherence strategy: one mutation declares what it invalidates, and every mounted query holding those tags refetches. No manual "refresh the dashboard" call exists anywhere.

5.5 The layout

`components/Layout/DashboardLayout.tsx` renders an AppBar, a 248 px drawer (permanent ≥ md, temporary below, toggled through `uiSlice`), and `<Outlet />`. `navItems.ts` is the single source of navigation — 8 entries of `{label, to, icon}`. Adding a page means one route line and one nav entry.

PART 6 · CHAPTER II

UI Pages

Eleven route components. Every one documented below exists in `frontend/src/pages/`.

Page	Route	Data	Behaviour
LoginPage	`/login`	`useLoginMutation` → `POST /auth/login/`	AuthShell wrapper; react-hook-form (email, password); on success dispatches `credentialsReceived`, persists to localStorage, toasts "Welcome back, {name}", navigates `/`. Errors render inline in an `<Alert>`. Demo credentials shown on the card
RegisterPage	`/register`	`useRegisterMutation`	name, email, password, confirm. Client rule: passwords must match, min 8. Backend returns user + both tokens, so registration logs you straight in
DashboardPage	`/`	`useGetDashboardQuery()` → `GET /api/dashboard/`	The densest page: 4 StatCards (balance, income, expenses, savings + rate), SpendingTrendChart (6 months), IncomeExpenseChart, CategoryBreakdown, TopExpenses, RecentTransactionsTable, AnomalyAlerts. Everything except the alerts is computed in Django, so it renders fully during an analytics outage
TransactionsPage	`/transactions`	list + create/update/delete mutations; accounts and categories for the form	Server-side pagination (`TablePagination`), 8 filters via `TransactionFilters`, sortable columns, row actions. `ConfirmDialog` before delete. `isFetching` dims the table so the page never jumps
AccountsPage	`/accounts`	accounts CRUD	Card grid; create dialog takes `opening_balance` (write-only, create-only). Delete is blocked server-side when transactions exist — the 400 surfaces as a toast telling you to deactivate instead
CategoriesPage	`/categories`	categories CRUD	Split into Income and Expense sections. Default categories show a "Default" chip and cannot be edited or deleted (enforced by `IsOwnerOrReadOnlyDefault`). Delete offers reassignment via `?reassign_to=`
StatementsPage	`/statements`	`GET /statements/`; CSV via a direct `fetch`	Year/month/account selectors. Shows opening balance, income, expenses, net, closing balance, per-category totals and the transaction list. CSV download bypasses RTK Query because it is a file, not JSON — it attaches the token by hand and uses a Blob URL
AnalyticsPage	`/analytics`	`GET /api/analytics/analyze/` — one FastAPI round trip	Range selector (3/6/12 months). StatCards, trend chart with direction icon and MoM %, category donut + table, insights list, anomaly cards. On 503 shows an amber warning with "Your transactions are safe." and a Retry — this page legitimately depends on the engine
AnomaliesPage	`/anomalies`	`useGetStatementQuery`-style list + detect / review / ignore / reopen	Filters for severity, status, detection method, category and date range; table with SeverityChip, method label, amount, the engine's `reason` sentence, and status actions. "Run detection" POSTs `/anomalies/detect/` and toasts the count
SettingsPage	`/settings`	`useGetMeQuery`, `useUpdateMeMutation`, `useChangePasswordMutation`	Profile (name, currency, default account, two notification toggles) and a separate password form requiring the current password. PARTIAL — the notification flags persist but nothing sends notifications
NotFoundPage	`*`	—	404 with a link home
AuthShell	—	—	Not a route: the centred branded card shared by Login and Register

The shared state components

```
// components/common/states.tsx — used by every page, identically
<LoadingState />                          // centred spinner
<ErrorState message={...} onRetry={refetch} />   // always offers a retry
<EmptyState title=... description=... action={...} />   // first-run guidance
```

Every page follows the same three-branch pattern, which is why loading and error behaviour is consistent across the app:

```
if (isLoading) return <LoadingState />;
if (error || !data) return <ErrorState message={errorMessage(error)} onRetry={refetch} />;
// …render
```

PART 7 · CHAPTER II

Frontend Request Flow

"Add Transaction", traced through the real code.

```
USER      clicks Add transaction → dialog opens
   ↓
FORM      TransactionFormDialog.tsx — react-hook-form
          rules: amount required & > 0 · account required · category required · date required
          the category dropdown is already filtered to the chosen transaction_type
   ↓
SUBMIT    onSubmit(values) → createTxn(payload).unwrap()
          payload = {transaction_type, amount, account, category,
                     merchant, description, transaction_date}
   ↓
RTK QUERY transactionsApi.createTransaction → baseQueryWithReauth
          await mutex.waitForUnlock()
          prepareHeaders → Authorization: Bearer <access>
   ↓
HTTP      POST /api/transactions/            (Vite proxy in dev, nginx in Docker)
   ↓
DJANGO    JWTAuthentication → IsAuthenticated → TransactionViewSet.create
          TransactionSerializer: FK querysets narrowed to the user's own rows
                                 amount > 0 · date ≤ tomorrow · category.type == type
          services.create_transaction(**validated_data)
            @atomic: SELECT … FOR UPDATE → INSERT → balance += signed_amount → COMMIT
   ↓
RESPONSE  201 {"success": true, "data": {…, "account_name": "HDFC Savings",
                                         "category_name": "Food", "currency": "INR"}}
   ↓
UNWRAP    baseQuery sees "success" → returns env.data to the component
   ↓
REACT     .unwrap() resolves → toast "Transaction added successfully." → dialog closes
          invalidatesTags: Transaction · Account · Dashboard · Analytics · Anomaly
          → mounted queries refetch → table repaints, account card shows the new balance

ERROR PATH  400 → errorMessage(err) reads data.message
            → "An EXPENSE transaction must use an EXPENSE category (got a INCOME category)."
            → error toast, dialog stays open, nothing was written
```

The four validation layers, and why each is separate

Layer	Checks	Purpose
react-hook-form	required, positive amount	Instant feedback; no round trip
TypeScript	payload shape	Compile-time — a renamed field breaks the build
DRF serializer	ownership, amount, date, type↔category agreement	**The authoritative layer.** The client cannot be trusted
Database	NOT NULL, FK, unique, `MinValueValidator`	Last line of defence, including against the ORM being bypassed

PART 30 · CHAPTER II

State Management

The key insight: FinSight has two kinds of state, and only one of them belongs in a hand-written reducer.

Kind	Examples	Managed by
Client state	JWTs, current user, sidebar open	Hand-written slices — `authSlice.ts`, `uiSlice.ts`
Server cache	Transactions, accounts, dashboard, anomalies	RTK Query — *never* hand-written

30.1 The store

```
export const store = configureStore({
  reducer: {
    auth: authReducer,
    ui: uiReducer,
    [api.reducerPath]: api.reducer,          // RTK Query's cache lives here
  },
  middleware: (getDefault) => getDefault().concat(api.middleware),  // required for caching
});
export type RootState = ReturnType<typeof store.getState>;
```

30.2 authSlice — the only truly global state

```
const STORAGE_KEY = "finsight.auth";

function loadInitialState(): AuthState {
  try {
    const raw = localStorage.getItem(STORAGE_KEY);
    if (raw) {
      const parsed = JSON.parse(raw);
      return { access: parsed.access ?? null, refresh: parsed.refresh ?? null,
               user: parsed.user ?? null };
    }
  } catch { /* corrupt storage → start signed out */ }
  return { access: null, refresh: null, user: null };
}

const authSlice = createSlice({
  name: "auth",
  initialState: loadInitialState(),          // ← rehydrated before the first render
  reducers: {
    credentialsReceived(state, { payload }) {
      state.access = payload.access; state.refresh = payload.refresh;
      state.user = payload.user ?? state.user;
      persist(state);
    },
    tokenRefreshed(state, { payload }) { state.access = payload; persist(state); },
    userUpdated(state, { payload })   { state.user = payload;    persist(state); },
    loggedOut(state) {
      state.access = null; state.refresh = null; state.user = null;
      localStorage.removeItem(STORAGE_KEY);
    },
  },
});
```

Four actions, and each corresponds to a real event: login/register, silent refresh, profile save, logout. Rehydrating in `initialState` — not in a `useEffect` — is what prevents a flash of the login page on reload. Immer inside Redux Toolkit is why these mutations are safe.

> **THE LOCALSTORAGE TRADE-OFF, STATED HONESTLY**
>
> Tokens in `localStorage` are readable by any JavaScript on the page, so a successful XSS can exfiltrate them. The more secure design is an httpOnly, SameSite refresh cookie with the access token in memory only. What limits the damage here: a 30-minute access lifetime, refresh rotation, React escaping by default, and no `dangerouslySetInnerHTML` anywhere in `src/`. This is the top item in Part 32's improvements list.

30.3 Why RTK Query replaced ~84 hand-written reducers

Without it	With it
3 actions × 7 resources × 4 operations ≈ 84 reducers	~28 endpoint definitions, ~6 lines each
An `isLoading` and `error` flag per resource, by hand	`isLoading`, `isFetching`, `error`, `refetch` supplied
Two components mounting the same list = two requests	Deduplicated automatically
Manually refreshing the dashboard after every mutation	`invalidatesTags` — declared once
Refetch on every navigation	Cached; `keepUnusedDataFor` 60 s

The generated hooks are fully typed end to end, so `data.savings_rate` autocompletes and a backend field rename becomes a compile error rather than `undefined` on screen.

PART 8 · CHAPTER III

Authentication & Authorization

```
USER → password → Django → PBKDF2 verify → JWT pair → React → localStorage
                                                        ↓
                                  every later request: Authorization: Bearer <access>
                                                        ↓
                                  JWTAuthentication → request.user → IsAuthenticated → IsOwner
```

8.1 The custom user model — `users/models.py`

```python
class User(AbstractBaseUser, PermissionsMixin):
    """FinSight user. Email is the unique login identifier."""
    name  = models.CharField(max_length=150)
    email = models.EmailField(unique=True)
    is_active = models.BooleanField(default=True)
    is_staff  = models.BooleanField(default=False)

    currency = models.CharField(max_length=3, default="INR")
    default_account = models.ForeignKey("accounts.Account", null=True, blank=True,
                                        on_delete=models.SET_NULL, related_name="+")
    notify_anomalies      = models.BooleanField(default=True)
    notify_weekly_summary = models.BooleanField(default=False)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    objects = UserManager()
    USERNAME_FIELD  = "email"        # ← log in with email, not a username
    REQUIRED_FIELDS = ["name"]
```

`AbstractBaseUser` supplies `password`, `last_login`, `set_password()` and `check_password()`; `PermissionsMixin` supplies `is_superuser`, groups and permissions — which is what makes the admin work. `related_name="+"` on `default_account` suppresses the reverse accessor: without it, `Account` would gain a `user_set` that collides confusingly with its own `user` FK.

Choosing a custom user model on day one is the single most important Django decision in this project. Swapping `AUTH_USER_MODEL` after migrations exist is famously painful.

The manager

```python
def _create_user(self, email, password, **extra):
    if not email:
        raise ValueError("Users must have an email address")
    email = self.normalize_email(email).lower()      # ← case-insensitive identity
    user = self.model(email=email, **extra)
    user.set_password(password)                      # ← NEVER assign user.password directly
    user.save(using=self._db)
    return user
```

`set_password()` is what turns the plaintext into `pbkdf2_sha256$870000$<salt>$<hash>`. Django 4.2 defaults to **PBKDF2-SHA256 with 870,000 iterations** and a per-user random salt — deliberately slow, so brute-forcing a stolen database is expensive, and salted so a rainbow table is useless.

8.2 Registration

```python
class RegisterSerializer(serializers.ModelSerializer):
    password = serializers.CharField(write_only=True, min_length=8,
                                     style={"input_type": "password"})

    def validate_email(self, value):
        value = value.lower().strip()
        if User.objects.filter(email__iexact=value).exists():
            raise serializers.ValidationError("An account with this email already exists.")
        return value

    def validate_password(self, value):
        validate_password(value)        # runs all 4 AUTH_PASSWORD_VALIDATORS
        return value

    def create(self, validated_data):
        return User.objects.create_user(**validated_data)   # goes through set_password
```

`write_only=True` means the password can be written but never appears in any response — including the 201 that immediately follows. The four validators configured in settings reject passwords under 8 characters, too similar to the user's own attributes, in Django's list of ~20,000 common passwords, or entirely numeric.

`RegisterView` returns **201 with the user plus both tokens**, so signing up logs you in — one round trip instead of two.

8.3 Login and the JWT

```python
class FinSightTokenObtainPairSerializer(TokenObtainPairSerializer):
    """Adds the user profile to the login response."""
    @classmethod
    def get_token(cls, user):
        token = super().get_token(user)
        token["name"]  = user.name       # extra claims, readable by the client
        token["email"] = user.email
        return token

    def validate(self, attrs):
        data = super().validate(attrs)   # authenticate() + build the token pair
        data["user"] = UserSerializer(self.user).data
        return data
```

```python
# config/settings.py
SIMPLE_JWT = {
    "ACCESS_TOKEN_LIFETIME":  timedelta(minutes=int(os.getenv("JWT_ACCESS_TOKEN_LIFETIME_MIN", "30"))),
    "REFRESH_TOKEN_LIFETIME": timedelta(days=int(os.getenv("JWT_REFRESH_TOKEN_LIFETIME_DAYS", "7"))),
    "ROTATE_REFRESH_TOKENS": True,
    "BLACKLIST_AFTER_ROTATION": False,
    "SIGNING_KEY": os.getenv("JWT_SECRET_KEY", SECRET_KEY),
    "AUTH_HEADER_TYPES": ("Bearer",),
    "USER_ID_FIELD": "id", "USER_ID_CLAIM": "user_id",
}
```

Setting	Reasoning
30-minute access	Short enough that a stolen token expires quickly; the silent refresh makes the shortness invisible
7-day refresh	A week of "stay signed in" without re-entering a password
`ROTATE_REFRESH_TOKENS`	Each refresh issues a *new* refresh token — this is the reason the frontend needs a mutex (Part 5.3)
Separate `JWT_SECRET_KEY`	Token signing is isolated from `SECRET_KEY`, so one can be rotated without invalidating the other's uses

WHAT A TOKEN ACTUALLY CONTAINS


```
{"token_type":"access","exp":1756399200,"iat":1756397400,
 "jti":"a1b2…","user_id":7,"name":"Chandrakant","email":"demo@finsight.app"}
```


Signed with HS256, **not encrypted** — anyone can read the payload. That is fine here because it holds no secret, but it is why a password or card number must never go in a claim.

8.4 Logout — an honest limitation

```python
class LogoutView(APIView):
    """Best-effort logout — blacklists the refresh token when possible."""
    def post(self, request):
        token = request.data.get("refresh")
        if token:
            try:
                RefreshToken(token).blacklist()
            except (TokenError, AttributeError):
                pass    # blacklist app not installed / token already invalid
        return Response({"detail": "Logged out."})
```

PARTIAL **LOGOUT DOES NOT REVOKE SERVER-SIDE**

`rest_framework_simplejwt.token_blacklist` is **not** in `INSTALLED_APPS`, so `.blacklist()` raises `AttributeError` and is swallowed. Logout is therefore purely client-side: React clears Redux and localStorage. **A token copied before logout stays valid until it expires** — up to 30 minutes for access, 7 days for refresh. The fix is two lines (add the app, run `migrate`, set `BLACKLIST_AFTER_ROTATION=True`). Say this plainly in an interview; claiming full revocation would be false.

8.5 Authorization — two independent layers

```python
# Layer 1 — queryset scoping. The only rows that exist for this request.
def get_queryset(self):
    return Transaction.objects.filter(account__user=self.request.user) \
                              .select_related("account", "category")
```

```python
# Layer 2 — object permission (common/permissions.py)
class IsOwner(BasePermission):
    def has_object_permission(self, request, view, obj):
        owner = getattr(obj, "user", None)
        if owner is None and hasattr(obj, "account"):
            owner = getattr(obj.account, "user", None)   # transactions own via account
        return owner == request.user


class IsOwnerOrReadOnlyDefault(BasePermission):
    def has_object_permission(self, request, view, obj):
        if getattr(obj, "user", None) != request.user:
            return False
        if request.method in SAFE_METHODS:
            return True
        return not getattr(obj, "is_default", False)   # can't edit the 19 seeded categories
```

WHY "NOT FOUND" AND "NOT YOURS" LOOK IDENTICAL

Because the queryset is already filtered, requesting another user's transaction returns **404, not 403**. That is the correct behaviour: a 403 would confirm the row exists. The permission class is defence in depth — if a queryset were ever written without the user filter, `IsOwner` still blocks the object. Verified by `test_user_isolation_on_detail` in `transactions/tests.py`.

The global default in `REST_FRAMEWORK["DEFAULT_PERMISSION_CLASSES"]` is `IsAuthenticated`, so **a new endpoint is protected unless it explicitly opts out**. Only register, login, refresh and the schema/docs views do.

PART 9 · CHAPTER III

Django Architecture

9.1 settings.py — the parts that matter

```python
REST_FRAMEWORK = {
    "DEFAULT_AUTHENTICATION_CLASSES": ("rest_framework_simplejwt.authentication.JWTAuthentication",),
    "DEFAULT_PERMISSION_CLASSES":     ("rest_framework.permissions.IsAuthenticated",),
    "DEFAULT_PAGINATION_CLASS": "common.pagination.StandardResultsPagination",
    "PAGE_SIZE": 20,
    "DEFAULT_FILTER_BACKENDS": ("django_filters.rest_framework.DjangoFilterBackend",
                                "rest_framework.filters.SearchFilter",
                                "rest_framework.filters.OrderingFilter"),
    "DEFAULT_RENDERER_CLASSES": ("common.responses.EnvelopeJSONRenderer",),
    "EXCEPTION_HANDLER": "common.responses.envelope_exception_handler",
```

```
    "DEFAULT_SCHEMA_CLASS": "drf_spectacular.openapi.AutoSchema",
    # We wrap everything in our own envelope; disable DRF's ?format= negotiation
    # so query params like ?format=csv reach the view instead of 404-ing.
    "URL_FORMAT_OVERRIDE": None,
    "DEFAULT_THROTTLE_CLASSES": ("rest_framework.throttling.ScopedRateThrottle",),
    "DEFAULT_THROTTLE_RATES": {"auth": "20/min", "analytics": "60/min"},
}
```

Every one of these is a global policy decision made once. `URL_FORMAT_OVERRIDE = None` is a real bug fix with a comment explaining it: DRF would otherwise intercept `?format=csv` and 404 before the statements view ever saw it.

Other notable settings

Setting	Effect
`DATABASE_URL` branch	Present → `dj_database_url.parse(url, conn_max_age=600)`; absent → SQLite at `backend/db.sqlite3`
`REDIS_URL` branch	Present → `RedisCache`; absent → `LocMemCache`
Security headers	`SECURE_CONTENT_TYPE_NOSNIFF`, `SECURE_REFERRER_POLICY="same-origin"`, `X_FRAME_OPTIONS="DENY"`; when `DEBUG=False` also HSTS 1 year + secure cookies
`TIME_ZONE`	`Asia/Kolkata`, `USE_TZ=True` — timestamps stored in UTC, rendered locally
`LOGGING`	Structured `level=… time=… logger=… msg=…` lines to stdout. Loggers are named `finsight.auth`, `finsight.transactions`, `finsight.analytics`, `finsight.anomalies`
`STORAGES.staticfiles`	WhiteNoise `CompressedManifestStaticFilesStorage` — hashed, gzipped static files served straight from Gunicorn

9.2 URL layout — `config/urls.py`

```
api_patterns = [
    path("auth/", include("users.urls")),
    path("", include("accounts.urls")),      # router → /api/accounts/
    path("", include("categories.urls")),
    path("", include("transactions.urls")),
    path("", include("statements.urls")),
    path("", include("analytics.urls")),
    path("", include("anomalies.urls")),
]
urlpatterns = [
    path("admin/", admin.site.urls),
    path("health/", healthcheck),
    path("api/schema/", schema_view, name="schema"),
    path("api/docs/",  SpectacularSwaggerView.as_view(url_name="schema", permission_classes=[AllowAny])),
    path("api/redoc/", SpectacularRedocView.as_view(url_name="schema", permission_classes=[AllowAny])),
    path("api/", include(api_patterns)),
]
```

Each app owns its own URL prefix through its router, so the root file never needs editing when an endpoint is added. `/health/` is deliberately outside `/api/` and unauthenticated — Docker's healthcheck calls it.

9.3 The three-layer write path

```
ViewSet        HTTP semantics · queryset scoping · permissions · filter wiring
   ↓
Serializer     field validation · type conversion · FK narrowing to the user's rows
   ↓
Service        business rules · atomicity · row locking · balance arithmetic
   ↓
Model / ORM    schema, constraints, indexes
```

The rule that keeps this honest: **a view never contains balance arithmetic and a service never knows about HTTP.** `create_transaction()` takes model instances and returns a model instance — which is why `seed_demo` can call it directly.

9.4 The response envelope — `common/responses.py`

```python
class EnvelopeJSONRenderer(JSONRenderer):
    def render(self, data, accepted_media_type=None, renderer_context=None):
        response = (renderer_context or {}).get("response")
        status_code = getattr(response, "status_code", 200)
        if isinstance(data, dict) and "success" in data and "message" in data:
            return super().render(data, ...)            # already an envelope → pass through
        if status_code >= 400:
            envelope = {"success": False, "data": data,
                        "message": _first_error_message(data) or "Unable to process request",
                        "error_code": data.get("error_code") if isinstance(data, dict) else None}
        else:
            message = "Request successful"
            if isinstance(data, dict) and "detail" in data and len(data) == 1:
                message, data = str(data["detail"]), None
            envelope = {"success": True, "data": data, "message": message, "error_code": None}
        return super().render(envelope, ...)
```

`_first_error_message()` walks nested serializer errors to lift a human sentence into `message`, so the frontend's `errorMessage()` helper works for every error in the system without knowing anything about the endpoint. Registered globally, so no view formats its own response.

9.5 The five models, field by field

Model	Fields	Constraints, indexes, behaviour
User `users/`	id · name · **email (unique)** · password (hashed) · is_active · is_staff · is_superuser · currency · default_account (FK, null) · notify_anomalies · notify_weekly_summary · last_login · created_at · updated_at	Unique email; `USERNAME_FIELD="email"` ; `ordering=["-created_at"]` ; `default_account` is `SET_NULL` so deleting an account does not delete the user
Account `accounts/`	id · user (FK CASCADE) · account_name · account_type ∈ {BANK, CASH, CREDIT_CARD, WALLET, INVESTMENT, OTHER} · balance `NUMERIC(14,2)` · currency · is_active · created_at · updated_at	`UniqueConstraint(user, account_name)` ; ordering by name; `apply_delta()` saves only the balance column; delete blocked in the view when transactions exist
Category `categories/`	id · user (FK CASCADE) · name · type ∈ {INCOME, EXPENSE} · icon · is_default · created_at · updated_at	`UniqueConstraint(user, name, type)` — the same name may exist once as income and once as expense; 19 defaults seeded by a `post_save` signal; defaults are read-only
Transaction `transactions/`	id · account (FK **CASCADE**) · category (FK **PROTECT**) · amount `NUMERIC(14,2)` ≥ 0.01 · transaction_type · description · merchant · transaction_date · created_at · updated_at	Two composite indexes: `(account, transaction_date)` and `(transaction_type, transaction_date)` ; ordering `-transaction_date, -created_at` ; `signed_amount` property returns ±amount
Anomaly `anomalies/`	id · user (FK CASCADE) · transaction (FK **SET_NULL**, nullable) · detection_method · anomaly_score (float) · severity · reason (Text) · *snapshot:* amount, category, merchant, transaction_date · status ∈ {OPEN, REVIEWED, IGNORED} · reviewed · detected_at · updated_at	`UniqueConstraint(user, transaction, detection_method)` — the key `update_or_create` upserts on; indexes on `(user, status)` and `(user, severity)` ; ordering `anomaly_score, -detected_at` (lower score = more anomalous)

THREE `ON_DELETE` CHOICES, THREE DIFFERENT INTENTIONS

Account → CASCADE. Deleting an account should remove its transactions; they are meaningless without it. The view still blocks deletion while transactions exist, so this fires only through the admin or a user delete.

Category → PROTECT. Deleting a category that has transactions raises `ProtectedError` . Financial history must not silently lose its classification — hence the `?reassign_to=` flow. This is also why `seed_demo` must delete transactions before deleting the demo user.

Transaction → SET_NULL on Anomaly. An anomaly survives its transaction being deleted, because the snapshot columns keep it displayable. The audit trail outlives the row it describes.

Database Design

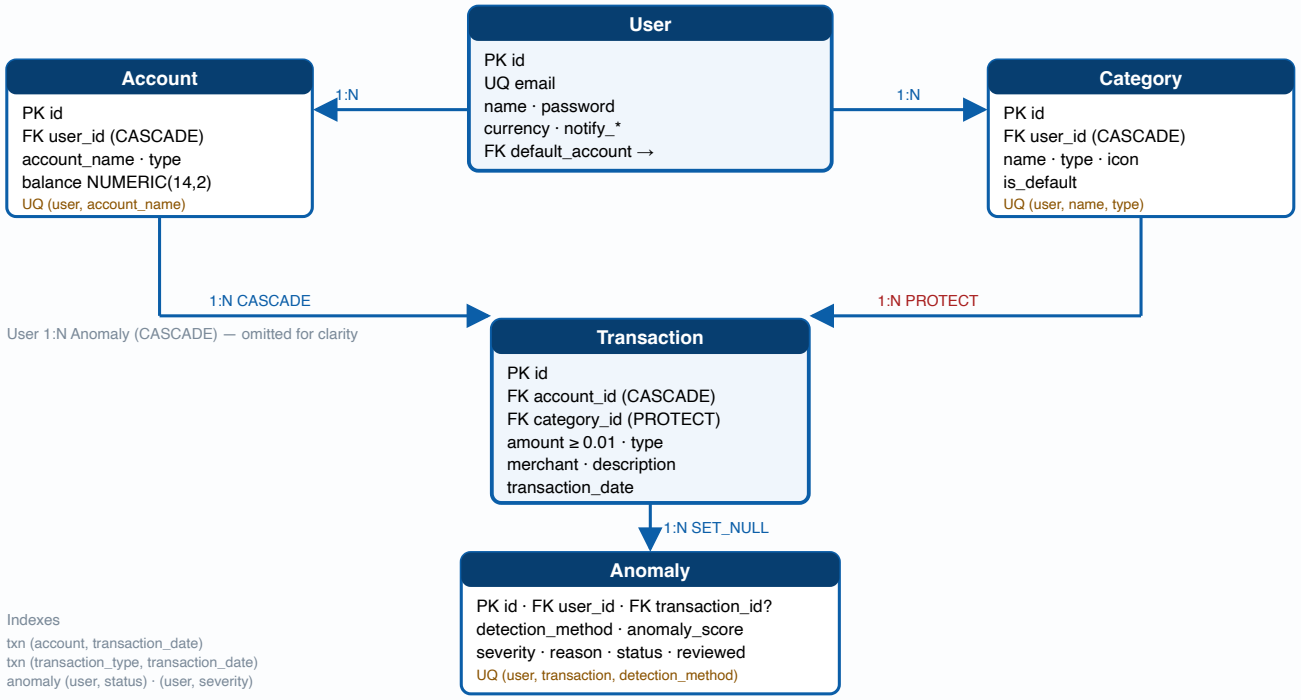


FIGURE 3 · ENTITY RELATIONSHIPS AS MIGRATED

10.1 Relationships

Relationship	Type	Meaning
User → Accounts	1:N	Many accounts per user; deleting the user removes them
User → Categories	1:N	Categories are per-user, not global — your "Food" is a different row from mine
User → Anomalies	1:N	Denormalised FK so anomaly queries never join through transaction→account
Account → Transactions	1:N	CASCADE. **Transactions have no direct user FK** — ownership is via `account__user`
Category → Transactions	1:N	PROTECT — history keeps its classification
Transaction → Anomalies	1:N	One per detection method, enforced by the unique constraint

There are **no many-to-many and no one-to-one relationships** in the schema. Every relationship is a simple foreign key, which is why no join table exists.

10.2 Indexes and query optimisation

Index	Serves
Every FK (automatic)	Joins and ownership filters
`uniq_account_name_per_user`	Prevents duplicate account names; also a lookup index
`uniq_category_per_user_type`	Prevents duplicate categories per type
`uniq_anomaly_per_txn_method`	The `update_or_create` key — makes re-detection idempotent
`(account, transaction_date)`	**The workhorse.** Every dashboard, statement and analytics query filters by account and date range
`(transaction_type, transaction_date)`	"Income this month" / "expenses this month" aggregates
`(user, status)` · `(user, severity)`	The anomalies page's two default filters

Query optimisation actually present: `select_related("account", "category")` on transactions and `select_related("transaction", "transaction__account")` on anomalies (both eliminate N+1); database-side aggregation

with `Sum` / `Count` / `annotate` rather than Python loops; `values_list("id", flat=True)` when only ids are needed; `update_fields=["balance", "updated_at"]` so balance writes touch two columns, not the whole row.

PART 11 · CHAPTER III

SQLite vs PostgreSQL

```python
# config/settings.py
_database_url = os.getenv("DATABASE_URL", "").strip()
if _database_url:
    DATABASES = {"default": dj_database_url.parse(_database_url, conn_max_age=600)}
else:
    DATABASES = {"default": {"ENGINE": "django.db.backends.sqlite3",
                             "NAME": BASE_DIR / "db.sqlite3"}}
```

One environment variable decides. `conn_max_age=600` keeps Postgres connections alive for 10 minutes, avoiding a TCP + TLS handshake on every request.

Aspect	SQLite (no DATABASE_URL)	PostgreSQL (DATABASE_URL set)
Setup	None — a file appears	A running server
Concurrent writes	One writer, database-level lock	MVCC — many concurrent writers
`select_for_update()`	**Ignored**	Real `SELECT … FOR UPDATE` row lock
`DecimalField`	Stored as REAL — floating point	True `NUMERIC(14,2)`
Used for	Local dev, the 43-test suite	Docker and any real deployment

> **THE CONSEQUENCE YOU MUST BE ABLE TO STATE**
>
> The balance-keeping code's concurrency guarantee comes from `select_for_update()`, which **SQLite ignores**. The tests pass on SQLite because they are sequential, but the row-locking behaviour they document is only truly exercised on Postgres. Equally, money as REAL invites `0.1 + 0.2 ≠ 0.3` rounding drift. Both are reasons SQLite is a convenience for development and Postgres is the answer for anything real.

`localhost` vs `postgres`

```
Local (no Docker)   DATABASE_URL unset            → SQLite file
Local + a Postgres  postgres://…@localhost:5432/… → your machine's own network stack
Inside Compose      postgres://…@postgres:5432/… → Docker's DNS resolves the service name

Inside a container, "localhost" is that container itself — Django would be looking
for a database inside its own image. On the compose network, every service is
reachable by its service name: postgres, redis, fastapi, django.
```

PART 12 · CHAPTER III

Django Admin

All five models are registered, in five short `admin.py` files. It exists because a back-office is free here: support workflows, data inspection while developing, and a way to fix a bad row without writing SQL.

```python
@admin.register(Transaction)
class TransactionAdmin(admin.ModelAdmin):
    list_display = ("transaction_date", "merchant", "category", "account",
                    "transaction_type", "amount")
    list_filter  = ("transaction_type", "transaction_date", "category")
    search_fields = ("merchant", "description", "account__account_name")
    readonly_fields = ("created_at", "updated_at")
    date_hierarchy = "transaction_date"
```

`UserAdmin` extends Django's own and redefines `fieldsets` because the model has no `username`; the login form asks for the **email**. `AnomalyAdmin` makes the engine-produced fields read-only, which is correct — a human should change `status`, never `anomaly_score`.

ADMIN AND THE DATABASE ARE INDEPENDENT

The admin UI looks identical whether the rows come from SQLite or PostgreSQL, because it goes through the same ORM. **Nothing in the admin tells you which database you are on** — only `DATABASE_URL` does. Worth knowing: seeing data in the admin is not evidence that Postgres is configured.

PRODUCTION CAUTION

`/admin/` is exposed on the same host as the API with no IP allow-list and no 2FA. For a real deployment: restrict by network, add `django-otp`, or disable it entirely.

PART 13 · CHAPTER III

Transaction & Balance Management

The financial core. Three functions in `transactions/services.py`, each atomic, each locking the rows it touches.

The rule, stated once

```
signed_amount = +amount   if INCOME
                -amount   if EXPENSE

create   balance += new_signed
update   balance -= old_signed ; balance += new_signed     ← always reverse first
delete   balance -= old_signed
```

```python
@db_transaction.atomic
def create_transaction(**fields) -> Transaction:
    account = _locked_account(fields["account"].pk if isinstance(fields["account"], Account)
                              else fields["account"])          # SELECT … FOR UPDATE
    txn = Transaction.objects.create(**fields)
    account.balance = (account.balance or Decimal("0")) + txn.signed_amount
    account.save(update_fields=["balance", "updated_at"])
    logger.info("txn_created id=%s account=%s type=%s delta=%s",
                txn.id, account.id, txn.transaction_type, txn.signed_amount)
    return txn


@db_transaction.atomic
def update_transaction(txn: Transaction, **fields) -> Transaction:
    txn = Transaction.objects.select_for_update().get(pk=txn.pk)
    old_account = _locked_account(txn.account_id)
    old_delta = txn.signed_amount                       # ← capture BEFORE mutating

    new_account_id = fields.get("account").pk if isinstance(fields.get("account"), Account) \
                     else fields.get("account", txn.account_id)
    for key, value in fields.items():
        setattr(txn, key, value)
    txn.full_clean(exclude=["account", "category"])    # model validators run again
    txn.save()

    if new_account_id != old_account.id:               # moved between accounts
        old_account.balance -= old_delta
        old_account.save(update_fields=["balance", "updated_at"])
        new_account = _locked_account(new_account_id)
        new_account.balance += txn.signed_amount
        new_account.save(update_fields=["balance", "updated_at"])
    else:
        old_account.balance = old_account.balance - old_delta + txn.signed_amount
        old_account.save(update_fields=["balance", "updated_at"])
    return txn


@db_transaction.atomic
def delete_transaction(txn: Transaction) -> None:
    txn = Transaction.objects.select_for_update().get(pk=txn.pk)
    account = _locked_account(txn.account_id)
    account.balance = (account.balance or Decimal("0")) - txn.signed_amount
```

```
    account.save(update_fields=["balance", "updated_at"])
    txn.delete()
```

Detail	Why
`@db_transaction.atomic`	The INSERT and the balance UPDATE either both commit or both roll back. Without it, a crash between them leaves the ledger and the balance permanently disagreeing
`select_for_update()`	Locks the account row until commit. Two concurrent transactions on the same account serialise instead of both reading ₹50,000 and both writing their own answer — the classic lost-update bug
`old_delta` captured first	The single most important line in `update`. Reading it after `setattr` would reverse the *new* impact and corrupt the balance
Account-change branch	Moving a transaction between accounts must debit one and credit the other, with both rows locked
`update_fields=[…]`	Writes two columns instead of the whole row — smaller UPDATE, narrower race window
`Decimal` throughout	Exact base-10 arithmetic. Money never becomes a float in Django

WORKED EXAMPLE — THE ₹2,500 FOOD EXPENSE

```
BEGIN
  SELECT … FROM accounts_account WHERE id = 2 FOR UPDATE      -- row locked
  INSERT INTO transactions_transaction (amount=2500.00, type='EXPENSE', …)
  signed_amount = -2500.00
  50,000.00 + (-2,500.00) = 47,500.00
  UPDATE accounts_account SET balance=47500.00, updated_at=now() WHERE id=2
COMMIT                                                        -- lock released

Then edited to ₹3,000:
  old_delta = -2500.00  (captured first)
  47,500.00 - (-2,500.00) + (-3,000.00) = 47,000.00   ✓

Then deleted:
  47,000.00 - (-3,000.00) = 50,000.00                 ✓ back to the start
```

Six of the fifteen transaction tests exist purely to protect this arithmetic: `test_expense_decreases_balance`, `test_income_increases_balance`, `test_update_reverses_previous_balance_impact`, `test_changing_type_recomputes_balance`, `test_move_transaction_between_accounts` and `test_delete_reverses_balance`.

Where statements get their numbers

`statements/services.py` deliberately does *not* trust `Account.balance`. It derives the opening balance from the ledger:

```
opening_balance = current_balance − Σ signed_amount(date ≥ period_start)
closing_balance = opening_balance + net_savings(period)
```

So even if the cached balance column were ever wrong, statements remain internally consistent with the immutable transaction history.

PART 14 · CHAPTER III

REST API Reference

All paths are relative to `/api/`. Unless noted, every endpoint requires `Authorization: Bearer <access>` and returns the standard envelope.

14.1 Authentication — `users/urls.py`

Method	Path	Auth	Body / behaviour
POST	`/auth/register/`	Public · 20/min	`{name, email, password}` → 201 `{user, access, refresh}`
POST	`/auth/login/`	Public · 20/min	`{email, password}` → 200 `{access, refresh, user}`; 401 on bad credentials
POST	`/auth/refresh/`	Public	`{refresh}` → 200 `{access, refresh}` (rotation on)
POST	`/auth/logout/`	Required	`{refresh}` → 200. PARTIAL blacklist is a no-op
GET / PUT / PATCH	`/auth/me/`	Required	Read or update name, currency, default_account, notification flags
POST	`/auth/change-password/`	Required	`{current_password, new_password}`; verifies the current password first

14.2 Resources

Method	Path	Notes
GET/POST	`/accounts/`	List (paginated, annotated with `transaction_count`) / create. Filters: `account_type`, `is_active`, `currency`. `opening_balance` is write-only and honoured on create only
GET/PUT/PATCH/DELETE	`/accounts/{id}/`	Delete returns 400 if the account has transactions
GET/POST	`/categories/`	Annotated with `transaction_count` and `total_amount`. Filters: `type`, `is_default`
GET/PUT/PATCH/DELETE	`/categories/{id}/`	Defaults are read-only. `DELETE ?reassign_to={id}` moves the transactions first
GET	`/categories/defaults/`	Extra action — just the seeded set
GET/POST	`/transactions/`	The busiest endpoint. Filters: `account`, `category`, `category_name`, `type`, `start_date`, `end_date`, `min_amount`, `max_amount`; `search` over merchant/description/category name; `ordering` by date, amount, created_at
GET/PUT/PATCH/DELETE	`/transactions/{id}/`	Every write path goes through the balance-keeping service
GET	`/statements/`	`?year&month&account&format=json|csv`. CSV streams a download

Example — create a transaction

```
POST /api/transactions/          Authorization: Bearer eyJ…
{"account": 2, "category": 1, "amount": "2500.00", "transaction_type": "EXPENSE",
 "merchant": "Swiggy", "description": "", "transaction_date": "2026-08-28"}

201
{"success": true, "error_code": null, "message": "Request successful",
 "data": {"id": 412, "account": 2, "account_name": "HDFC Savings",
          "category": 1, "category_name": "Food", "category_icon": "restaurant",
          "amount": "2500.00", "transaction_type": "EXPENSE", "merchant": "Swiggy",
          "description": "", "transaction_date": "2026-08-28", "currency": "INR",
          "created_at": "2026-08-28T18:42:11Z", "updated_at": "2026-08-28T18:42:11Z"}}
```

```
400  — category type mismatch
{"success": false, "error_code": "invalid",
 "message": "An EXPENSE transaction must use an EXPENSE category (got a INCOME category).",
 "data": {"category": ["An EXPENSE transaction must use an EXPENSE category (got a INCOME category)."]}}
```

Paginated list shape

```
{"success": true, "message": "Request successful", "error_code": null,
 "data": {"results": [ … ], "count": 187, "page": 1, "page_size": 20,
          "total_pages": 10, "next": "http://…?page=2", "previous": null}}
```

14.3 Analytics — `analytics/urls.py`

Method	Path	Computed by
GET	`/dashboard/`	**Django locally**, enriched with FastAPI anomaly alerts. Params `year`, `month`. Never 503s
GET	`/analytics/summary/`	FastAPI. Params `start_date`, `end_date`
GET	`/analytics/trends/`	FastAPI. Adds `granularity=daily|weekly|monthly`
GET	`/analytics/categories/`	FastAPI
GET	`/analytics/anomalies/`	FastAPI — detection only, **no persistence**
GET	`/analytics/analyze/`	FastAPI — summary + trends + categories + anomalies + insights in one call

All five FastAPI-backed endpoints are throttled at `60/min`, cached for 900 s, and return **503** with `error_code: "ANALYTICS_UNAVAILABLE"` when the engine is unreachable.

14.4 Anomalies — `anomalies/urls.py`

Method	Path	Purpose
GET	`/anomalies/`	List. Filters: `severity`, `detection_method`, `status`, `reviewed`, `category`, `start_date`, `end_date`. Default ordering by `anomaly_score`
GET	`/anomalies/{id}/`	Retrieve
POST	`/anomalies/detect/`	`{method: "ALL"|"ZSCORE"|"IQR"|"ISOLATION_FOREST"}` → `{detected, anomalies[]}`. **The only endpoint that persists anomalies**
POST	`/anomalies/{id}/review/`	status → REVIEWED
POST	`/anomalies/{id}/ignore/`	status → IGNORED
POST	`/anomalies/{id}/reopen/`	status → OPEN

The viewset is `ListModelMixin + RetrieveModelMixin` only — **there is deliberately no create, update or delete**. Anomalies are produced by detection and mutated only through the three status actions.

14.5 Meta endpoints

Path	Purpose
`/health/`	Unauthenticated liveness check (Docker healthcheck)
`/api/schema/` · `/api/docs/` · `/api/redoc/`	OpenAPI 3 YAML, Swagger UI, Redoc — assets self-hosted via `drf-spectacular-sidecar`
`/admin/`	Django admin (email login)

PART 37 · CHAPTER III

Database Migrations

```
models.py  ──makemigrations──►  0001_initial.py  ──migrate──►  PostgreSQL / SQLite
                                (Python, committed)        (CREATE TABLE / ALTER)
```

Migrations are Django's schema version control: every change to a model becomes a committed Python file describing the transition, applied in order and recorded in the `django_migrations` table so each runs exactly once.

What is actually in the repository

App	Migrations	Note
users	`0001_initial`	Must be first — everything references `AUTH_USER_MODEL`
accounts	`0001`, `0002_initial`	Split because User ↔ Account reference each other (`default_account`)
categories	`0001`, `0002_initial`	Same circularity pattern
transactions	`0001_initial`	Table + both composite indexes
anomalies	`0001`, `0002`, `0003_initial`	References both user and transaction
statements · analytics	empty package only	**No models** — both compute from existing tables

The multi-file splits are Django resolving circular dependencies: it creates the tables first, then adds the foreign keys that point back. This is normal and worth being able to explain.

Handling schema changes safely

1. **Review the generated file before committing.** Renaming a field can be emitted as drop + add, which destroys data.
2. **Adding a non-null column to a populated table needs a default** or a three-step deploy: add nullable → backfill in a data migration → make non-null.
3. **Expand then contract** for renames: add the new column, dual-write, backfill, switch reads, drop the old one. Never in a single deploy.
4. **Test on a copy of production data** — `migrate --plan` shows what will run.
5. **Backfills belong in `RunPython`** data migrations with a reverse function.

> **HOW MIGRATIONS ARE APPLIED HERE**
>
> In Docker they run automatically — the backend image's CMD is `sh -c "python manage.py migrate --noinput && gunicorn config.wsgi:application …"`, so every container start migrates before serving. Locally you run `make migrate`.
>
> **The caveat worth naming:** with more than one replica, every replica races to migrate on boot. Django takes a lock per migration so this is usually safe, but the correct production pattern is a *separate release step* that migrates once before any new container starts serving. PLANNED

PART 15 · CHAPTER IV

FastAPI Architecture

A stateless calculator: JSON in, JSON out. No database, no session, no migrations. ~1,360 lines across eleven modules in `analytics-service/app/`.

15.1 Startup — `app/main.py`

```python
app = FastAPI(title="FinSight Analytics Engine", version="1.0.0",
              description="Stateless spending analytics and anomaly detection for FinSight.")

app.add_middleware(CORSMiddleware,
    allow_origins=["*"],        # only Django talks to this service; it is not public.
    allow_methods=["POST", "GET"], allow_headers=["*"])

@app.exception_handler(RequestValidationError)
async def validation_exception_handler(request, exc):
    logger.warning("invalid_input path=%s errors=%s", request.url.path, exc.errors())
    return JSONResponse(422, {"success": False, "message": "Invalid analytics payload",
                              "detail": exc.errors()})

@app.exception_handler(Exception)
async def unhandled_exception_handler(request, exc):
    logger.exception("analytics_processing_error path=%s", request.url.path)
    return JSONResponse(500, {"success": False, "message": "Analytics processing failed"})

@app.get("/health", tags=["meta"])
def health() -> dict:
    return {"status": "ok", "service": settings.service_name, "environment": settings.environment}

app.include_router(analytics_router)
```

Element	Why
`allow_origins=["*"]`	Reads alarming; the comment is the justification. Only Django calls this, server-to-server, where CORS is irrelevant. **The real control is the bearer token.** Note that compose still publishes `9000:9000` to the host — flagged in Part 32
Catch-all handler	An unexpected pandas error becomes a clean 500 with **no internal detail in the body**, and a full traceback in the logs
`/health`	Outside the router, so it carries no token dependency — the Docker healthcheck can reach it. It exposes no data

15.2 Configuration — `app/config.py`

```python
class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_file=".env", extra="ignore")
    service_name: str = "finsight-analytics"
    environment: str = "development"
    fastapi_service_token: str = "dev-service-token"
    require_service_token: bool = False        # local dev default
    zscore_threshold: float = 3.0
    iqr_multiplier: float = 1.5
    isolation_forest_contamination: float = 0.05
    min_group_size: int = 5
    max_transactions: int = 100_000
    log_level: str = "INFO"

settings = Settings()
```

pydantic-settings maps each attribute to an environment variable, typed and validated at startup. `extra="ignore"` lets the service share a `.env` with Django.

> **ONE DECLARED KNOB IS NOT WIRED UP**
>
> `max_transactions` is defined and documented but **no code reads it** — there is no payload size limit anywhere. PARTIAL
> Enforcing it in `AnalyticsRequest` is a one-line validator; listed in Part 33.

15.3 The contract — `app/schemas/analytics.py`

The docstring states the intent: "*These are the only contract between Django and the engine. They intentionally mirror nothing in Django's ORM.*"

```
class TransactionIn(BaseModel):
    id: int
    amount: float = Field(gt=0, description="Positive magnitude of the transaction.")
    category: str = "Other"
    type: TransactionType = TransactionType.EXPENSE
    account: Optional[str] = None
    merchant: Optional[str] = ""
    date: date

    @field_validator("category", mode="before")
    @classmethod
    def _default_category(cls, v):
        return v or "Other"          # coerce None and "" → "Other"

class AnalyticsRequest(BaseModel):
    user_id: int
    transactions: list[TransactionIn] = Field(default_factory=list)

class TrendRequest(AnalyticsRequest):   granularity: Granularity = Granularity.MONTHLY
class AnomalyRequest(AnalyticsRequest): method: DetectionMethod = DetectionMethod.ALL
```

`mode="before"` runs prior to type coercion — the only way to catch a `None` that would fail the `str` annotation. It matters because `groupby("category")` silently *drops* NaN groups: an empty category would make transactions vanish from the analysis rather than error.

Four enums (`TransactionType`, `DetectionMethod`, `Severity`, `Granularity`) all inherit `str`, so they serialise directly and compare to plain strings. Eight response models; the important one:

> **`ANOMALYITEM` — THE SWAP POINT**
>
> `transaction_id · amount · category · merchant · date · detection_method · anomaly_score · severity · reason`. Django's `detect_and_store` reads exactly these nine keys and the React `Anomaly` type mirrors them. **Any new detector that emits this shape can replace the maths without touching Django or the frontend.** That is the concrete payoff of the service split.

15.4 Routes — `app/api/analytics.py`

```
router = APIRouter(prefix="/analytics", tags=["analytics"],
                   dependencies=[Depends(require_service_token)])   # applies to EVERY route

@router.post("/summary", response_model=SummaryResponse)
def summary(request: AnalyticsRequest) -> SummaryResponse:
    started = time.perf_counter()
    result = build_summary(request)
    _log_duration("summary", request.user_id, len(request.transactions), started)
    return result
```

Six routes, all the same five-line shape: start a timer, delegate to a service, log the duration, return. **No business logic in the route layer.** The duration log carries the transaction count and milliseconds together, so a slow request is immediately attributable.

```
@router.post("/analyze", response_model=AnalyzeResponse)
def analyze(request: AnalyticsRequest) -> AnalyzeResponse:
    """One-shot: everything the Analytics page needs in a single call."""
    summary_res    = build_summary(request)
    trends_res     = build_trends(TrendRequest(**request.model_dump()))
    categories_res = build_categories(request)
    anomalies_res  = detect_anomalies(AnomalyRequest(**request.model_dump()))
    insights = build_insights(summary_res, trends_res, categories_res, anomalies_res)
    return AnalyzeResponse(...)
```

The four calls each re-run `to_dataframe()` — a known inefficiency noted in Part 33, and the reason the Analytics page makes one `/analyze` call instead of four.

EVERY ENDPOINT IS `def`, NOT `async def` — DELIBERATELY

pandas and scikit-learn are synchronous and CPU-bound. Declaring them `async` would block the event loop for the whole model fit. With a plain `def`, Starlette runs each handler in a threadpool, so one long Isolation Forest fit does not stall other requests. Concurrency comes from `uvicorn --workers 2` plus the threadpool — not from async I/O, because there is no I/O to await.

15.5 Service-to-service auth — `app/security.py`

```python
async def require_service_token(authorization: str | None = Header(default=None)) -> None:
    if not settings.require_service_token:
        return                                  # local dev escape hatch
    if not authorization or not authorization.lower().startswith("bearer "):
        raise HTTPException(401, "Missing bearer token")
    presented = authorization.split(" ", 1)[1].strip()
    if not hmac.compare_digest(presented, settings.fastapi_service_token):
        raise HTTPException(403, "Invalid service token")
```

`hmac.compare_digest` is a **constant-time comparison**. A plain `==` short-circuits on the first differing byte, leaking token content through response timing. This is the most security-literate line in the repository. 401 means "you presented nothing", 403 means "you presented something wrong" — the correct semantics.

The default `require_service_token=False` makes `uvicorn app.main:app --reload` work with zero setup; `docker-compose.yml` sets it to `"true"`, so any containerised run enforces it. **The insecure default only applies to a laptop.**

PART 16 · CHAPTER IV

Django → FastAPI Communication

Two files carry the whole integration: `analytics/client.py` (67 lines — transport and failure) and `analytics/services.py` (271 lines — payload, caching, orchestration).

16.1 The HTTP client, annotated

```python
class AnalyticsUnavailable(Exception):
    default_message = "Analytics service is temporarily unavailable."

def _headers() -> dict[str, str]:
    return {"Authorization": f"Bearer {settings.FASTAPI_SERVICE_TOKEN}",
            "Content-Type": "application/json"}

def call_analytics(path: str, payload: dict) -> dict:
    url = f"{settings.FASTAPI_URL.rstrip('/')}{path}"                          # ①
    try:
        with httpx.Client(timeout=settings.FASTAPI_TIMEOUT_SECONDS) as client: # ②
            response = client.post(url, json=payload, headers=_headers())
    except (httpx.TimeoutException, httpx.TransportError) as exc:              # ③
        logger.warning("analytics_call_failed path=%s error=%s", path, exc.__class__.__name__)
        raise AnalyticsUnavailable() from exc

    if response.status_code >= 500:                                            # ④
        logger.error("analytics_upstream_5xx path=%s status=%s", path, response.status_code)
        raise AnalyticsUnavailable()
    if response.status_code >= 400:                                            # ⑤
        logger.warning("analytics_bad_request path=%s status=%s body=%s",
                       path, response.status_code, response.text[:300])
        raise AnalyticsUnavailable("Analytics request was rejected by the engine.")
    try:
        return response.json()                                                 # ⑥
    except ValueError as exc:
        raise AnalyticsUnavailable("Analytics engine returned an invalid response.") from exc
```

	Explanation
①	`rstrip('/')` makes the client indifferent to a trailing slash in `FASTAPI_URL`
②	**Every call has a timeout** (default 8 s). Without it a hung engine would hold a Gunicorn worker forever; a handful of those and Django stops serving logins
③	Timeout and transport errors become one domain exception — **the caller never sees an httpx type**, so the transport library is fully encapsulated
④⑤	5xx logs at `error` (the engine is broken); 4xx logs at `warning` with **300 chars of body** — the line that tells you which field Pydantic rejected, truncated so a huge payload cannot flood the logs
⑥	Even a 200 is distrusted: a malformed body becomes the same friendly failure

THE KEY PROPERTY

Five failure modes — timeout, transport error, 5xx, 4xx, bad JSON — collapse into one exception type. Callers write one `except AnalyticsUnavailable` and are correct for every way the engine can fail. That is what makes the graceful degradation in Part 31 short enough to actually be right.

`ping()` uses a tighter 2-second timeout, because a health check that takes 8 seconds to say "down" is not a health check. PARTIAL — defined but not currently called; Docker hits `/health` directly.

16.2 Building the payload

```python
def serialize_for_engine(qs) -> list[dict]:
    return [{"id": t.id,
             "amount": float(t.amount),                  # Decimal → float for JSON
             "category": t.category.name,                # NAME, not id
             "type": t.transaction_type,
             "account": t.account.account_name,
             "merchant": t.merchant or "",
             "date": t.transaction_date.isoformat()}
            for t in qs]
```

THREE DELIBERATE CHOICES IN SEVEN LINES

Category name, not id. The engine groups by `"Food"`, not `17` — which is what keeps FastAPI free of any knowledge of Django's primary keys.

`float(t.amount)`. Money stays `Decimal` in Django and becomes a float only for statistics. Precision loss in an average is acceptable; in a balance it is not — and the balance never crosses this boundary.

No PII. An integer `user_id` and transaction facts. No name, no email, no account number.

16.3 Caching

```python
def _digest(payload: list[dict]) -> str:
    raw = json.dumps(payload, sort_keys=True, separators=(",", ":")).encode()
    return hashlib.sha1(raw).hexdigest()[:12]

def _cache_key(user_id, kind, payload, extra="") -> str:
    return f"analytics:user:{user_id}:{kind}:{extra}:{_digest(payload)}"

def _cached_call(user_id, kind, path, txns, extra="", **body):
    key = _cache_key(user_id, kind, txns, extra)
    hit = cache.get(key)
    if hit is not None:
        return hit
    result = call_analytics(path, {"user_id": user_id, "transactions": txns, **body})
    cache.set(key, result, settings.ANALYTICS_CACHE_TTL_SECONDS)
    return result
```

Four key components: `user:{id}` (isolation), `{kind}` (which analysis), `{extra}` (date range, granularity, method), and **a SHA-1 digest of the payload**.

CONTENT-ADDRESSING REMOVES THE INVALIDATION PROBLEM

Because the key contains a hash of the exact transaction set, **adding, editing or deleting any transaction changes the key.** The old entry is not invalidated — it simply stops being addressed and expires after 900 s. Cache correctness needs no invalidation logic at all. The honest cost: the digest requires fetching and serialising the transactions even on a hit, so a hit saves the HTTP round trip and the computation, not the DB read. `sort_keys=True` guarantees a stable digest.

PARTIAL `invalidate_user()` exists in this module with a `delete_pattern`-to-`clear()` fallback, but **nothing calls it**. It is unnecessary given content-addressed keys — but it is dead code, and worth saying so.

16.4 The five wrappers and the view base

```python
def summary(user, start=None, end=None) -> dict:
    txns = serialize_for_engine(user_transactions(user, start, end))
    return _cached_call(user.id, "summary", "/analytics/summary", txns, extra=f"{start}_{end}")

def anomalies(user, method="ALL", start=None, end=None) -> dict:
    txns = serialize_for_engine(user_transactions(user, start, end))
    return _cached_call(user.id, "anomalies", "/analytics/anomalies", txns,
                        extra=f"{method}_{start}_{end}", method=method)
```

```python
class _AnalyticsView(APIView):                # template method — 5 views subclass it
    permission_classes = [IsAuthenticated]
    throttle_scope = "analytics"
    query_serializer = DateRangeSerializer

    def get(self, request):
        params = self.query_serializer(data=request.query_params)
        params.is_valid(raise_exception=True)
        try:
            result = self.run(request.user, params.validated_data)
        except AnalyticsUnavailable as exc:
            payload = dict(UNAVAILABLE_PAYLOAD)     # copy, so the override can't mutate the constant
            payload["message"] = exc.message or payload["message"]
            return Response(payload, status=503)
        return Response(result)
```

`SummaryView.run` is a single line; the `except` is written once for all five views. **503, not 500** — Django is healthy, an upstream dependency is not, and 503 tells clients a retry may succeed.

16.5 What the wire carries

```
POST http://fastapi:9000/analytics/anomalies
Authorization: Bearer shared-service-to-service-secret

{"user_id": 7, "method": "ALL",
 "transactions": [
   {"id": 402, "amount": 18000.0, "category": "Rent", "type": "EXPENSE",
    "account": "HDFC Savings", "merchant": "Landlord Transfer", "date": "2026-08-03"},
   {"id": 399, "amount": 45000.0, "category": "Shopping", "type": "EXPENSE",
    "account": "HDFC Credit Card", "merchant": "Apple Store", "date": "2026-08-22"}]}
```

```
HTTP 200
{"user_id": 7, "method": "ALL", "analysed_count": 187, "anomaly_count": 2,
 "anomalies": [
   {"transaction_id": 399, "amount": 45000.0, "category": "Shopping",
    "merchant": "Apple Store", "date": "2026-08-22",
    "detection_method": "ISOLATION_FOREST", "anomaly_score": -0.284, "severity": "CRITICAL",
    "reason": "This Shopping transaction of ₹45,000 is unusual compared with your historical
               spending pattern (~12.4x your Shopping average)."}]}
```

16.6 How the integration is tested

`backend/analytics/tests.py` mocks `httpx.Client` entirely, so the Django suite never needs FastAPI running.

Test	Asserts
`test_successful_summary_proxied_and_cached`	200 passes through; the second identical call constructs no HTTP client at all
`test_timeout_returns_friendly_503`	`TimeoutException` → 503 with `ANALYTICS_UNAVAILABLE`
`test_upstream_5xx_returns_503`	A 500 from the engine never becomes a Django 500
`test_invalid_json_returns_503`	A 200 with an unparseable body → 503
`test_dashboard_survives_analytics_outage`	`ConnectError` → dashboard still 200, `analytics_available: false`

```
with patch("analytics.client.httpx.Client") as should_not_call:
    self.client.get("/api/analytics/summary/")
    should_not_call.assert_not_called()        # ← proves the cache hit
```

PART 17 · CHAPTER IV

Why FastAPI Is Stateless

A stateless service keeps **no memory between requests**. Everything it needs arrives in the request; everything it produces leaves in the response. Any replica can serve any request.

```
PostgreSQL      the only durable state in the system
    ▲
  Django        source of truth
    ▲
 FastAPI        Input ──► Calculation ──► Output   (remembers nothing)
```

17.1 What the engine provably does not have

Absent	Verification
Database driver	No psycopg, SQLAlchemy or sqlite3 in `requirements.txt`
Models or migrations	No ORM, no `alembic/`, no schema of any kind
Sessions or cookies	No session middleware; auth is a stateless header check
User table	`user_id` is an opaque integer used only for logging and echoing back
Persisted model artefacts	The forest is fitted per request and discarded — no `.pkl`, no registry

17.2 Why fitting per request is defensible

Cost	Benefit	Verdict
~50–200 ms to fit 200 trees	The model is always current — a transaction added a second ago is in the baseline	Correct for this workload
Recomputed per user	**Personalisation is free.** Your normal is not my normal — which is exactly what anomaly detection needs	Architecturally load-bearing
No warm model cache	No registry, no versioning, no staleness, no retraining pipeline, no drift monitoring	Large operational saving
Repeated work on repeat calls	Django's Redis cache absorbs it — the same payload never reaches the engine twice within 15 minutes	Mitigated one layer up

`random_state=42` makes it reproducible: the same payload always yields the same findings — essential for the tests and for a user who refreshes and expects the same answer.

17.3 What statelessness buys

Property	Because there is no state…
Horizontal scaling	`--scale fastapi=5`. No sticky sessions, no shared cache, no leader election
Deployment	Kill and replace at will. No migrations, no data to drain, no warm-up
Testing	Every function is `f(input) → output`. The 27 tests need no database and no mocking of persistence
Reliability	A crash loses nothing. Restart and the next request is correct
Replaceability	Rewrite the engine in Rust tomorrow; honour the JSON contract and Django cannot tell. No data to migrate, because there is no data
Security surface	Compromising the engine yields no stored financial data — only the payload in flight

THE RULE THIS ENFORCES

Suppose the engine wrote anomalies straight to Postgres. You would have two writers to one table with no shared transaction, a second place the "reviewed" flag can change, a schema coupling that makes the ML service block on a Django migration, and a scaling story involving connection pools. FinSight avoids all of it with one rule, visible in `anomalies/services.py`: **the engine returns findings; Django decides what to keep.**

PART 18 · CHAPTER IV

Spending Analytics

18.1 Summary — `app/services/spending_service.py`

```python
def build_summary(request: AnalyticsRequest) -> SummaryResponse:
    df       = to_dataframe(request.transactions)
    expenses = expenses_only(df)
    income   = income_only(df)
    stats = describe(expenses["amount"].to_numpy(dtype="float64"))   # 7 statistics, one pass

    total_spending = stats["sum"]
    total_income   = safe_float(income["amount"].sum())
    months  = month_span(df)                       # distinct calendar months, min 1
    savings = safe_float(total_income - total_spending)
    savings_rate = safe_float(savings / total_income * 100) if total_income > 0 else 0.0
    ...
    monthly_average_spending = safe_float(total_spending / months)
    transaction_count = int(len(df))          # ALL transactions
    expense_count     = int(len(expenses))    # expenses only
```

THE DISTINCTION THAT TRIPS PEOPLE UP

Average, median, min and max are computed over **expenses only** — an ₹85,000 salary would otherwise dominate each of them and make "average transaction" meaningless as a spending signal. But `transaction_count` counts everything. Both are returned so the frontend never has to guess.

Worked example — six months of seeded data

Metric	Formula	Value
total_income	Σ INCOME	6 × 85,000 = **510,000.00**
total_spending	Σ EXPENSE	**231,900.00**
average_transaction	Σx / n over expenses	231,900 / 187 = **1,240.11**
median_transaction	middle of the sorted expenses	**720.00** — far below the mean
monthly_average_spending	total / distinct months	231,900 / 6 = **38,650.00**
savings · savings_rate	income − spending; ÷ income × 100	**278,100.00** · **54.53 %**

The gap between mean ₹1,240 and median ₹720 *is* the insight: a right-skewed distribution, most transactions small, a few large. The median answers "what does a typical purchase cost me"; the mean answers "spending ÷ count". Both are shown so the skew is visible.

The two helpers everything depends on

```python
def describe(values: np.ndarray) -> dict:
    values = np.asarray(values, dtype="float64")
    if values.size == 0:
        return {k: 0.0 for k in ("mean","median","std","min","max","sum","count")}
    return {"mean": safe_float(values.mean()), "median": safe_float(np.median(values)),
            "std": safe_float(values.std(ddof=0)), "min": safe_float(values.min()),
            "max": safe_float(values.max()), "sum": safe_float(values.sum()),
            "count": int(values.size)}

def safe_float(value, default: float = 0.0) -> float:
    try:    f = float(value)
    except (TypeError, ValueError): return default
    if np.isnan(f) or np.isinf(f):  return default
    return round(f, 2)
```

The empty guard is why a brand-new user gets a zeroed dashboard instead of a 500. `ddof=0` is the *population* standard deviation — correct because the transactions are the complete set being described, not a sample. `safe_float` is called on essentially every number leaving the engine: `0/0` gives `nan` and `x/0` gives `inf`, and **neither is valid JSON**.

18.2 Trends — `app/services/trend_service.py`

```python
_PERIOD_COLUMN = {Granularity.DAILY: "day",         # "2026-08-28"
                  Granularity.WEEKLY: "iso_week",   # "2026-W35"
                  Granularity.MONTHLY: "year_month"}# "2026-08"

df["_spend"]  = np.where(df["type"] == "EXPENSE", df["amount"], 0.0)     # ①
df["_income"] = np.where(df["type"] == "INCOME",  df["amount"], 0.0)

grouped = (df.groupby(col)
             .agg(spending=("_spend","sum"), income=("_income","sum"),
                  transaction_count=("id","count"))
             .reset_index().sort_values(col))

monthly = df.groupby("year_month")["_spend"].sum().sort_index()          # ②
avg_monthly = safe_float(monthly.mean())
mom = pct_growth(monthly.iloc[-2], monthly.iloc[-1]) if monthly.size >= 2 else None

slope = linear_trend(np.array([p.spending for p in points]))             # ③
spend_scale = max(avg_monthly, 1.0)
trend = ("increasing" if slope >  0.05 * spend_scale else
         "decreasing" if slope < -0.05 * spend_scale else "stable")
```

	Explanation
①	`np.where` creates two *parallel* columns instead of filtering into two frames, so one `groupby` yields both series and a period with expenses but no income still appears with income 0. Filtering first would misalign the chart
②	**Month-over-month is always computed monthly**, whatever granularity was requested — "MoM growth" over daily buckets would be meaningless
③	The threshold is **relative** — 5% of average monthly spending. An absolute threshold would call ₹200/month drift "increasing" for someone spending ₹5,000 and "stable" for someone spending ₹500,000

```python
def pct_growth(previous, current) -> float | None:
    if previous in (0, None):
        return None                    # undefined, not 0 — growth from nothing has no meaning
    return round((current - previous) / abs(previous) * 100, 1)

def linear_trend(y: np.ndarray) -> float:
    if y.size < 2: return 0.0
    slope, _, _, _, _ = stats.linregress(np.arange(y.size, dtype="float64"), y)
    return safe_float(slope)
```

Example. 40,000 → 48,000 → 55,000: MoM = (55,000−48,000)/48,000×100 = **14.6 %**; slope over x=[0,1,2] is **7,500**; average monthly 47,667 gives a threshold of 2,383; 7,500 > 2,383 → `"increasing"`. Fitting a line across all periods is more robust than comparing the last two — one unusual month cannot flip the verdict.

18.3 Category breakdown & insights

```
df = expenses_only(to_dataframe(request.transactions))     # income excluded entirely
total = float(df["amount"].sum())
grouped = (df.groupby("category")["amount"]
             .agg(total="sum", count="count", average="mean", maximum="max")
             .reset_index().sort_values("total", ascending=False))
# percentage = row["total"] / total * 100 ;  largest_category = categories[0].category
```

Sorting descending means `categories[0]` is by definition the largest, so `largest_category` needs no second pass. Income is excluded because "Salary is 68% of your spending" would be nonsense.

Input	total	count	average	maximum	percentage
Food: ₹100, ₹300	400.00	2	200.00	300.00	40.0 %
Bills: ₹600	600.00	1	600.00	600.00	60.0 %

`test_per_category_math` asserts all five Food values; `test_category_breakdown_endpoint` asserts the percentages sum to 100 ± 0.5 and that "Salary" is absent.

```
# insight_service.py — pure presentation over the four computed responses
if   pct >  1: f"Your spending increased by {pct:.0f}% compared with last month."
elif pct < -1: f"Your spending decreased by {abs(pct):.0f}% compared with last month."
else:          "Your spending is roughly flat compared with last month."
...
if   savings_rate >= 20: f"You saved {savings_rate:.0f}% of your income — a healthy rate."
elif savings_rate >  0:  f"Your savings rate is {savings_rate:.0f}%. Aim for 20% or more."
else:                    "You spent more than you earned this period."
```

The ±1% dead zone avoids "increased by 0%". The 20% threshold is a hard-coded rule of thumb, not a computed target — worth acknowledging as such.

18.4 Where each metric is computed

Metric	Computed by	Shown on
Total balance · monthly income/expenses/savings/rate	**Django** (`Sum` aggregates)	Dashboard StatCards
6-month trend · monthly category breakdown · top 5 · recent 8	**Django**	Dashboard widgets
Total, average, median, savings rate	**FastAPI** `build_summary`	Analytics StatCards
Trend points, MoM %, direction	**FastAPI** `build_trends`	Analytics chart + subtitle
Category total/%/count/avg/max · insights	**FastAPI**	Analytics donut, table, insight list
Anomalies	**FastAPI** detects, **Django** stores	All three surfaces

> **THE DELIBERATE DUPLICATION**
>
> Django computes summary numbers locally *and* FastAPI computes similar ones. This is the resilience strategy, not an accident: the dashboard is opened daily, so its numbers must never depend on a second service. The Analytics page is the deep dive, so it may require the engine. The cost is two implementations of "sum of expenses this month"; the benefit is that an outage is invisible on the page that matters most.

PART 19 · CHAPTER IV

pandas in FinSight

Exactly one function builds a DataFrame, and every analytics service consumes it.

```
# app/utils/preprocessing.py
def to_dataframe(transactions: list[TransactionIn]) -> pd.DataFrame:
    if not transactions:                                                    # ①
        return pd.DataFrame(columns=COLUMNS).astype({"amount": "float64"})

    df = pd.DataFrame([t.model_dump() for t in transactions])               # ②
    df["date"]     = pd.to_datetime(df["date"])                             # ③
```

```
    df["amount"]   = pd.to_numeric(df["amount"], errors="coerce").fillna(0.0)   # ④
    df["type"]     = df["type"].astype(str)
    df["category"] = df["category"].fillna("Other").replace("", "Other")        # ⑤
    df["merchant"] = df["merchant"].fillna("")

    df["day_of_week"]  = df["date"].dt.dayofweek                # 0 = Monday
    df["day_of_month"] = df["date"].dt.day
    df["month"]        = df["date"].dt.month
    df["year"]         = df["date"].dt.year
    df["year_month"]   = df["date"].dt.to_period("M").astype(str)   # "2026-08"
    df["iso_week"]     = df["date"].dt.strftime("%G-W%V")           # "2026-W35"
    df["day"]          = df["date"].dt.strftime("%Y-%m-%d")
    return df
```

	Why this line exists
①	**Load-bearing.** `pd.DataFrame([])` has no columns, so `df["amount"]` would raise `KeyError` everywhere downstream. A typed empty frame means "new user with no transactions" flows through the same code path as a full one
③	Converts to `datetime64`, which unlocks the entire `.dt` accessor used below
④	`errors="coerce"` + `fillna` means a future caller bypassing Pydantic still cannot inject NaN into the statistics
⑤	**The most consequential line.** `groupby` silently *drops* NaN-keyed rows — a missing category would make transactions disappear from the analysis with no error. Coercing to "Other" guarantees every transaction is counted somewhere

Seven derived columns computed once, up front, are why no downstream service ever parses a date. `%G-W%V` is the correct ISO-8601 week format — `%Y-W%U` produces wrong week numbers around New Year.

```
def expenses_only(df): return df[df["type"] == "EXPENSE"].copy()
def month_span(df) -> int:  return max(1, df["year_month"].nunique()) if not df.empty else 1
```

`.copy()` is not decoration: without it pandas returns a view, and the anomaly service adds columns to `expenses`. `max(1, …)` guarantees the `total / months` division can never divide by zero.

19.1 The pandas vocabulary actually used

Operation	Where
`pd.DataFrame(list_of_dicts)` · `to_datetime` · `to_numeric` · `.dt.*` · `fillna` · `replace`	preprocessing
`df[df["type"] == "EXPENSE"]` boolean masking	preprocessing
`np.where(cond, a, b)`	trend_service
`.groupby(col).agg(name=(col, fn))`	trend, category
`.groupby(col)[c].transform("mean")`	anomaly, isolation_forest
`.reset_index()` · `.sort_values()` · `.iterrows()` · `.to_numpy()` · `.nunique()` · `df.loc[idx]`	throughout

19.2 The key idiom: `groupby().transform()`

```
agg()  → one row per group          transform() → one value per ORIGINAL row

  category  amount                        category  amount   cat_avg
  Food        500      agg("mean")        Food        500      750   ← Food's mean
  Food       1000      ──────────►        Food       1000      750
  Food        750      Food   750         Food        750      750
  Rent      18000      Rent 18000         Rent      18000    18000
```

`transform` is what makes the feature `deviation = amount − cat_avg` possible: it aligns a *group-level* statistic with *row-level* data in one vectorised operation, with no join and no loop.

19.3 One function end to end

```
INPUT  Food ₹100 · Food ₹300 · Bills ₹600   (all EXPENSE)
  ↓ to_dataframe        3 rows × 14 columns
  ↓ expenses_only       all 3 survive
  ↓ total = 1000.0
  ↓ groupby("category").agg(total,count,average,maximum)
      Bills 600.0 1 600.0 600.0    ·    Food 400.0 2 200.0 300.0
  ↓ sort_values("total", ascending=False)   → Bills first
OUTPUT  total_spending 1000.0 · largest_category "Bills"
        Bills 60.0 %  ·  Food 40.0 %
```

The test `test_large_dataset_performs` posts **5,000 transactions** to `/analytics/analyze` — summary, trends, categories and all three detectors — and asserts a 200. That is the practical evidence the pandas path scales to a realistic history.

PART 20 · CHAPTER IV

NumPy & SciPy

NumPy carries every statistic. SciPy is used for exactly one function. Both live in `app/analytics/statistics.py`, which imports no pandas and does no I/O — every function is `ndarray → number`, and therefore trivially testable.

NumPy call	Used for
`np.asarray(v, dtype="float64")`	Normalising input so the helpers accept lists, Series or arrays
`.mean()` · `np.median()` · `.std(ddof=0)` · `.min()/.max()/.sum()`	`describe`, `zscores`, `modified_zscores`
`np.percentile(values, [25, 75])`	`iqr_bounds` — both quartiles in one call
`np.abs()`	Absolute deviations for the MAD
`np.isnan()` · `np.isinf()`	`safe_float` — the JSON-safety guard
`np.zeros_like()`	Degenerate-case returns of the right shape
`np.arange()` · `np.where()`	The regression x-axis · vectorised conditional columns

```python
def zscores(values: np.ndarray) -> np.ndarray:
    values = np.asarray(values, dtype="float64")
    if values.size < 2:  return np.zeros_like(values)     # ① undefined for one value
    std = values.std(ddof=0)
    if std == 0:         return np.zeros_like(values)     # ② identical values → 0/0 = nan
    return (values - values.mean()) / std                 # ③ one expression, whole array
```

The vectorised form reads like the mathematical definition — which is what you want in statistical code you must audit — and runs in compiled C. Both guards return zeros meaning "nothing here is anomalous", which is the correct semantic: a subscription that is ₹199 every month is not an outlier, and without the guards it would produce NaN and break JSON serialisation.

20.1 SciPy — one function, deliberately

```python
from scipy import stats

def linear_trend(y: np.ndarray) -> float:
    if y.size < 2: return 0.0
    x = np.arange(y.size, dtype="float64")
    slope, _, _, _, _ = stats.linregress(x, y)     # discards intercept, rvalue, pvalue, stderr
    return safe_float(slope)
```

It saves writing `slope = Σ((xᵢ−x̄)(yᵢ−ȳ)) / Σ((xᵢ−x̄)²)` — implementable in three NumPy lines, but `linregress` is numerically stabler and makes the intent unmistakable.

> **HONEST ASSESSMENT OF THE DEPENDENCY**
> SciPy is large, justified by one call. That is a fair criticism. The defence: the discarded `rvalue` (goodness of fit) and `pvalue` (significance) are exactly what a next iteration would use to say "*spending is increasing, and the trend is statistically significant*". The dependency is paid for; the capability is one line away.

20.2 Which helpers are actually consumed

Helper	Consumer	Status
`safe_float` · `describe`	Nearly every service	USED
`modified_zscores` · `iqr_bounds`	The production detectors	USED
`linear_trend` · `pct_growth`	`build_trends`	USED
`severity_from_ratio` · `severity_from_isolation_score`	Z-score and forest detectors	USED
`zscores` (classic)	Only `tests/test_anomalies.py`	TEST ONLY
`severity_from_zscore`	Nothing	UNUSED

The last two rows matter for accuracy: **the production Z-score detector uses the MAD-based `modified_zscores` and maps severity from the amount/mean ratio**. Part 22 explains why that substitution was the right call.

PART 21 · CHAPTER V

What Is an Anomaly?

```
Your last seven Food transactions:
   ₹500   ₹700   ₹800   ₹1,000   ₹600   ₹750   ₹650      ← a pattern

Then this arrives:
   ₹8,000                                          ← does not belong to that pattern

Nothing is wrong with ₹8,000 — you may have chosen to pay for a team dinner.
The point is that it is different, and you should see it now rather than
discover it at the end of the month.
```

21.1 Why "unusual" must be relative

Transaction	Amount	Unusual?
Rent	₹18,000	**No.** It is ₹18,000 every month — the most predictable row in the ledger
Food (team dinner)	₹8,000	**Yes.** Your Food transactions live between ₹150 and ₹1,200

A detector comparing everything to one global average does precisely the wrong thing on both. The global mean sits around ₹1,240, so rent looks like a massive outlier every month — a false positive that trains the user to ignore alerts — while ₹8,000 looks only moderately above average and may not clear the threshold at all.

> **THE SINGLE MOST IMPORTANT DESIGN DECISION IN THE DETECTOR**
>
> **Statistics are computed within category groups, never globally.** Both the Z-score and IQR detectors begin with `for category, group in expenses.groupby("category")` . Rent is compared to rent; food to food. This is why FinSight flags the ₹8,000 dinner and stays silent about the rent — and it is the answer when an interviewer asks what makes your anomaly detection non-trivial.

21.2 Why it is useful, and the three detectors

Scenario	What the alert gives the user
Fraud	A charge that matches none of your habits, surfaced within a day
Billing errors	A subscription that silently renewed at 10× its usual price
Data-entry mistakes	₹45,000 typed where ₹4,500 was meant
Awareness	"This was a big one" — no judgement. Which is why a user can mark an anomaly Reviewed or Ignored rather than only dismiss it

Method	Question it asks	Grouping	Blind spot
Z-score (MAD)	How many robust standard deviations from this category's centre?	Per category	Univariate — sees only the amount
IQR	Outside this category's typical middle range?	Per category	Also univariate; needs ≥4 points
Isolation Forest	Across 8 dimensions, how easily can this point be isolated?	Whole dataset	Less interpretable; always flags ~contamination%

The first two catch "too big for this category" — cheap and explainable. The third catches combinations no single threshold can: a moderate amount, on an unusual day, in a category you rarely use. Running all three and merging gives coverage none provides alone.

PART 22 · CHAPTER V

Z-Score — and the MAD Variant Actually Used

$$z = \frac{x - \mu}{\sigma}$$

x this transaction's amount
μ the mean of the group
σ the standard deviation
z how many σ from the centre

A Z-score is unit-free: it turns "₹8,000 in a group averaging ₹768" into "7.4 standard deviations above centre", which is comparable across categories of wildly different scales.

22.1 Why the classic Z-score fails here

```
Food: [500, 700, 800, 900, 1000, 600, 750, 8000]

μ = 13,250 / 8 = 1,656.25        σ ≈ 2,412.8

z(8000) = (8000 − 1656.25) / 2412.8 ≈ +2.63   ← below the threshold of 3!
```

> **MASKING**
> The obvious outlier scores only 2.63 because **it inflated both μ and σ** — pulling the mean from ~750 to 1,656 and blowing σ up to 2,413. The outlier hid itself inside the very statistics used to detect it. This is the fundamental weakness of the classic Z-score for outlier detection.

22.2 The fix — the modified (MAD) Z-score

```
MAD = median( |xᵢ − median(x)| )

               0.6745 · (x − median)
      z_mod = ──────────────────────
                        MAD

0.6745 = Φ⁻¹(0.75), the constant making MAD a consistent estimator of σ for
normal data — so z_mod = 3 means roughly what a classic z = 3 means.
```

```python
def modified_zscores(values: np.ndarray) -> np.ndarray:
    """Median-absolute-deviation based score — robust to the outliers we hunt for."""
    values = np.asarray(values, dtype="float64")
    if values.size < 2:  return np.zeros_like(values)
    median = np.median(values)                      # resistant centre
    mad = np.median(np.abs(values - median))        # resistant spread
    if mad == 0:         return np.zeros_like(values)   # identical values → nothing unusual
    return 0.6745 * (values - median) / mad
```

```
Same data, recomputed:

sorted:  500  600  700  750  800  900  1000  8000
median = (750 + 800) / 2 = 775            ← the 8,000 barely moves it

|xᵢ − 775| sorted: 25  25  75  125  175  225  275  7225
MAD = (125 + 175) / 2 = 150               ← the 8,000 does not inflate it

z_mod(8000) = 0.6745 × (8000 − 775) / 150 ≈ +32.5   ✓ flagged
z_mod(500)  ≈ −1.24        z_mod(1000) ≈ +1.01              ✓ normal
```

2.63 → 32.5. The outlier no longer hides. `test_zscore_flags_strong_outlier` asserts exactly this: the classic score clears 2, the modified score clears 3.

22.3 The production detector

```python
def _zscore_anomalies(expenses: pd.DataFrame, threshold: float) -> list[AnomalyItem]:
    items = []
    for category, group in expenses.groupby("category"):              # ① per category
        if len(group) < settings.min_group_size:                      # ② need ≥ 5 points
            continue
        amounts = group["amount"].to_numpy(dtype="float64")
        z = modified_zscores(amounts)                                 # ③ robust
        mean = float(amounts.mean())                                  # ④ for the explanation
        for (_, row), zi in zip(group.iterrows(), z):
            if abs(zi) <= threshold:                                  # ⑤ two-tailed
                continue
            ratio = row["amount"] / mean if mean else float("inf")
            severity = severity_from_ratio(ratio)                     # ⑥
            if row["amount"] > mean:
```

```
                reason = (f"{_money(row['amount'])} is about {ratio:.1f}x higher than your "
                          f"average {category} expense ({_money(mean)}).")
            else:
                reason = (f"{_money(row['amount'])} is unusually low for {category} "
                          f"(you normally spend around {_money(mean)}).")
            items.append(_row_item(row, DetectionMethod.ZSCORE,
                                   safe_float(-abs(zi)), severity, reason))   # ⑦
    return items
```

	Explanation
②	`min_group_size = 5`. With four Food transactions a median and MAD are noise. This is what prevents a flood of meaningless alerts for a new user. `test_zscore_skips_small_groups` pins it: ₹500 and ₹20,000 alone produce **zero** anomalies
④	The *mean* is computed separately purely for the explanation. "10.4× higher than your average" is a sentence a user understands; "a modified Z-score of 32.5" is not
⑤	`abs(zi)` makes it two-tailed — unusually *low* amounts are flagged too. A ₹5 Rent transaction is as suspicious as a ₹50,000 one
⑦	`-abs(zi)` — the score is **negated** so lower always means more anomalous, matching scikit-learn's `score_samples` convention. This is what lets all three detectors sort in one list, and what Django's model comment records: "*Lower = more anomalous (engine convention).*"

22.4 Why the threshold is 3, and what it means

```
For normally distributed data (the empirical rule):

   |z| > 2   →  ~4.6  % of points     ~9 alerts per 200 transactions — noisy
   |z| > 3   →  ~0.27 % of points     ~1 per 370 — rare enough to mean something
   |z| > 4   →  ~0.006 % of points    misses genuine outliers
```

Three σ is the long-standing convention for "unusual enough to investigate", balancing the two error types: too low floods the user; too high misses the ₹8,000 dinner. In FinSight it is `ZSCORE_THRESHOLD` — tunable per deployment without a code change. In plain language, `z > 3` means "this amount is further from your typical spending in this category than 99.7% of your transactions would be, if your spending were normally distributed."

22.5 Limitations

Limitation	Consequence and response
Assumes normality	Spending is right-skewed, so the 0.27% figure is approximate. Mitigated by the robust variant and by treating the threshold as tunable rather than a guarantee
Masking	**Solved** by `modified_zscores`
Univariate	Sees only the amount — cannot notice a normal-sized transaction at an unusual time in a rarely-used category. **This is exactly why the Isolation Forest exists**
Needs enough data	Handled by `min_group_size = 5`: small groups are skipped, not guessed at
Zero MAD	A perfectly constant category (₹199 Spotify) would divide by zero. The guard returns zeros — correctly treating a regular bill as unremarkable
No temporal awareness	A gradual upward drift never produces a large Z-score because the baseline drifts with it. Trend analysis covers this separately

CATEGORY-SPECIFIC VS GLOBAL — THE CONCRETE NUMBERS

Approach	₹18,000 Rent	₹8,000 Food	Result
Global · μ≈1,240 σ≈3,100	z ≈ +5.4	z ≈ +2.2	**Wrong on both** — flags rent monthly, misses the dinner
Per category	z = 0 (MAD guard)	z ≈ +32.5	**Right on both**

PART 23 · CHAPTER V

IQR — Tukey Fences

```
Sort the values:
    Q1 = 25th percentile · Q2 = median · Q3 = 75th percentile
    IQR = Q3 − Q1            the range of the middle half of your data

    Lower = Q1 − 1.5 × IQR      Upper = Q3 + 1.5 × IQR

    ├──────┼═══════════════════┼──────┤
  Lower     Q1       median      Q3      Upper
            └──────── IQR ──────┘
   ◄ outlier                          outlier ►

Makes NO assumption about the shape of the distribution.
```

Numerical example

```
Food: [100, 120, 130, 140, 150, 160, 170, 5000]    (n = 8)

Q1 ≈ 127.5   Q3 ≈ 162.5   IQR = 35
Lower = 127.5 − 52.5 = 75        Upper = 162.5 + 52.5 = 215

100 … 170  inside [75, 215]  ✓ normal
5000       far above 215     ✗ OUTLIER

The 5,000 did NOT distort Q1 or Q3 — quartiles are positional, so a single
extreme value cannot move them. Same robustness as the median.
```

23.1 Implementation

```python
def iqr_bounds(values, multiplier: float = 1.5) -> tuple[float, float, float, float]:
    """Return (q1, q3, lower_bound, upper_bound)."""
    values = np.asarray(values, dtype="float64")
    if values.size < 4:                                  # quartiles need ≥ 4 points
        lo = safe_float(values.min()) if values.size else 0.0
        hi = safe_float(values.max()) if values.size else 0.0
        return lo, hi, lo, hi                            # bounds = data range → nothing flagged
    q1, q3 = np.percentile(values, [25, 75])
    iqr = q3 - q1
    return (safe_float(q1), safe_float(q3),
            safe_float(q1 - multiplier * iqr), safe_float(q3 + multiplier * iqr))
```

```python
for _, row in group.iterrows():
    amt = row["amount"]
    if lower <= amt <= upper:
        continue
    distance = (amt - upper) / iqr if amt > upper else (lower - amt) / iqr   # in IQRs
    severity = (Severity.CRITICAL if distance >= 3 else Severity.HIGH if distance >= 2
                else Severity.MEDIUM if distance >= 1 else Severity.LOW)
    side = "above" if amt > upper else "below"
    reason = (f"{_money(amt)} is well {side} your typical {category} range "
              f"({_money(max(lower, 0))}–{_money(upper)}, median {_money(median)}).")
    items.append(_row_item(row, DetectionMethod.IQR, safe_float(-distance), severity, reason))
```

Distance measured in IQRs beyond the fence converts a raw rupee overshoot into a scale-free quantity, so ₹5,000 over a ₹35 IQR and ₹50,000 over a ₹350 IQR are treated identically. `iqr = max(q3 - q1, 1e-9)` guards a zero IQR; `max(lower, 0)` prevents printing a nonsensical negative lower bound. `test_iqr_bounds_formula` verifies the arithmetic directly.

Why 1.5? Tukey's constant. For normal data the fences sit at roughly ±2.7σ, capturing ~99.3% — deliberately close to the 3σ convention. Tunable via `IQR_MULTIPLIER`.

23.2 Z-score vs IQR

Dimension	Modified Z-score (MAD)	IQR (Tukey)
Basis	Distance from the median in MAD units	Position relative to quartile fences
Distribution assumption	Approximately normal for the 3σ reading	**None** — purely positional
Minimum data	2 in the helper; 5 enforced by the detector	4 in the helper; 5 enforced
Output	Continuous score — *how* unusual	Binary in/out plus a distance
Skewed data	Symmetric around the median; can over-flag the long tail	Naturally asymmetric — Q1 and Q3 adapt independently
Explanation produced	"about 10.4× higher than your average Food expense (₹768)"	"well above your typical Food range (₹75–₹215, median ₹140)"

Why run both. IQR is the better fit for skewed spending data; the MAD Z-score gives a continuous magnitude that ranks findings sensibly. Keeping the more severe verdict means a transaction must look normal to *both* univariate tests to pass unflagged — and the two produce different, complementary sentences.

PART 24 · CHAPTER V

Isolation Forest

Most anomaly detection models what *normal* looks like and measures distance from it. Isolation Forest inverts the question: **how hard is this point to separate from everything else?**

```
Pick a random feature, a random split value, cut the data in two. Repeat on the
half containing your point until it is alone.

   Normal point (dense cluster)        Outlier (sparse region)
        · ·:·:· ·                              ·
       ·:·X·:·:·                      · ·:·:·          X
        ·:· ·:·                          ·:·:·:·

   Many cuts to isolate → deep         One or two cuts → shallow
   → normal                            → anomalous

Average path length across 200 random trees = the anomaly signal.
```

Isolating an outlier is *cheap* precisely because it sits in a sparse region where a random split is likely to separate it early. No distance metric, no density estimate, no distributional assumption — and it works in many dimensions at once, which is exactly what the univariate detectors cannot do.

24.1 The vocabulary

Term	Meaning in FinSight
`n_estimators = 200`	Random trees. More trees → a more stable average path length
`contamination = 0.05`	Expected proportion of anomalies. **It sets the decision threshold** — the model calibrates its cut-off so ~5% of the input is labelled −1. A prior, not a discovery
`prediction = −1 / +1`	scikit-learn's labels for anomaly / normal
`score_samples()`	Continuous score, roughly [−0.5, 0]. **Lower = more anomalous** — the convention adopted by all three detectors
`random_state = 42`	Fixes the randomness so the same payload always yields the same findings

> **THE CONSEQUENCE OF `CONTAMINATION` YOU MUST UNDERSTAND**
>
> Because contamination fixes the threshold, **the forest always labels roughly 5% of the input as anomalous** — even on perfectly uniform data where nothing is genuinely unusual. It is a relative ranking, not an absolute judgement. FinSight handles this with an explicit suppression filter. Knowing this is the difference between having used Isolation Forest and having understood it.

24.2 Implementation — `app/ml/isolation_forest.py`

```python
def detect(expenses: pd.DataFrame, contamination: float = 0.05,
           random_state: int = 42) -> pd.DataFrame:
```

```python
    if len(expenses) < 8:                                          # ①
        return pd.DataFrame(columns=["prediction", "score", "severity"])

    features = _engineer_features(expenses)                        # ② 8 features
    X = StandardScaler().fit_transform(                            # ③ essential
        features[FEATURE_COLUMNS].to_numpy(dtype="float64"))

    model = IsolationForest(n_estimators=200,
                            contamination=min(max(contamination, 0.01), 0.5),   # ④ clamp
                            random_state=random_state)
    model.fit(X)                                                   # ⑤ fitted per request

    predictions = model.predict(X)          # -1 anomaly, 1 normal
    scores      = model.score_samples(X)    # lower = more anomalous

    out = pd.DataFrame(index=expenses.index)                       # ⑥ index alignment
    out["prediction"] = predictions
    out["score"]      = [safe_float(s) for s in scores]
    out["severity"]   = [severity_from_isolation_score(s).value for s in scores]
    return out
```

	Explanation
①	Below 8 rows the trees cannot build meaningful structure. `test_isolation_forest_too_few_rows` covers this
③	**`StandardScaler` is load-bearing.** Raw features span wildly different ranges (amount 100–45,000; day_of_week 0–6). Random split points would almost always land in the amount dimension, and the model would degenerate into a univariate amount detector. Scaling to mean 0 / variance 1 gives every feature an equal chance of being split on
⑤	Fitted on the same data it scores — standard for unsupervised outlier detection: there are no labels, and the task is "find the odd ones in *this* set"
⑥	`index=expenses.index` is what allows `expenses.loc[idx]` in the caller. Getting this wrong would silently attach reasons to the wrong transactions

24.3 The suppression filter — the most interesting lines

```python
cat_avg = expenses.groupby("category")["amount"].transform("mean")
cat_std = expenses.groupby("category")["amount"].transform("std").fillna(0.0)
for idx, res in result.iterrows():
    if res["prediction"] != -1:
        continue
    row = expenses.loc[idx]
    avg = float(cat_avg.loc[idx]) or 1.0
    ratio = row["amount"] / avg
    # Suppress noise: the forest always isolates ~contamination% of rows. Keep only
    # findings that are also strongly scored or materially off the category norm.
    near_constant = float(cat_std.loc[idx]) <= 0.01 * avg
    if near_constant or (res["score"] > -0.12 and 0.7 <= ratio <= 1.5):
        continue
    severity = severity_from_isolation_score(res["score"])
    if ratio >= 1.5:
        reason = (f"This {row['category']} transaction of {_money(row['amount'])} is unusual "
                  f"compared with your historical spending pattern "
                  f"(~{ratio:.1f}x your {row['category']} average).")
    else:
        reason = (f"This {row['category']} transaction stands out from your usual "
                  f"spending pattern (timing and amount combined).")
```

TWO SUPPRESSION RULES, EACH EARNING ITS PLACE

1 · `near_constant` — if a category's standard deviation is within 1% of its mean, every transaction in it is essentially identical (Netflix ₹199, rent ₹18,000). The forest may still isolate one on a timing feature, but calling a perfectly regular bill "unusual" destroys user trust. Suppressed unconditionally.

2 · weak score *and* normal amount — score milder than -0.12 *and* amount between $0.7\times$ and $1.5\times$ the category average is almost certainly the contamination quota being filled. Note the conjunction: a strong score survives even at a normal amount (the multivariate catch the forest exists for), and an extreme amount survives even at a weak score.

The two-branch reason mirrors the same logic: quote a number when the amount really is large; otherwise be honest that the signal is "*timing and amount combined*" rather than inventing a spurious explanation.

Feature Engineering

```
FEATURE_COLUMNS = ["transaction_amount", "category_encoded", "day_of_week", "day_of_month",
                   "month", "average_category_spending", "transaction_frequency",
                   "deviation_from_category_average"]

def _engineer_features(expenses: pd.DataFrame) -> pd.DataFrame:
    df = expenses.copy()
    cat_avg  = df.groupby("category")["amount"].transform("mean")    # broadcast group mean
    cat_freq = df.groupby("category")["id"].transform("count")       # broadcast group count

    features = pd.DataFrame(index=df.index)
    features["transaction_amount"] = df["amount"].astype("float64")
    features["category_encoded"]   = OrdinalEncoder(
        handle_unknown="use_encoded_value", unknown_value=-1).fit_transform(df[["category"]])
    features["day_of_week"]  = df["day_of_week"].astype("float64")
    features["day_of_month"] = df["day_of_month"].astype("float64")
    features["month"]        = df["month"].astype("float64")
    features["average_category_spending"] = cat_avg.astype("float64")
    features["transaction_frequency"]     = cat_freq.astype("float64")
    features["deviation_from_category_average"] = (df["amount"] - cat_avg).astype("float64")
    return features
```

#	Feature	What it represents and why it helps
1	`transaction_amount`	The raw magnitude — the primary signal. Everything else exists to give it context
2	`category_encoded`	Category as a number, so a tree can split on it. Lets the forest learn "large is normal *here* but not *there*" without explicit grouping
3	`day_of_week`	Weekly rhythm — weekend versus weekday spending
4	`day_of_month`	Monthly rhythm: salary on the 1st, rent on the 3rd, bills mid-month. **The feature that catches "right amount, wrong time"**
5	`month`	Seasonality — festival months, holiday travel. Weakest on six months of data, but costs nothing and matures with history
6	`average_category_spending`	**The context feature.** Every row carries its category's mean, so the forest compares a transaction to its own baseline — the multivariate equivalent of per-category grouping
7	`transaction_frequency`	A large amount in a category you use 80 times differs from the same amount in one you have used twice. Encodes "how well do we know your behaviour here"
8	`deviation_from_category_average`	**The most directly discriminative.** An explicit "how far from your own norm", so the model need not learn the relationship between features 1 and 6 itself. Signed, so unusually low amounts are represented too

ORDINAL, NOT ONE-HOT — AND WHY THAT IS ACCEPTABLE

Ordinal encoding implies an ordering that does not exist: Food(2) is not "between" Entertainment(1) and Groceries(3). For a linear model this would be a serious mistake. For a *tree* method it is much less harmful — trees split on thresholds and can carve out arbitrary subsets with successive splits. One-hot would be more correct but would explode 19 categories into 19 sparse columns, diluting the seven informative features. Given that features 6, 7 and 8 already inject category context numerically, this is a reasonable trade — and worth naming as a trade rather than presenting as ideal.

A worked feature vector

The ₹45,000 Apple Store purchase, where Shopping averages ₹3,620 across 34 transactions:

Feature	Raw value	Reading
transaction_amount	45,000.0	Very large
category_encoded · day_of_week · day_of_month · month	11.0 · 5.0 · 22.0 · 8.0	Shopping, Saturday, late August
average_category_spending · transaction_frequency	3,620.0 · 34.0	A well-established category
deviation_from_category_average	**+41,380.0**	**Enormously above the norm**

After scaling, features 1 and 8 sit far into the tail, so any random split on either isolates this row almost immediately → short average path → score ≈ −0.28 → **CRITICAL**, and `ratio = 45,000/3,620 = 12.4` produces the sentence quoted in Part 16.5.

PART 26 · CHAPTER V

Severity & Human-Readable Reasons

26.1 Three severity mappings, one per detector

There is no single severity function — each detector produces a differently-scaled quantity.

```
def severity_from_ratio(ratio: float) -> Severity:       # Z-score detector
    """ratio = amount / group-average."""
    if ratio >= 10: return Severity.CRITICAL     # ten times your normal
    if ratio >= 5:  return Severity.HIGH
    if ratio >= 3:  return Severity.MEDIUM
    return Severity.LOW

def severity_from_isolation_score(score: float) -> Severity:
    """sklearn score_samples: lower (more negative) = more anomalous."""
    if score <= -0.25: return Severity.CRITICAL
    if score <= -0.15: return Severity.HIGH
    if score <= -0.05: return Severity.MEDIUM
    return Severity.LOW
```

IQR maps from distance past the fence (≥3, ≥2, ≥1 IQRs). **Why ratio and not the Z-score?** Because the ratio is the number the message quotes — "10.4× higher than your average" alongside CRITICAL is consistent; "modified Z-score 32.5" teaches the user nothing. Severity and explanation are derived from the same quantity, deliberately. The Isolation Forest thresholds are empirical, tuned on the seed dataset — the least principled numbers in the system, and honest to describe as such.

26.2 Where the sentence is generated

ANSWERING THE QUESTION DIRECTLY

A message like "*₹45,000 is significantly higher than your normal shopping transactions*" is generated in `analytics-service/app/services/anomaly_service.py`, inside whichever detector flagged the transaction. It travels to Django as the `reason` field of `AnomalyItem`, is stored verbatim in `anomalies_anomaly.reason`, and is rendered unchanged by React. **Neither Django nor React ever composes or modifies this text** — the service that understands why a transaction was flagged is the only one that explains it.

```
def _money(value: float) -> str:  return f"₹{value:,.0f}"     # 45000.0 → "₹45,000"

# 1 · Z-score high   "₹8,000 is about 10.4x higher than your average Food expense (₹768)."
# 2 · Z-score low    "₹50 is unusually low for Rent (you normally spend around ₹18,000)."
# 3 · IQR            "₹5,000 is well above your typical Food range (₹75–₹215, median ₹140)."
# 4 · Forest, ratio ≥ 1.5   "…unusual compared with your historical spending pattern (~12.4x …)."
# 5 · Forest, normal amount "…stands out from your usual spending pattern (timing and amount combined)."
```

- **Personal baseline, always** — "your average", "your typical range". Never an absolute claim that ₹8,000 is a lot.
- **The comparison number is included** — "(₹768)" lets the user verify rather than trust.
- **No jargon** — no Z-scores or quartiles in user-facing text; those live in separate fields.
- **Honest when the signal is weak** — template 5 refuses to invent a numeric justification.

26.3 Merging — one row per transaction

```
_SEVERITY_RANK = {LOW: 0, MEDIUM: 1, HIGH: 2, CRITICAL: 3}
_METHOD_RANK   = {IQR: 0, ZSCORE: 1, ISOLATION_FOREST: 2}

def _merge_strongest(items):
    best = {}
    for item in items:
        current = best.get(item.transaction_id)
        if current is None or (_SEVERITY_RANK[item.severity], _METHOD_RANK[item.detection_method]) > \
                              (_SEVERITY_RANK[current.severity], _METHOD_RANK[current.detection_method]):
            best[item.transaction_id] = item
    return list(best.values())

if method == DetectionMethod.ALL:
    items = _merge_strongest(items)
    items = [i for i in items if i.severity != Severity.LOW]   # only what deserves attention
items.sort(key=lambda i: (i.anomaly_score, -_SEVERITY_RANK[i.severity]))
```

Ranked by **(severity, then method)**, preferring Isolation Forest > Z-score > IQR at equal severity because the multivariate finding carries more information. `test_all_methods_merge_to_one_row_per_txn` asserts the id appears exactly once. **LOW is dropped from the combined view** — the dashboard shows five alerts and they should be the five that matter; querying a single method still returns LOW findings. The ascending sort by score (lower = worse) matches Django's `ordering = ["anomaly_score", "-detected_at"]`.

PART 27 · CHAPTER V

Anomaly Data Flow

```
REACT     "Run detection" → POST /api/anomalies/detect/  {"method": "ALL"}
   ↓
DJANGO    AnomalyViewSet.detect → detect_and_store(user, "ALL")
            start = today − 180 days
            analytics.services.anomalies(user, "ALL", start)
              SELECT … WHERE account.user_id = 7 AND transaction_date ≥ start
              serialize_for_engine → 187 dicts
              cache key analytics:user:7:anomalies:ALL_…:9f2a1c88b410 → MISS
   ↓  HTTP POST · Bearer service token · 8 s timeout
FASTAPI   require_service_token → Pydantic AnomalyRequest → detect_anomalies()
            to_dataframe        187 rows × 14 columns
            expenses_only       181 rows
            ├─ ZSCORE   groupby category → modified_zscores → |z| > 3   → 2 items
            ├─ IQR      groupby category → Tukey fences               → 3 items
            └─ IFOREST  8 features → OrdinalEncoder → StandardScaler
                          → 200 trees → predict/score → suppression       → 2 items
            _merge_strongest    → 4 unique transactions
            drop LOW severity   → 3 remain
            sort by anomaly_score ascending
            200 {"analysed_count": 181, "anomaly_count": 3, "anomalies": [...]}
   ↓
DJANGO    cache.set(key, result, 900 s)
            valid_txn_ids = {ids belonging to user 7}          ← ownership guard
            for each item: skip if transaction_id ∉ valid_txn_ids
              Anomaly.objects.update_or_create(
                  user, transaction_id, detection_method,      ← the unique key
                  defaults={score, severity, reason, amount,
                            category, merchant, transaction_date})
              ← status and reviewed are NOT in defaults, so a prior decision survives
            prune: DELETE stale OPEN findings, for the methods re-run only
   ↓
POSTGRES  anomalies_anomaly — current findings + historical reviewed/ignored rows
   ↓
REACT     200 {"detected": 3, …} → toast → invalidatesTags ["Anomaly"] → table repaints
          Review / Ignore / Reopen → POST → status change → refetch
```

27.1 The persistence step

```python
def detect_and_store(user, method: str = "ALL", lookback_days: int = 180) -> list[Anomaly]:
    start = date.today() - timedelta(days=lookback_days)
    engine_result = analytics_services.anomalies(user, method=method, start=start)
    items = engine_result.get("anomalies", []) if isinstance(engine_result, dict) else []   # ①

    valid_txn_ids = set(Transaction.objects.filter(account__user=user)
                                           .values_list("id", flat=True))                   # ②
    stored, seen = [], set()
    for item in items:
        txn_id = item.get("transaction_id")
        if txn_id not in valid_txn_ids:                                                     # ③
            continue
        detection_method = item.get("detection_method", "ZSCORE")
        obj, _ = Anomaly.objects.update_or_create(                                          # ④
            user=user, transaction_id=txn_id, detection_method=detection_method,
            defaults={"anomaly_score": float(item.get("anomaly_score", 0.0)),
                      "severity": item.get("severity", "LOW"),
                      "reason": item.get("reason", ""), "amount": item.get("amount", 0),
                      "category": item.get("category", "") or "",
                      "merchant": item.get("merchant", "") or "",
```

```
                    "transaction_date": parse_date(item["date"]) if item.get("date") else None})
        stored.append(obj); seen.add((txn_id, detection_method))

    methods_run = ["ZSCORE","IQR","ISOLATION_FOREST"] if method == "ALL" else [method]      # ⑤
    stale = Anomaly.objects.filter(user=user, status="OPEN",
                                   detection_method__in=methods_run) \
                           .exclude(transaction_id__in=[t for t, _ in seen])
    stale.delete()
    return stored
```

	Explanation
①	Defensive parsing with `.get()` and an `isinstance` check. Django never assumes the engine's response shape, even though Pydantic guarantees it — cheap insurance across a network boundary
②③	**The security boundary.** Any finding referencing an unknown id is silently dropped, so even a compromised engine cannot create an anomaly against another user's transaction. **Django validates everything it receives back**
④	`update_or_create` on the model's unique constraint. Re-running **refreshes the score and reason but never touches `status` or `reviewed`**, because those are not in `defaults`. A user's decision survives every subsequent run — `test_detect_is_idempotent_and_preserves_review_state` proves it
⑤	**Pruning, carefully scoped.** Only `OPEN` findings, and only for the methods actually re-run. Reviewed and Ignored rows survive as an audit trail; running IQR alone cannot delete Z-score findings

27.2 The two entry points to detection

Path	Persists?	When
`POST /api/anomalies/detect/`	**Yes**	User clicks "Run detection" — upserts and prunes
`GET /api/analytics/anomalies/`	No	Live view, no side effects
`GET /api/dashboard/`	No	Up to 5 alerts; failure sets `analytics_available: false`
`GET /api/analytics/analyze/`	No	Part of the composite Analytics payload

Only the explicit `detect/` action writes. Reading the dashboard never mutates the anomaly table — a clean separation between "show me" and "record this".

PART 28 · CHAPTER VI

Complete Request Lifecycles

Four end-to-end traces. If you can narrate these from memory, you can answer almost any "walk me through your system" question.

28.1 User login

```
① REACT     LoginPage.onSubmit → react-hook-form validates → login({email,password}).unwrap()
            baseQuery: no token yet, so no Authorization header

② HTTP      POST /api/auth/login/   {"email":"demo@finsight.app","password":"DemoPass123"}
            (dev: Vite proxy from :5173 · Docker: nginx proxy)

③ DJANGO    LoginView (TokenObtainPairView) · AllowAny · throttle_scope "auth" 20/min
            FinSightTokenObtainPairSerializer.validate → authenticate(email, password)

④ DATABASE  SELECT * FROM users_user WHERE email = 'demo@finsight.app'
            check_password → PBKDF2-SHA256 re-derived with the stored salt,
            constant-time compare  ✓   (no row → Django still runs a dummy hash,
                                        so timing does not reveal existence)

⑤ JWT       RefreshToken.for_user(user)
            access  = HS256({user_id:7, exp:+30min, jti, name, email}, JWT_SECRET_KEY)
            refresh = HS256({user_id:7, exp:+7days,  jti},             JWT_SECRET_KEY)
            validate() appends UserSerializer(self.user).data

⑥ RESPONSE  200 {"success": true, "data": {"access":"eyJ…","refresh":"eyJ…","user":{…}},
                 "message": "Request successful", "error_code": null}

⑦ REACT     baseQuery unwraps → dispatch(credentialsReceived) → authSlice persists
            to localStorage → toast → navigate("/", {replace:true})
            RequireAuth re-evaluates → DashboardLayout mounts → dashboard query fires
            every later request carries Authorization: Bearer eyJ…

FAILURE     wrong password → 401 {"success": false,
                 "message": "No active account found with the given credentials",
                 "error_code": "no_active_account"} → inline <Alert> on the form
            (no refresh-retry loop, because there is no refresh token yet)
```

28.2 Add a transaction

```
① FORM      ₹2,500 · Food · Expense · HDFC Savings · today
            react-hook-form: amount > 0, account, category, date all required
            the category dropdown is already filtered to EXPENSE categories

② API       await mutex.waitForUnlock() → Authorization header → POST /api/transactions/

③ DJANGO    JWTAuthentication → IsAuthenticated → TransactionViewSet.create
            TransactionSerializer.__init__ narrows the FK querysets to the user's own
               → a forged account id fails here with 400, never reaching a permission check
            validate_amount · validate_transaction_date · validate (type ↔ category)

④ SERVICE   services.create_transaction(**validated_data)
            BEGIN
              SELECT … FROM accounts_account WHERE id=2 FOR UPDATE   ← row locked
              INSERT INTO transactions_transaction (…)               → id 412
              signed_amount = -2500.00
              50,000.00 + (-2,500.00) = 47,500.00
              UPDATE accounts_account SET balance=47500.00, updated_at=now() WHERE id=2
            COMMIT                                                    ← lock released
            log: txn_created id=412 account=2 type=EXPENSE delta=-2500.00

⑤ RESPONSE  201 with the read-only expansions account_name / category_name /
            category_icon / currency, wrapped in the envelope
```

```
⑥ REACT     toast → dialog closes
            invalidatesTags: Transaction · Account · Dashboard · Analytics · Anomaly
            → mounted queries refetch → table repaints, account card shows ₹47,500

FAILURE    income category on an expense → 400 with the field error lifted into
             "message" → error toast, dialog stays open, nothing written, balance untouched
```

28.3 View the dashboard

```
① REACT     useGetDashboardQuery() — cache hit and fresh? render instantly, no request

② DJANGO    DashboardView · throttle "analytics" 60/min
            DashboardQuerySerializer defaults year/month to today

③ LOCAL     five sections, pure Django ORM + SQL:
            local_month_summary()      SUM(amount) income · expenses · SUM(balance)
                                       savings = income − expenses; rate = savings/income×100
            local_spending_trend(6)    6 × (SUM expense, SUM income)
            local_category_breakdown() .values("category__name").annotate(Sum, Count)
            top_expenses(5)            ORDER BY amount DESC LIMIT 5
            recent_transactions(8)     ORDER BY transaction_date DESC LIMIT 8

④ ENRICH    try:  anomaly_alerts = anomalies(user,"ALL", month_start − 180d)["anomalies"][:5]
            except AnalyticsUnavailable:
                  analytics_available = False; log dashboard_served_without_analytics

⑤ FASTAPI   token → Pydantic → 3 detectors → merge → drop LOW → sort → 200

⑥ DJANGO    cache.set(key, result, 900 s) → 200 with all seven sections

⑦ REACT     4 StatCards · trend chart · income-vs-expense · breakdown ·
            top expenses · recent transactions · AnomalyAlerts(alerts, available)

DEGRADED   FastAPI down → step ④ catches → still 200, not 503
             analytics_available: false, anomaly_alerts: []
             → the alert strip renders "Analytics are temporarily unavailable.
               Your transactions are safe." Everything else is unaffected.
             → asserted by test_dashboard_survives_analytics_outage
```

28.4 Detect anomalies

Traced in full in Part 27. In one line: `POST /anomalies/detect/` → Django loads 180 days → cached call to the engine → three detectors, merge, drop LOW → Django validates ownership, upserts on `(user, transaction, method)` preserving review state, prunes stale OPEN rows → 200 `{detected, anomalies}` → RTK Query invalidates the `Anomaly` tag → the table repaints. On engine failure: **503**, an amber toast rather than red, and the previously stored anomalies remain on screen.

PART 29 · CHAPTER VI

The Database Request Flow, Layer by Layer

One request — `GET /api/transactions/?type=EXPENSE&start_date=2026-08-01` — through every layer.

```
1 · REACT       useListTransactionsQuery({page, page_size, type, start_date, ordering})
                 cleanParams() strips empty values; RTK Query builds a cache key
                 cache hit and fresh → return immediately

2 · HTTP        GET /api/transactions/?page=1&page_size=10&type=EXPENSE
                     &start_date=2026-08-01&ordering=-transaction_date
                 Authorization: Bearer eyJ…

3 · MIDDLEWARE  CORS → Security (nosniff, referrer-policy, HSTS in prod) → WhiteNoise
                 → Session → Common → CSRF → Auth → Messages → XFrameOptions

4 · URLS        config/urls.py "api/" → transactions/urls.py → router → viewset "list"

5 · DRF         JWTAuthentication: verify HS256 · check exp · SELECT user WHERE id=7
```

```
                IsAuthenticated ✓ · no throttle scope on this view

6 · VIEW        get_queryset(): filter(account__user=me).select_related("account","category")
                DjangoFilterBackend → transaction_type='EXPENSE', date >= '2026-08-01'
                OrderingFilter      → ORDER BY transaction_date DESC
                paginator           → LIMIT 10 OFFSET 0

7 · ORM → SQL   the queryset is lazy; slicing by the paginator forces evaluation

     SELECT t.id, t.amount, t.transaction_type, t.description, t.merchant,
            t.transaction_date, t.created_at, t.updated_at,
            a.id, a.account_name, a.currency, c.id, c.name, c.icon
     FROM transactions_transaction t
     INNER JOIN accounts_account    a ON t.account_id  = a.id
     INNER JOIN categories_category c ON t.category_id = c.id
     WHERE a.user_id = 7 AND t.transaction_type = 'EXPENSE'
       AND t.transaction_date >= '2026-08-01'
     ORDER BY t.transaction_date DESC LIMIT 10 OFFSET 0;

     plus one COUNT(*) with the same WHERE, for the paginator's total

8 · POSTGRES    parse → plan → index scan on (account, transaction_date) → joins
                → sort → LIMIT → rows over the wire to psycopg3

9 · ORM         psycopg3 → Python types (NUMERIC → Decimal, date → datetime.date)
                select_related also builds the Account and Category objects — no extra queries

10 · SERIALIZER direct fields + source-mapped (account.account_name → account_name, …)
                Decimal("2500.00") → "2500.00" (string, to preserve precision)
                date(2026,8,28)    → "2026-08-28"

11 · PAGINATOR  {"results":[…], "count":34, "page":1, "page_size":10,
                 "total_pages":4, "next":"…?page=2", "previous":null}

12 · RENDERER   EnvelopeJSONRenderer wraps it in {success, data, message, error_code}

13 · REACT      baseQuery unwraps → cached under the key, tagged "Transaction"
                isLoading → false → rows render, formatCurrency prints "₹2,500"
```

29.1 What each layer owns

Layer	Owns	Must never do
React component	Presentation, interaction	Build URLs or call fetch directly
RTK Query	Caching, dedup, loading/error state, invalidation	Contain business logic
baseQuery	Auth header, token refresh, envelope unwrapping	Know about specific endpoints
Middleware	CORS, security headers, sessions	Touch business data
Authentication	Establishing *who* the caller is	Decide what they may do
Permission	Deciding *what* they may do	Fetch or filter data
ViewSet	Queryset scoping, filter wiring, HTTP semantics	Contain balance arithmetic
Serializer	Validation, field mapping, type conversion	Perform multi-step business operations
Service	Business rules, atomicity, locking	Know about HTTP or requests
ORM	Python ↔ SQL, lazy evaluation	Be bypassed with raw SQL without reason
PostgreSQL	Storage, constraints, isolation, durability	Contain application logic

The one place the layering is deliberately bent: `TransactionSerializer.create()` calls `services.create_transaction()`. The serializer is the natural DRF write hook and the service is where atomicity lives. Documenting that seam is better than pretending the layering is pure.

PART 31 · CHAPTER VI

Error Handling

Four layers with four different jobs: FastAPI contains, Django translates, the envelope normalises, React presents.

31.1 FastAPI — contain

A bad payload is a 422 with detail; anything unexpected is a 500 with **no internal detail in the body** and a full traceback in the logs. Nothing about the engine's internals crosses the service boundary.

31.2 Django — translate

```
# five failure modes → one exception  (analytics/client.py)
except (httpx.TimeoutException, httpx.TransportError):   raise AnalyticsUnavailable()
if response.status_code >= 500:                          raise AnalyticsUnavailable()
if response.status_code >= 400:                          raise AnalyticsUnavailable("… rejected …")
except ValueError:                                       raise AnalyticsUnavailable("… invalid …")

# one place to translate it  (analytics/views.py)
except AnalyticsUnavailable as exc:
    payload = dict(UNAVAILABLE_PAYLOAD); payload["message"] = exc.message or payload["message"]
    return Response(payload, status=503)
```

Status	Source	Body
400	Serializer validation	Field errors in `data`, the first lifted into `message`
401	JWTAuthentication	Missing / invalid / expired — triggers the frontend refresh flow
403	Permission class	e.g. writing to a default category
404	Queryset scoping	Not found *or* not yours — deliberately indistinguishable
429	ScopedRateThrottle	auth 20/min or analytics 60/min exceeded
503	AnalyticsUnavailable	`error_code: "ANALYTICS_UNAVAILABLE"`

All of them pass through `envelope_exception_handler`, so every error has the same four keys and a human sentence in `message`.

31.3 The resilience rule

```
# analytics/services.py — build_dashboard()
data = {"summary": local_month_summary(...), "spending_trend": local_spending_trend(...),
        "category_breakdown": local_category_breakdown(...), "top_expenses": top_expenses(...),
        "recent_transactions": recent_transactions(user),
        "analytics_available": True, "anomaly_alerts": []}
try:
    engine = anomalies(user, method="ALL", start=date(year, month, 1) - timedelta(days=180))
    data["anomaly_alerts"] = engine.get("anomalies", [])[:5]
except AnalyticsUnavailable:
    data["analytics_available"] = False
    logger.info("dashboard_served_without_analytics user_id=%s", user.id)
```

WHY ANALYTICS FAILURE MUST NEVER BLOCK FINANCIAL OPERATIONS

Different consequences. If Django fails, a user cannot record that they spent money — their ledger becomes wrong, and wrong ledgers compound. If FastAPI fails, they do not see this week's insight. One is data loss; the other is a delayed convenience.

Different trust implications. A finance app that shows an error page because a statistics service is down looks broken. One that shows every number plus a small note looks robust.

Different failure rates. The analytics service does heavier work with more dependencies; it *will* fail more often. Coupling the core experience to the least reliable component is backwards. **The rule:** the dashboard is resilient by design; the dedicated Analytics and Anomalies pages may legitimately fail, because the user went there specifically for analytics.

31.4 React — present

Mechanism	Behaviour
`errorMessage(error, fallback)`	Digs out `data.message` → `data.detail` → `error` → fallback. One helper for every error in the app
401 auto-refresh	Invisible, handled by `baseQuery`. The user only notices if the refresh also fails, which triggers `loggedOut()` and the route guard
Inline `<Alert>`	Login and Register — the error belongs next to the form
Toast (notistack)	Every mutation. Success green, failure red, **analytics failure amber**
`<ErrorState onRetry={refetch}>`	Every failed page query — always offers a retry
Degraded rendering	`AnomalyAlerts` takes an `available` prop and renders a note instead of failing
Best-effort logout	Wrapped in `try/catch` that swallows everything — a network error must never trap a user in a signed-in state

PART 41 · CHAPTER VI

Diagram Gallery

Consolidated reference. Figures 1–3 appear in Parts 1, 3 and 10.

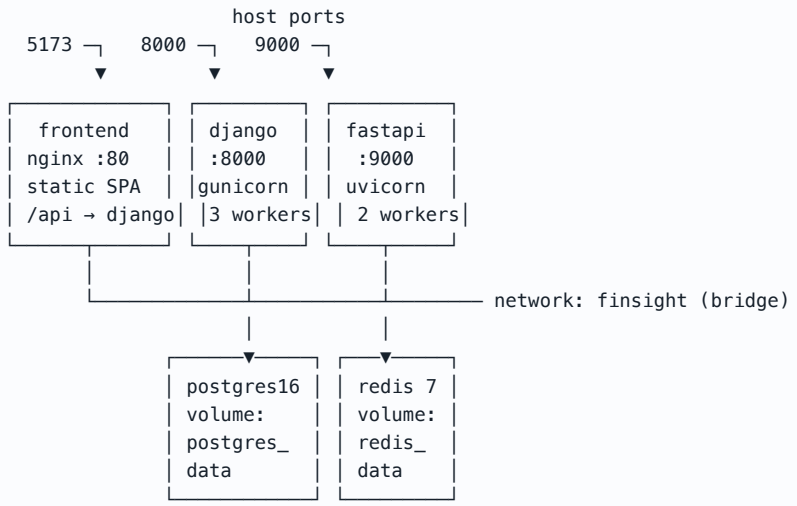
Authentication flow

```
Browser ──credentials──► React ──POST /auth/login/──► Django
                                                        │
                                          authenticate(email, password)
                                                        │
                                          SELECT user WHERE email = …
                                          PBKDF2-SHA256 verify (constant time)
                                                        │
                                          RefreshToken.for_user(user)
                                          access 30 min · refresh 7 days · HS256
                                                        │
React ◄──{access, refresh, user}──────────────────────┘
  │
  ├─ authSlice → localStorage["finsight.auth"]
  └─ every request: Authorization: Bearer <access>
         │
         ├─ 200 → done
         └─ 401 → mutex.acquire() → POST /auth/refresh/ → tokenRefreshed → replay
                  refresh fails → loggedOut() → RequireAuth redirects to /login
```

Analytics flow, including the cache

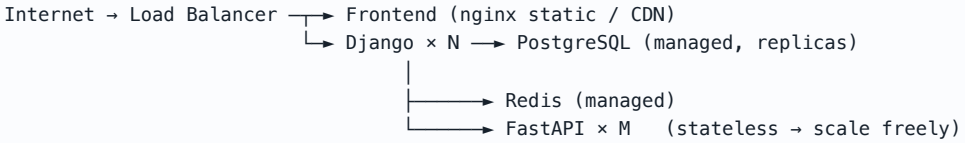
```
React ──GET /api/analytics/summary/──► Django
                                        ├─ DateRangeSerializer validates the params
                                        ├─ SELECT transactions WHERE account.user_id = 7
                                        ├─ serialize_for_engine → [{id, amount, category, …}]
                                        ├─ key = analytics:user:7:summary:{range}:{sha1[:12]}
                                        ├─ cache.get(key) ── HIT ──► return (no HTTP at all)
                                        └─ MISS ↓
                                           httpx.post(FASTAPI_URL + path,
                                             headers={Authorization: Bearer …},
                                             timeout=8s)
                                                │
                                        FastAPI: token → Pydantic → pandas/NumPy → 200
                                                │
                                        cache.set(key, result, 900s) → Response
                                                │
                    timeout / 5xx / 4xx / bad JSON → AnalyticsUnavailable → 503
```

Docker architecture



Service names are hostnames: postgres, redis, fastapi, django.
django depends_on postgres (healthy) + redis (healthy) + fastapi (started).

Production shape PLANNED



Nothing in the repository provisions this — no Terraform, no Kubernetes manifests,
no CI/CD. It is the shape the current architecture permits, not what exists today.

PART 32 · CHAPTER VII

Security

What is genuinely implemented, then what is not. Nothing below is described as secure unless the code implements it.

32.1 Implemented

Control	Where	What it does
Password hashing	`set_password()` via `UserManager`	PBKDF2-SHA256, 870k iterations, per-user salt. Plaintext is never stored or logged
Password strength	`AUTH_PASSWORD_VALIDATORS`	Min 8, not similar to user attributes, not in the common-password list, not all numeric
Timing-safe login	Django `authenticate()`	Runs a dummy hash when no user matches, so response time does not reveal whether an email exists
Short-lived tokens	`SIMPLE_JWT`	30-minute access, 7-day refresh, rotation on refresh
Separate signing key	`JWT_SECRET_KEY`	Token signing isolated from `SECRET_KEY`
Data isolation	Every `get_queryset()` + `IsOwner`	Two independent layers. Another user's row returns **404, not 403** — the 403 would confirm it exists
FK narrowing	`TransactionSerializer.__init__`	A forged `account` id fails validation with 400 before any permission check runs
Default-deny permissions	`DEFAULT_PERMISSION_CLASSES`	A new endpoint is protected unless it explicitly opts out
Input validation	DRF serializers + Pydantic	Amount > 0, no future dates, type ↔ category agreement, `gt=0` at the engine boundary
SQL injection	ORM everywhere	Every query is parameterised. **No raw SQL anywhere in the repository**
XSS	React escaping	No `dangerouslySetInnerHTML` in `src/`
CORS allow-list	`CORS_ALLOWED_ORIGINS`	Explicit origins, not a wildcard, on the Django side
Security headers	`SecurityMiddleware`	`nosniff`, `Referrer-Policy: same-origin`, `X-Frame-Options: DENY`; HSTS 1 year and secure cookies when `DEBUG=False`
Rate limiting	`ScopedRateThrottle`	auth 20/min (brute-force resistance), analytics 60/min (cost control)
Service-to-service auth	`app/security.py`	Bearer token compared with `hmac.compare_digest` — **constant time**
No PII to the engine	`serialize_for_engine`	Integer `user_id` and transaction facts only — no name, email or account number
Return-value validation	`detect_and_store`	`valid_txn_ids` guard — a compromised engine cannot attach an anomaly to another user's transaction
Secret management	`python-dotenv`, `.gitignore`	Everything from the environment; `.env` ignored, `.env.example` committed with placeholders. **No real secret is in the repository**
Log hygiene	All loggers	Ids, methods and durations — never amounts, tokens or passwords

32.2 Gaps, ranked

#	Gap	Risk	Fix
1	**Tokens in `localStorage`**	Any successful XSS can exfiltrate them	httpOnly + SameSite refresh cookie; access token in memory only. Mitigated today by the 30-minute lifetime and React's escaping
2	**Logout does not revoke** PARTIAL	A token copied before logout stays valid until expiry	Add `token_blacklist` to `INSTALLED_APPS`, migrate, set `BLACKLIST_AFTER_ROTATION=True`
3	**FastAPI published on `9000:9000`**	The engine is reachable from the host, not only from Django	Change to `expose` only. The token still protects it, but the port should not be public
4	Insecure defaults in code	`"insecure-dev-key-change-me"`, `"dev-service-token"`,	Fine for local dev, dangerous if deployed unset. Fail fast on boot when `DEBUG=False` and a secret is still

		`require_service_token=False`	the default
5	No payload size limit	`max_transactions` is declared but unread — a huge POST could exhaust engine memory	One Pydantic validator on `AnalyticsRequest`
6	`/admin/` publicly routed	No IP allow-list, no 2FA	Restrict by network or add `django-otp`
7	No account lockout	20/min throttling slows but does not stop distributed brute force	`django-axes` or a failed-attempt counter
8	No audit log	No record of who changed what	An append-only audit table on the mutating service functions
9	No email verification or password reset	Unverified addresses; no self-service recovery	Requires an email backend — none is configured
10	No dependency scanning	A known CVE could sit unnoticed	`pip-audit` / `npm audit` in CI (no CI exists)

PART 33 · CHAPTER VII

Performance

33.1 Implemented

Optimisation	Where and effect
`select_related`	Transactions (`account`, `category`) and anomalies (`transaction`, `transaction__account`). Turns 1 + 2N queries into one JOIN — **the N+1 fix**
Database aggregation	`Sum`, `Count`, `annotate`, `values()` — totals computed in SQL, never by looping in Python
Composite indexes	`(account, transaction_date)` serves every dashboard, statement and analytics query
Pagination	20 default, 200 max — an unbounded list is impossible
Analytics caching	900 s, content-addressed. A repeat dashboard view costs one Redis GET instead of an HTTP round trip plus a model fit
Local dashboard	The most-visited page needs no network hop at all
`update_fields`	Balance writes touch two columns, not the whole row
`conn_max_age=600`	Reuses Postgres connections for 10 minutes
Vectorised analytics	pandas/NumPy in compiled C; `test_large_dataset_performs` pushes 5,000 transactions through `/analyze`
Frontend caching	RTK Query dedup + 60 s `keepUnusedDataFor`; 4 manual vendor chunks in `vite.config.ts`; gzip via nginx

33.2 Known costs and recommendations NOT IMPLEMENTED

Issue	Impact	Fix
Full history sent per analytics call	10,000 transactions ≈ a few MB of JSON serialised, sent and parsed every cache miss. The dominant cost at scale	Pre-aggregate in SQL and send summaries; or paginate the window; or move to a push model
`/analyze` rebuilds the DataFrame 4×	Roughly 4× the preprocessing work in one request	Build once in the route and pass it to the four services
Cache hit still queries the DB	The digest requires the rows, so a hit saves the HTTP call and computation but not the read	A cheaper key — e.g. `MAX(updated_at)` plus `COUNT(*)` per user
Isolation Forest fitted per request	~50–200 ms of CPU	Acceptable and deliberate (Part 17). Async offload if it ever dominates
Balance is a cached column	Correct only because every write path goes through the service	A periodic reconciliation job comparing the column against `Σ signed_amount`
No `db_index` on `transaction_date` alone	Queries filtering by date across all accounts cannot use the composite index efficiently	Add a single-column index if such queries appear
No compression on the Django↔FastAPI hop	Payloads travel uncompressed	gzip the request body
No frontend route-splitting	All 11 pages ship in the initial bundle	`React.lazy` per route

SIZING IT HONESTLY

For a single user with a few thousand transactions, every number above is comfortably fast. The full-history payload is the first thing that would break at ~100k transactions per user — and it is the right thing to name first when asked "what would you fix?"

PART 34 · CHAPTER VII

Redis

```
_redis_url = os.getenv("REDIS_URL", "").strip()
if _redis_url:
    CACHES = {"default": {"BACKEND": "django.core.cache.backends.redis.RedisCache",
                          "LOCATION": _redis_url}}
else:
    CACHES = {"default": {"BACKEND": "django.core.cache.backends.locmem.LocMemCache"}}
ANALYTICS_CACHE_TTL_SECONDS = int(os.getenv("ANALYTICS_CACHE_TTL_SECONDS", "900"))
```

Aspect	Detail
Status	IMPLEMENTED as Django's cache backend; provisioned in Compose with a healthcheck and a `redis_data` volume
What is cached	**Only analytics responses.** Nothing else in the app touches the cache
Key	`analytics:user:{id}:{kind}:{extra}:{sha1(payload)[:12]}`
TTL	900 s, from `ANALYTICS_CACHE_TTL_SECONDS`
Invalidation	**None needed** — content-addressed keys. Changing any transaction changes the digest, so stale entries are simply never addressed again and expire on their own
Fallback	`LocMemCache` when `REDIS_URL` is unset — correct for one process, but **per-worker**, so three Gunicorn workers would keep three independent caches

NOT IMPLEMENTED, DESPITE COMMON ASSUMPTIONS

Celery is not installed — no `celery` in `requirements.txt`, no `celery.py`, no worker service in Compose. Redis is a cache only, not a broker. PLANNED Redis is also not used for sessions (there are none), rate-limit storage beyond DRF's default cache use, or pub/sub.

PART 35 · CHAPTER VII

Docker

35.1 The five services

Service	Image / build	Ports	Notes
`postgres`	`postgres:16-alpine`	internal	Healthcheck `pg_isready` every 5 s; volume `postgres_data`
`redis`	`redis:7-alpine`	internal	`--save 60 1` persistence; healthcheck `redis-cli ping`; volume `redis_data`
`fastapi`	build `./analytics-service`	9000:9000	`REQUIRE_SERVICE_TOKEN: "true"`; `uvicorn --workers 2`; healthcheck on `/health`
`django`	build `./backend`	8000:8000	`depends_on` postgres *healthy*, redis *healthy*, fastapi started; CMD migrates then runs `gunicorn --workers 3 --timeout 60`
`frontend`	build `./frontend`, arg `VITE_API_BASE_URL=/api`	5173:80	Multi-stage: Node builds the bundle, nginx serves it and proxies `/api/` to `django:8000`

One bridge network `finsight`, two named volumes. The `condition: service_healthy` dependencies mean Django does not start until Postgres actually accepts connections — not merely until the container exists.

35.2 The Docker vocabulary, concretely

Term	In this project
Image	The built filesystem — e.g. `python:3.12-slim` plus the requirements plus the code
Container	A running instance of an image
Port mapping	`8000:8000` = host:container. `expose` would publish to the network only
Network	`finsight`, a user-defined bridge. **Service names become hostnames**
Volume	Named storage that outlives the container — the database survives `docker compose down`
Healthcheck	A command Docker runs periodically; `depends_on: condition: service_healthy` waits on it
Build arg	`VITE_API_BASE_URL` — Vite inlines env vars at *build* time, so it must be an arg, not a runtime variable

WHY `POSTGRES` WORKS AS A HOSTNAME

Docker Compose runs an embedded DNS server on the user-defined network, resolving each service name to its container IP. Inside the Django container, `localhost` means *that container* — there is no database there. On the network, `postgres:5432` reaches the database container. Same for `redis:6379` and `http://fastapi:9000`. This is why `.env.example` ships `postgres://finsight:finsight@postgres:5432/finsight` and Compose sets `FASTAPI_URL: http://fastapi:9000`, while running locally without Docker you would use `localhost`.

35.3 nginx in the frontend image

Two jobs: serve the built SPA with a history fallback (so `/transactions` refreshed directly returns `index.html` rather than 404), and `proxy_pass` everything under `/api/` to `http://django:8000`. That proxy is why the browser sees a **single origin** in Docker and CORS never comes into play there — unlike dev, where Vite's proxy plays the same role.

PART 36 · CHAPTER VII

Configuration

Every service reads its configuration from the environment. Nothing secret is committed; `.env` is git-ignored and `.env.example` is not.

Variable	Default	Purpose
`DJANGO_SECRET_KEY`	`insecure-dev-key-change-me`	Session/CSRF signing. **Must be replaced in production**
`DJANGO_DEBUG`	`true`	Gates HSTS, secure cookies and error detail
`DJANGO_ALLOWED_HOSTS`	`localhost,127.0.0.1`	Host-header allow-list
`DATABASE_URL`	*unset*	Set → PostgreSQL; unset → SQLite
`JWT_SECRET_KEY`	falls back to `SECRET_KEY`	Token signing key
`JWT_ACCESS_TOKEN_LIFETIME_MIN` · `JWT_REFRESH_TOKEN_LIFETIME_DAYS`	30 · 7	Token lifetimes
`FASTAPI_URL`	`http://localhost:9000`	Engine base URL (`http://fastapi:9000` in Compose)
`FASTAPI_SERVICE_TOKEN`	`dev-service-token`	**Must match on both sides**
`FASTAPI_TIMEOUT_SECONDS`	`8`	Client timeout for every engine call
`REDIS_URL`	*unset*	Set → Redis cache; unset → local memory
`ANALYTICS_CACHE_TTL_SECONDS`	`900`	Analytics cache lifetime
`CORS_ALLOWED_ORIGINS`	`http://localhost:5173,…`	Browser origin allow-list
`VITE_API_BASE_URL`	`/api`	Frontend API base. **Inlined at build time**
`REQUIRE_SERVICE_TOKEN`	`false`	Engine-side auth switch; `"true"` in Compose
`ZSCORE_THRESHOLD` · `IQR_MULTIPLIER` · `ISOLATION_FOREST_CONTAMINATION` · `MIN_GROUP_SIZE`	3.0 · 1.5 · 0.05 · 5	Detector tuning without code changes
`LOG_LEVEL`	`INFO`	Both services

Django loads `backend/.env` first, then the repo-root `.env` — the more specific file wins because `load_dotenv` does not override an already-set variable.

Setting	Development	Production
`DJANGO_DEBUG`	`true`	`false` — enables HSTS and secure cookies
Database	SQLite	PostgreSQL via `DATABASE_URL`
Cache	LocMem	Redis
Engine auth	Off	`REQUIRE_SERVICE_TOKEN=true` with a real secret
Server	`runserver` / `--reload`	Gunicorn 3 workers / uvicorn 2 workers

PART 38 · CHAPTER VII

Testing

70 automated tests — 43 Django, 27 FastAPI. Counted from the source, not estimated.

38.1 Django — 43 tests

File	Tests	Covers
`transactions/tests.py`	**15**	Balance on create/update/delete, type changes, moving between accounts, positive-amount rule, category-type agreement, cross-user account rejection, future dates, isolation, pagination, filters, search, ordering
`users/tests.py`	6	Registration, duplicate email, weak password, login, `/me/`, change password
`accounts/tests.py`	5	CRUD, opening balance, delete-with-transactions rejection, isolation
`categories/tests.py`	5	Defaults seeded on signup, duplicate rejection, default immutability, reassignment
`analytics/tests.py`	5	Proxy + cache, timeout → 503, 5xx → 503, bad JSON → 503, dashboard survives an outage
`anomalies/tests.py`	4	Persistence, **idempotency preserving review state**, ignore + severity filter, isolation
`statements/tests.py`	3	Monthly figures, CSV export, account scoping

```python
# the pattern used throughout — DRF's APITestCase with a real JWT
class TransactionBalanceTests(APITestCase):
    def setUp(self):
        self.user = User.objects.create_user(email="a@b.com", password="StrongPass123", name="A")
        self.client.force_authenticate(self.user)     # bypasses the login round trip
        self.account = Account.objects.create(user=self.user, account_name="HDFC",
                                              balance=Decimal("50000.00"))

    def test_expense_decreases_balance(self):
        self.client.post("/api/transactions/", {...}, format="json")
        self.account.refresh_from_db()
        self.assertEqual(self.account.balance, Decimal("47500.00"))
```

38.2 FastAPI — 27 tests

File	Tests	Covers
`test_anomalies.py`	**9**	Z-score beats the classic on a masked outlier, catches the ₹8,000 food case, skips small groups; IQR formula and detection; forest predictions and the <8-row guard; ALL merges to one row per transaction; empty dataset
`test_validation.py`	6	Negative amount, bad date, unknown type, missing `user_id`, health, **5,000-transaction performance**
`test_trends.py`	5	Monthly points and MoM growth, average monthly, decreasing detection, weekly granularity, empty
`test_summary.py`	4	Endpoint shape, savings-rate formula, empty dataset, month-span division
`test_categories.py`	3	Endpoint, per-category maths, empty

Because the engine is stateless, these need **no database and no fixtures** — `test_per_category_math` passes three dictionaries and asserts five numbers. A shared `client` and `payload` fixture live in `tests/conftest.py`.

38.3 What is not tested GAP

Missing	Why it matters
Any frontend test	No Vitest, no Testing Library, no Playwright. `make test-frontend` only type-checks and builds. The largest single gap — the mutex refresh logic in particular deserves a test
True end-to-end	Django's suite mocks httpx, so the real HTTP contract between the two services is never exercised together
Concurrency	Nothing tests two simultaneous writes to one account — and SQLite could not prove it anyway
Coverage measurement	No `coverage` configuration, so the real percentage is unknown
CI	No workflow file — tests run only when someone remembers

PART 47 · CHAPTER VII

Production Readiness

A deliberately unflattering assessment. This is a strong portfolio and learning project that is **not** ready to hold real money for real users.

Dimension	Verdict	Reasoning
Data model	**Strong**	Correct types, constraints, indexes and `on_delete` semantics; migrations committed
Transactional correctness	**Strong**	`atomic()` + `select_for_update()` on every mutation, with six tests protecting it
API design	**Strong**	Consistent envelope, pagination, filtering, throttling, OpenAPI docs on both services
Authorization	**Strong**	Two independent layers, default-deny, verified by isolation tests
Resilience	**Strong**	Analytics failure degrades rather than breaks, and this is tested
Authentication	Adequate	Solid hashing and short tokens, but **no server-side revocation** and tokens in localStorage
Error handling	Adequate	Consistent and user-friendly; no error tracker (Sentry) anywhere
Backend testing	Adequate	70 meaningful tests on the risky paths; coverage unmeasured
Docker	Adequate	Complete local stack with healthchecks; images run as root and are not multi-stage on the Python side
Logging	Adequate	Structured and clean, but plain text to stdout with no aggregation
Performance	Adequate	Right optimisations for the current scale; the full-history payload is the first thing to break
Frontend testing	**Weak**	Zero tests
Monitoring	**Weak**	No metrics, no tracing, no alerting, no uptime checks
CI/CD	**Absent**	No pipeline of any kind
Deployment	**Absent**	No cloud config, no TLS termination, no secret manager, no backups
Ops procedures	**Absent**	No runbook, no restore drill, no on-call, no incident process

THE MINIMUM TO RUN THIS FOR REAL

1. Replace every default secret and fail fast at boot if one is still in place
2. Enable the token blacklist so logout revokes
3. Move refresh tokens to httpOnly cookies
4. Stop publishing FastAPI's port to the host
5. TLS everywhere, with HSTS already configured for `DEBUG=False`
6. Automated Postgres backups *and a tested restore*
7. Sentry (or equivalent) plus uptime monitoring on both `/health` endpoints
8. A CI pipeline running both test suites and the frontend build on every push
9. Migrations as a separate release step rather than a per-replica race
10. Restrict or remove `/admin/`

The honest one-line summary: the application logic is production-grade; the operational envelope around it is not.

PART 40 · CHAPTER VIII

Code Walkthroughs

Twelve code paths worth knowing cold. Each: what it does, why it exists, what calls it, what it calls, and what happens next.

40.1 `users/managers.py` — creating a user

```
def _create_user(self, email, password, **extra):
    if not email: raise ValueError("Users must have an email address")
    email = self.normalize_email(email).lower()
    user = self.model(email=email, **extra)
    user.set_password(password)
    user.save(using=self._db)
    return user
```

Why: email replaces username, so Django's default manager cannot be used. **Called by:** `RegisterSerializer.create()`, `createsuperuser`, `seed_demo`. **Calls:** `set_password()` → PBKDF2-SHA256. **Next:** the `post_save` signal fires and seeds 19 default categories. **Key line:** `set_password` — assigning `user.password` directly would store plaintext.

40.2 `categories/signals.py` — the signal

```
@receiver(post_save, sender=settings.AUTH_USER_MODEL)
def seed_default_categories(sender, instance, created, **kwargs):
    if created:
        Category.create_defaults_for(instance)
```

Why: a user with zero categories cannot record anything, so the first-run experience would be broken. **Registered in:** `CategoriesConfig.ready()`. **Calls:** `bulk_create(..., ignore_conflicts=True)` — one INSERT for all 19 rows, and the `ignore_conflicts` makes it safe against the unique constraint if it ever runs twice. **The `if created` guard is essential** — without it, every profile save would re-seed.

40.3 `users/views.py` — registration returns tokens

```
def create(self, request, *args, **kwargs):
    serializer = self.get_serializer(data=request.data)
    serializer.is_valid(raise_exception=True)
    user = serializer.save()
    logger.info("user_registered user_id=%s", user.id)
    refresh = RefreshToken.for_user(user)
    return Response({"user": UserSerializer(user).data,
                     "access": str(refresh.access_token), "refresh": str(refresh)},
                    status=201)
```

Why overridden: the default `CreateAPIView` would return only the user, forcing an immediate second login call. **Note:** the log records the id, never the email or password. **Next:** React dispatches `credentialsReceived` and navigates to the dashboard.

40.4 `transactions/serializers.py` — narrowing the FK querysets

```
account  = serializers.PrimaryKeyRelatedField(queryset=Account.objects.none())   # ← none()
category = serializers.PrimaryKeyRelatedField(queryset=Category.objects.none())

def __init__(self, *args, **kwargs):
    super().__init__(*args, **kwargs)
    user = getattr(self.context.get("request"), "user", None)
    if user is not None and user.is_authenticated:
        self.fields["account"].queryset  = Account.objects.filter(user=user)
        self.fields["category"].queryset = Category.objects.filter(user=user)
```

> **THE MOST SECURITY-RELEVANT FEW LINES IN THE DJANGO APP**
>
> Declaring `none()` at class level and narrowing per request means a forged `account: 999` belonging to another user fails **validation with 400** — it never reaches a permission check, and the error reveals nothing about whether account 999 exists. The `none()` default is the safe failure mode: if the context were ever missing, nothing validates rather than everything.

40.5 `transactions/serializers.py` — cross-field validation

```
def validate(self, attrs):
    txn_type = attrs.get("transaction_type") or getattr(self.instance, "transaction_type", None)
    category = attrs.get("category") or getattr(self.instance, "category", None)
    if category and txn_type and category.type != txn_type:
        raise serializers.ValidationError({"category":
            f"An {txn_type} transaction must use an {txn_type} category "
            f"(got a {category.type} category)."})
    return attrs
```

Why: classifying salary as "Food" would corrupt every category analytic. **The `or getattr(self.instance, …)` pattern** is what makes this work for PATCH: if only `transaction_type` is supplied, the category is read from the existing row, so a partial update cannot sneak past the rule.

40.6 `transactions/services.py` — the balance update

```
@db_transaction.atomic
def update_transaction(txn, **fields):
    txn = Transaction.objects.select_for_update().get(pk=txn.pk)
    old_account = _locked_account(txn.account_id)
    old_delta = txn.signed_amount            # ← BEFORE mutating: the whole correctness argument
    for key, value in fields.items():
        setattr(txn, key, value)
    txn.full_clean(exclude=["account", "category"])
    txn.save()
    old_account.balance = old_account.balance - old_delta + txn.signed_amount
    old_account.save(update_fields=["balance", "updated_at"])
```

Called by: `TransactionSerializer.update()`. **Why a service and not the serializer:** so `seed_demo` and the shell can use it, and so the atomic block is defined in one place. **The single line that matters:** capturing `old_delta` first — reading it after `setattr` would reverse the *new* impact and silently corrupt the balance.

40.7 `accounts/views.py` — a whole CRUD API

```
class AccountViewSet(viewsets.ModelViewSet):
    serializer_class = AccountSerializer
    permission_classes = [IsOwner]
    filterset_fields = ["account_type", "is_active", "currency"]
    search_fields = ["account_name"]
    ordering_fields = ["account_name", "balance", "created_at"]

    def get_queryset(self):
        if getattr(self, "swagger_fake_view", False):
            return Account.objects.none()
        return Account.objects.filter(user=self.request.user) \
                              .annotate(transaction_count=Count("transactions"))

    def perform_destroy(self, instance):
        if instance.transactions.exists():
            raise ValidationError("This account has transactions. Deactivate it instead of deleting.")
        instance.delete()
```

Thirty-three lines yielding five endpoints with pagination, filtering, search and ordering — the clearest single illustration of what DRF buys you. `annotate(Count(...))` computes the transaction count **in SQL**. The `swagger_fake_view` guard exists because drf-spectacular introspects views without a real request, and `self.request.user` would be anonymous.

40.8 `common/permissions.py` — ownership

```
class IsOwner(BasePermission):
    def has_object_permission(self, request, view, obj):
        owner = getattr(obj, "user", None)
        if owner is None and hasattr(obj, "account"):
            owner = getattr(obj.account, "user", None)
        return owner == request.user
```

One class covering two ownership shapes: a direct `user` FK (Account, Category, Anomaly) and the `account.user` chain (Transaction). **Called by:** DRF, automatically, on every object-level operation. Belt and braces alongside queryset filtering.

40.9 `analytics/client.py` — the service call

```
with httpx.Client(timeout=settings.FASTAPI_TIMEOUT_SECONDS) as client:
    response = client.post(url, json=payload, headers=_headers())
```

Why a timeout is non-negotiable: without it a hung engine holds a Gunicorn worker indefinitely; a few of those and Django stops serving logins. **Why one exception type:** five failure modes collapse into `AnalyticsUnavailable`, so callers write one `except` and are correct for all of them. **Next:** `_AnalyticsView` turns it into a 503, or `build_dashboard` swallows it and sets `analytics_available: false`.

40.10 `app/services/anomaly_service.py` — the Z-score detector

```
for category, group in expenses.groupby("category"):     # ← the core idea
    if len(group) < settings.min_group_size: continue
    z = modified_zscores(group["amount"].to_numpy(dtype="float64"))
    mean = float(group["amount"].mean())
    for (_, row), zi in zip(group.iterrows(), z):
        if abs(zi) <= threshold: continue
        ratio = row["amount"] / mean if mean else float("inf")
        items.append(_row_item(row, DetectionMethod.ZSCORE,
                               safe_float(-abs(zi)), severity_from_ratio(ratio), reason))
```

Four decisions in nine lines: group by category (rent vs food); `min_group_size` (no statistics on four points); MAD-based scores (robust to the outlier being hunted); negate the score (lower = worse, matching sklearn). **Next:** merged with the other two detectors, LOW dropped, sorted, returned.

40.11 `app/ml/isolation_forest.py` — scaling before fitting

```
X = StandardScaler().fit_transform(features[FEATURE_COLUMNS].to_numpy(dtype="float64"))
model = IsolationForest(n_estimators=200, contamination=..., random_state=42)
model.fit(X)
predictions = model.predict(X)        # -1 anomaly, +1 normal
scores      = model.score_samples(X)  # lower = more anomalous
```

Why scaling is not optional: amount spans 100–45,000 while day_of_week spans 0–6. Unscaled, random split thresholds land in the amount dimension almost every time and the forest degenerates into a univariate amount detector — which the Z-score already does better and more explainably. **Why `random_state=42`:** reproducible findings across refreshes and test runs.

40.12 `anomalies/services.py` — upsert without clobbering

```
Anomaly.objects.update_or_create(
    user=user, transaction_id=txn_id, detection_method=detection_method,
    defaults={"anomaly_score": …, "severity": …, "reason": …, "amount": …,
              "category": …, "merchant": …, "transaction_date": …})
    # status and reviewed are deliberately absent from defaults
```

The design insight: the engine owns the *measurement*; the user owns the *decision*. Re-running detection refreshes scores and reasons but cannot un-review something a human already reviewed. The lookup keys match the model's `UniqueConstraint` exactly, which is what makes the operation atomic and idempotent. `test_detect_is_idempotent_and_preserves_review_state` is the proof.

40.13 `services/api.ts` — the frontend refresh

```
if (result.error?.status === 401) {
  if (!mutex.isLocked()) {
    const release = await mutex.acquire();
    try {
      const refreshed = await rawBaseQuery({url: "/auth/refresh/", method: "POST",
                                            body: {refresh}}, api, extra);
      if (data?.access) { api.dispatch(tokenRefreshed(data.access));
                          result = unwrap(await rawBaseQuery(args, api, extra)); }
      else              { api.dispatch(loggedOut()); }
    } finally { release(); }
  } else {
    await mutex.waitForUnlock();
    result = unwrap(await rawBaseQuery(args, api, extra));
  }
}
```

Why the mutex: the dashboard fires four queries at once; on an expired token all four 401 simultaneously. Because `ROTATE_REFRESH_TOKENS=True`, four concurrent refreshes would each invalidate the previous and three would fail — logging the user out at random. Exactly one request refreshes; the rest wait and replay. **The `finally` is critical** — without it, a thrown error would leave the mutex locked and freeze every subsequent request.

PART 39 · CHAPTER IX

Thirty Interview Questions

Answers grounded in the actual implementation. Where something is not implemented, the answer says so — that is the answer that earns credibility.

1 Explain your FinSight project.

FinSight is a personal-finance platform where a user manages accounts and transactions and gets spending analytics plus automatic detection of unusual spending. It is three services: a React and TypeScript frontend, a Django REST backend that owns all financial state, and a stateless FastAPI service that does the statistics and machine learning. Django is the system of record — auth, accounts, categories, transactions, statements, balance keeping. FastAPI is a pure calculator: Django sends it a payload, it returns analytics and anomaly findings, and it never touches the database. The interesting part is the anomaly detection — three algorithms, all computed *per category*, so a ₹18,000 rent payment is normal while an ₹8,000 restaurant bill is flagged.

2 Why Django?

Because roughly 60% of a financial backend is work Django already does correctly: a hashed-password user model, permissions, migrations, an admin, security middleware, and an ORM with real transaction control. My balance-keeping code depends on `transaction.atomic()` and `select_for_update()` — first-class in Django. Hand-writing authentication and authorization for something holding money is how you ship a vulnerability.

3 Why FastAPI?

The analytics workload is a pure function — data in, numbers out — with no state, and it needs the Python scientific stack. Pydantic validates at the boundary so bad data can never reach pandas, and the OpenAPI schema comes free from the type hints. Six endpoints in about ninety lines, because the schemas carry the weight.

4 Why not Django for everything?

It would work, and for today's load it would be simpler. I split it for four reasons. Scaling: an Isolation Forest fit is CPU-bound, and in Gunicorn's sync workers that would block logins under load. Dependencies: pandas, NumPy, SciPy and scikit-learn are about 200 MB that every Django replica would carry. Blast radius: a scikit-learn upgrade should not force a redeploy of the service holding the money. And boundaries: with an ORM in scope it is irresistible to read transactions directly inside the detector, and within months the ML code is untestable without a database. The physical split makes that impossible. I'd add honestly that for a single-user app the split is justified by where it is heading, not by current necessity.

5 Why not FastAPI for everything?

I'd rebuild by hand what Django ships: SQLAlchemy plus Alembic, password hashing, a permission system, an admin, CRUD scaffolding, pagination, filtering, throttling. That is a large amount of security-critical code I'd own — and none of it would make the analytics better.

6 Why PostgreSQL?

Three properties I actually depend on: true `NUMERIC(14,2)` so money is exact rather than floating point; row-level locking, which is what `select_for_update()` compiles to and what prevents lost updates on balances; and enforced multi-column unique constraints. SQLite is the fallback when `DATABASE_URL` is unset — great for onboarding and tests, but it *ignores* `select_for_update()`, so the concurrency guarantee is only truly exercised on Postgres.

7 Why is FastAPI stateless?

It has no database driver, no models, no sessions, and it does not persist the model — the forest is fitted per request and discarded. Everything it needs arrives in the payload. That means scaling is just adding replicas, deployment is kill-and-replace with no migrations, and every function is `f(input) → output`, which is why 27 tests run with no fixtures. Fitting per request costs 50–200 ms but makes the model always current and personalised per user, and Django's cache absorbs repeat calls.

8 How does Django communicate with FastAPI?

A synchronous HTTP POST with httpx, from `analytics/client.py`. Django loads the user's transactions, serialises a compact payload using category *names* rather than primary keys, checks a Redis cache keyed by a SHA-1 digest of the payload, and on a miss POSTs to `{FASTAPI_URL}{path}` with an `Authorization: Bearer` service token and an 8-second timeout. Five failure modes — timeout, transport error, 5xx, 4xx, unparseable JSON — all collapse into one `AnalyticsUnavailable` exception.

9 What happens if FastAPI is down?

It depends on the endpoint, deliberately. The dashboard computes its cards, trend, breakdown, top expenses and recent transactions *locally in Django* and only *enriches* with anomaly alerts, so it returns **200** with `analytics_available: false` and every number still correct. The dedicated Analytics and Anomalies pages return **503** with `error_code: ANALYTICS_UNAVAILABLE` and a message saying transactions are safe. There is a test — `test_dashboard_survives_analytics_outage` — that kills the connection and asserts exactly this.

10 How do you stop one user seeing another's transactions?

Two independent layers. Every queryset is scoped — `Transaction.objects.filter(account__user=request.user)` — so another user's row simply does not exist for that request and returns 404 rather than 403, which would confirm it exists. On top, `IsOwner` re-checks ownership at the object level. There is a third: the serializer narrows the account and category querysets to the user's own rows, so a forged foreign key fails validation with a 400 before any permission check runs. Tested by `test_user_isolation_on_detail`.

11 How is JWT authentication implemented?

SimpleJWT. Login verifies the password with PBKDF2-SHA256 and returns a 30-minute access token and a 7-day refresh token, signed HS256 with a dedicated `JWT_SECRET_KEY`. React stores them in Redux and localStorage, and `prepareHeaders` attaches the access token to every request. On a 401 the base query transparently refreshes and replays — behind a mutex, because rotation is on and concurrent refreshes would invalidate each other. I'd flag one gap: the blacklist app is not installed, so **logout is client-side only** and a copied token stays valid until it expires.

12 How do you update the account balance safely?

Every create, update and delete goes through `transactions/services.py`. Each function is decorated `@transaction.atomic` and locks the account row with `select_for_update()`, so the insert and the balance update commit together or not at all, and two concurrent writes serialise instead of both reading the same starting balance. Updates always reverse the previous signed impact before applying the new one, and the old delta is captured *before* the fields are mutated — that ordering is the whole correctness argument. Six tests protect it.

13 How does anomaly detection work?

Django sends up to 180 days of transactions to FastAPI. The engine builds a pandas DataFrame, keeps expenses, and runs three detectors: a MAD-based Z-score and an IQR test, both grouped by category, and an Isolation Forest over eight engineered features across the whole dataset. Each finding is normalised to the same shape — transaction id, score, severity, and a plain-English reason. Findings are merged so one transaction produces one row, keeping the most severe, LOW is dropped from the combined view, and the list is sorted worst-first. Django then validates that each transaction id really belongs to the user and upserts on `(user, transaction, method)` without touching the review status.

14 Why Z-score, and why the modified version?

A Z-score is unit-free, so it compares across categories with different scales. But the classic version *masks* outliers: an ₹8,000 food bill pulls the mean from ₹750 to ₹1,656 and inflates σ to ₹2,413, so it scores only 2.63 and slips under the threshold of 3. I use the modified Z-score — median and median-absolute-deviation instead of mean and standard deviation. The same transaction scores 32.5, because neither the median nor the MAD is moved by the outlier. The threshold of 3 is the conventional ~0.27% tail, and it is an environment variable so it can be tuned.

15 Why IQR?

Because it makes no distributional assumption at all — it is purely positional. Q1 and Q3 bound the middle half; anything outside Q1 − 1.5·IQR to Q3 + 1.5·IQR is an outlier. Real spending is right-skewed, and IQR adapts to that naturally because the two quartiles move independently. I measure how many IQRs past the fence a transaction sits, which makes the severity scale-free.

16 Why Isolation Forest?

The other two are univariate — they only see the amount. Isolation Forest works across eight features at once, so it can catch a moderate amount at an unusual time in a category you rarely use. It isolates points with random splits and measures how few cuts it takes; outliers sit in sparse regions and are cheap to isolate. Two implementation points I'd raise: `StandardScaler` is essential or the amount dimension dominates every split and it collapses into a univariate detector, and `contamination` fixes the threshold, so the forest *always* labels about 5% as anomalous — which is why I added an explicit suppression filter for near-constant categories and weakly-scored normal amounts.

17 Z-score versus IQR?

Z-score gives a continuous magnitude and assumes rough normality; IQR gives a clean in/out decision and assumes nothing. Both are robust in my implementation — median/MAD and quartiles are equally resistant to extreme values. IQR is the better statistical fit for skewed spending; the Z-score ranks severity better. I run both and keep the more severe verdict, so a transaction has to look normal to both to pass unflagged.

18 Statistical detection versus ML?

Statistical methods are cheap, deterministic, need no training, and are trivially explainable — "₹8,000 is 10.4× your average Food expense" is a sentence a user acts on. But they are univariate and threshold-based. ML captures multivariate interactions no threshold can express, at the cost of interpretability and a contamination parameter that guarantees findings whether or not any exist. I use both, and I let the statistical methods do the explaining — the forest's reason string is deliberately vaguer when the amount itself isn't unusual, because inventing a number there would be dishonest.

19 What features does the Isolation Forest use?

Eight: the amount; the category ordinally encoded; day of week, day of month and month; the category's average spending; the transaction frequency in that category; and the signed deviation from the category average. The last one is the most discriminative — it hands the model "how far from your own norm" directly rather than making it learn the relationship. Day-of-month matters more than it sounds: salary on the 1st, rent on the 3rd, bills mid-month, so it catches "right amount, wrong time". I'd note that ordinal encoding implies a false ordering, which is acceptable for a tree method but would be wrong for a linear model.

20 How would you scale FastAPI?

It is stateless, so horizontally — add replicas behind a load balancer. No sticky sessions, no shared cache, nothing to coordinate. Right now it runs two uvicorn workers in one container. Beyond that I'd cap the payload size, which is currently a declared but unenforced setting, and move heavy detection to a queue so the request path stays fast.

21 How would you handle 10 million transactions?

The current design would break in one specific place: analytics sends the user's full history in the payload, so the JSON grows unbounded. I would pre-aggregate in SQL — daily and monthly rollup tables maintained incrementally — and send summaries rather than raw rows. Then partition the transactions table by month, add a covering index for the common range query, and move detection to an asynchronous job writing results to the anomaly table rather than computing on request. The CRUD side already paginates and indexes correctly, so it would hold up.

22 How do you cache analytics?

Redis, 900-second TTL, with the key `analytics:user:{id}:{kind}:{range}:{sha1(payload)[:12]}`. The interesting part is the digest: because the key contains a hash of the exact transaction set, **changing any transaction changes the key**. I never need to invalidate — stale entries simply stop being addressed and expire. That sidesteps cache invalidation entirely. The honest cost is that computing the digest still requires reading the rows, so a hit saves the HTTP call and the computation, not the database query.

23 How would you make analytics asynchronous?

The `analytics.services` layer is the single seam. Today it is `Django → HTTP → FastAPI → response`. I'd change it to enqueue a Celery task, return a "pending" state, and have the worker call FastAPI and store the result. The frontend already tolerates pending and unavailable states, so the React data layer would not change. That is deliberate — the degradation path was designed with this in mind.

24 What is Redis's role?

Cache only. It backs Django's `CACHES["default"]` when `REDIS_URL` is set, and the only thing cached is analytics responses. Without it, Django falls back to local-memory caching, which is per-worker and therefore inconsistent across three Gunicorn workers. It is provisioned in Compose with persistence and a healthcheck. It is *not* used for sessions, queues or pub/sub.

25 What is Celery's role?

None — it is not installed. No `celery` dependency, no worker service. If asked what it *would* do: scheduled nightly anomaly detection, weekly summary emails, CSV imports, and offloading expensive analytics. Redis would be the broker, which is why it is already provisioned.

26 How would you deploy this to AWS?

Nothing in the repo does this today, so I'd describe the shape the architecture permits. RDS PostgreSQL with automated backups; ElastiCache Redis; both services as ECS Fargate tasks behind an ALB with FastAPI on an internal target group only; the React bundle on S3 behind CloudFront; secrets in Secrets Manager; migrations as a one-off task in the release step rather than on every container start. FastAPI scales on CPU because it is stateless; Django scales on request count.

27 How would you monitor the services?

Currently there is structured stdout logging with named loggers and duration logs on every analytics call, plus `/health` endpoints on both services wired to Docker healthchecks — and nothing else. No metrics, tracing or alerting. I'd add Sentry for errors, Prometheus metrics for request rate, latency and error rate, a correlation id propagated from Django into the FastAPI call for tracing, and alerts on 503 rate and analytics p95 latency.

28 How would you handle database failures?

Today, poorly and honestly so: a Postgres outage takes the API down. `conn_max_age=600` reuses connections, and Compose waits for a healthy database before starting Django, but there is no read replica, no retry logic, no circuit breaker and no automated backup. I'd add automated backups with a *tested* restore first — an untested backup is not a backup — then a read replica for analytics queries, and connection retry with exponential backoff.

29 How would you improve anomaly detection?

Four things. First, a feedback loop: users can already mark findings Reviewed or Ignored, and that signal is stored but unused — I'd feed it back to suppress patterns a user repeatedly ignores. Second, merchant-level baselines, not just category — ₹5,000 at a supermarket differs from ₹5,000 at a jeweller. Third, time-series methods for gradual drift, which Z-score structurally cannot see because the baseline drifts with the spending. Fourth, calibrate the Isolation Forest severity thresholds against labelled data rather than the seed dataset — they are currently the least principled numbers in the system.

30 What would you change if you rebuilt it?

Refresh tokens in httpOnly cookies rather than localStorage — that is the top security item. Frontend tests from day one; there are currently zero. Pre-aggregated analytics tables instead of sending full history. A correlation id across services for tracing. CI running both suites on every push. And I'd consider whether the service split earns its keep at this scale — I still think yes for where it is heading, but I'd want to say that as a considered trade rather than an assumption.

PART 42 · CHAPTER IX

Design Decisions

Decision	Reason	Alternative	Trade-off accepted
React	A dashboard is many widgets over one dataset that must all update on a write	Vue, Svelte, Django templates	Larger bundle and a build step versus server-rendered simplicity
TypeScript	A money app: a renamed backend field should be a compile error, not ₹NaN	Plain JS	Slower to write; no runtime validation (Zod would add it)
Django	Auth, ORM, migrations, admin and transaction control for free	FastAPI-only, Node	Heavier framework; more magic to learn
FastAPI	Stateless computation with Pydantic validation and the SciPy stack	A Django analytics app	A second service to deploy and monitor; a network hop
PostgreSQL	Exact decimals, row locking, real constraints	MySQL, MongoDB	Needs a running server — mitigated by the SQLite fallback
Separate analytics	Independent scaling, isolated dependencies, enforced boundaries	In-process	More moving parts; duplicate dashboard maths
Stateless engine	Scale by replicas; no model registry; testable as pure functions	Give it a database	Full payload per call — the first thing to break at scale
pandas	Category grouping is one line and is the core idea of the detector	Plain Python, Polars	A large dependency; higher memory per request
Three detectors	Univariate methods explain well; the forest catches combinations	One method	3× the compute; overlapping findings, hence the merge step
Per-category stats	₹18,000 rent is normal; ₹8,000 food is not	One global baseline	Needs ≥5 transactions per category, so new users see fewer alerts
REST	Simple, cacheable, well-tooled; DRF routers generate most of it	GraphQL, gRPC	Some over-fetching — not a problem at these payload sizes
JWT	Cross-origin frontend; stateless API; no server session store	Session cookies	**No revocation without a blacklist** — currently unimplemented
Response envelope	One error-handling path on the client for every endpoint	Bare DRF responses	Non-standard shape; needs the custom renderer and unwrap step
Content-addressed cache	Removes invalidation logic entirely	Explicit invalidation	A hit still reads the database to build the digest
Local dashboard	The daily page must not depend on a second service	Route it all through FastAPI	Two implementations of similar sums
Docker Compose	One command brings up the whole stack	Manual setup	Requires Docker; images are not production-hardened
Redis	Shared cache across workers; future broker	LocMem only	Another service; unused as a broker today

PART 43 · CHAPTER IX

Trade-offs

The service split

Pros	Cons
Independent scaling of a CPU-bound workload	Two deployments, two health checks, two log streams
Heavy ML dependencies isolated from the money service	A network hop with latency and a new failure mode
Boundaries enforced physically — no ORM in the detector	A contract to keep in sync across two languages of tooling
The engine is replaceable behind one response shape	Duplicate dashboard computation in Django
27 tests run with no database	No true end-to-end test — Django mocks httpx

JWT versus sessions

Pros	Cons
Stateless API; no session store; clean cross-origin story	**Cannot revoke** without a blacklist — not currently enabled
Claims carried in the token; no lookup per request	localStorage is XSS-readable; a cookie would be safer
Short access + rotating refresh limits the window	Rotation forces client-side mutex complexity

Cached balance column versus computing on read

Pros	Cons
O(1) reads; the accounts page is instant	Denormalised — correct only because every write path goes through the service
Statements recompute from the ledger, so they cross-check it	No reconciliation job exists to detect drift

Three detectors versus one

Pros	Cons
Complementary coverage; univariate methods explain well	3× compute; overlapping findings need a merge step
Merging keeps the most severe verdict per transaction	Three severity mappings to reason about and tune

Stateless engine

Pros	Cons
Trivial horizontal scaling and deployment; no model registry	Full transaction history in every payload
Always-current, per-user model — personalisation is free	Recomputes work that a persisted model could reuse

PART 44 · CHAPTER IX

What to Say in an Interview

30 seconds

> "FinSight is a personal-finance analytics platform. React frontend, Django REST backend that owns all the financial data, and a separate stateless FastAPI service that does spending analytics and anomaly detection with pandas and scikit-learn. The interesting part is that it detects unusual spending per category — so your monthly rent isn't flagged, but an ₹8,000 restaurant bill when you normally spend ₹700 is."

1 minute

> "FinSight is a personal-finance management and spending-analytics platform. Users manage accounts and transactions, and get a dashboard, statistical analysis, and automatic detection of unusual spending.
>
> It's three services. React with TypeScript and Redux Toolkit Query on the front. Django REST as the system of record — authentication, accounts, categories, transactions, statements, and balance keeping, where every write runs in a database transaction with row locking. And a stateless FastAPI service that does the analytics: pandas for reshaping, NumPy and SciPy for statistics, and scikit-learn for anomaly detection.
>
> The split is deliberate — Django owns state that must be correct forever, FastAPI owns computation that has to be fast and replaceable. FastAPI never touches the database; Django sends it a payload and decides what to keep from the response."

2 minutes

> Everything above, then:
>
> "The anomaly detection is the part I'd highlight. It runs three algorithms. A Z-score, but the modified median-absolute-deviation version — the classic one *masks* outliers, because a large transaction inflates the mean and standard deviation and hides itself. On my test data the classic score for an ₹8,000 food bill was 2.63, under the threshold of 3; the modified score is 32.5. Then an IQR test with Tukey fences, which assumes nothing about the distribution — useful because spending is right-skewed. And an Isolation Forest over eight engineered features, which catches combinations the univariate methods can't.
>
> Crucially, the statistics are computed *within categories*, never one global mean. A global baseline would flag rent every month and miss the food bill. Every finding carries a plain-English reason generated in the engine — 'this is 10.4× higher than your average Food expense of ₹768'.
>
> The other thing I'd point to is graceful degradation. The dashboard computes its numbers locally in Django and only *enriches* with the analytics service, so if FastAPI is down the user still sees every balance and total — the response returns 200 with a flag saying analytics are unavailable. There's a test that kills the connection and asserts that."

5 minutes — the deep version

> All of the above, then work through:
>
> **The write path.** "A transaction create goes React → RTK Query → Django. The serializer narrows the account and category querysets to the user's own rows, so a forged foreign key fails validation with a 400 before any permission check. Then a service function runs in `transaction.atomic()` with `select_for_update()` on the account row: insert the transaction, apply the signed amount to the balance, commit together. Updates reverse the old impact before applying the new one, and the old delta is captured before the fields are mutated — that ordering is the whole correctness argument. Six tests protect it."
>
> **The service boundary.** "Django serialises transactions using category *names*, not primary keys, so the engine knows nothing about my schema. It checks a Redis cache keyed by a SHA-1 digest of the payload — which means changing any transaction changes the key, so I never write invalidation logic. On a miss it POSTs with a bearer token and an 8-second timeout. Five failure modes collapse into one exception, which the views turn into a 503 and the dashboard swallows entirely."
>
> **Security.** "Two isolation layers plus serializer-level narrowing. Constant-time comparison on the service token. Django re-validates the transaction ids the engine returns, so even a compromised engine can't attach an anomaly to someone else's data. And I'd name the gaps: logout doesn't revoke because the blacklist app isn't installed, and tokens sit in localStorage rather than an httpOnly cookie."
>
> **What I'd change.** "Frontend tests — there are none. Move refresh tokens to cookies. And pre-aggregate analytics instead of sending full history, which is the first thing that breaks at scale."

> **WHY NAMING THE GAPS HELPS YOU**
>
> Interviewers probe for the boundary of your knowledge. Volunteering "logout doesn't actually revoke, here's why and here's the two-line fix" demonstrates you understand the system rather than having assembled it. It also stops them finding it themselves, which reads very differently.

PART 45 · CHAPTER IX

Resume Presentation

Title: FinSight — Personal Finance & Spending Analytics Platform
Stack: React · TypeScript · Redux Toolkit · Material UI · Django · Django REST Framework · FastAPI · PostgreSQL · Redis · pandas · NumPy · SciPy · scikit-learn · Docker

Four bullets — every number verifiable from the codebase

> **1** · Built a 3-service personal-finance platform (React/TypeScript SPA, Django REST system-of-record, stateless FastAPI analytics engine) exposing **25 REST endpoints** across **8 Django apps**, with OpenAPI documentation on both services.
>
> **2** · Implemented anomaly detection using **three algorithms** — a MAD-based Z-score, IQR Tukey fences, and a scikit-learn Isolation Forest over **8 engineered features** — computed per spending category, each producing a severity rating and a plain-English explanation.
>
> **3** · Guaranteed balance integrity across all transaction mutations using `transaction.atomic()` with `select_for_update()` row locking, verified by **70 automated tests** (43 Django, 27 FastAPI).
>
> **4** · Designed the dashboard to degrade gracefully — core financial data is computed in Django and only enriched by the analytics service, so an outage returns a complete **HTTP 200** response rather than an error; backed by a Redis analytics cache with content-addressed keys (**900 s TTL**).

What each bullet invites

Bullet	Expect to be asked
1	Why three services? Why Django *and* FastAPI? How do they communicate? What if one is down? How do you version the contract? → Parts 3, 16, 31
2	Why three algorithms? What is a MAD Z-score and why not the classic one? What is contamination? Which features and why? How do you evaluate accuracy? *(Answer honestly: there is no labelled data, so it is not measured — that is a real limitation.)* → Parts 21–26
3	What does atomic actually do? What race does the lock prevent? Why is order-of-operations critical on update? What about deadlocks between two accounts? → Part 13
4	Why 200 and not 503? How do you decide what degrades? How do you invalidate the cache? *(The best answer in the project: you don't — the key is content-addressed.)* → Parts 16, 31

> **NUMBERS NOT TO CLAIM**
>
> No user counts, no "reduced latency by X%", no "handles N requests/second", no accuracy or precision/recall figures. None of that is measured anywhere in this repository. Inventing it is the fastest way to fail a follow-up question.

PART 46 · CHAPTER X

Implemented vs Planned

Feature	Status	How it is implemented	Next step
PostgreSQL	IMPLEMENTED	`dj_database_url` parses `DATABASE_URL`; `conn_max_age=600`	Read replica; connection pooling
SQLite fallback	IMPLEMENTED	Automatic when `DATABASE_URL` is unset	Note that it ignores row locks
JWT auth	IMPLEMENTED	SimpleJWT, 30 min / 7 days, rotation on	httpOnly cookies
Token revocation	PARTIAL	`.blacklist()` called but the app is not installed, so it no-ops	Install `token_blacklist`, migrate, enable
Balance integrity	IMPLEMENTED	`atomic()` + `select_for_update()` on all three paths	Periodic reconciliation job
Analytics (summary / trends / categories / insights)	IMPLEMENTED	FastAPI services over pandas + NumPy + SciPy	Pre-aggregated tables
Z-score · IQR · Isolation Forest	IMPLEMENTED	Per-category MAD scores; Tukey fences; 200 trees over 8 features	Merchant baselines; calibrated thresholds
Anomaly review workflow	IMPLEMENTED	OPEN / REVIEWED / IGNORED, preserved across re-detection	**Feed the signal back into detection**
Redis caching	IMPLEMENTED	Analytics only, 900 s, content-addressed keys	Cache more; cheaper key
Graceful degradation	IMPLEMENTED	Local dashboard + `analytics_available` flag, with a test	Circuit breaker
Docker Compose	IMPLEMENTED	5 services, healthchecks, 2 volumes, 1 network	Multi-stage Python images; non-root user

Backend tests	IMPLEMENTED	70 tests across both services	Measure coverage; add E2E
API documentation	IMPLEMENTED	drf-spectacular + FastAPI native, assets self-hosted	Publish a versioned schema
Notification preferences	PARTIAL	Two boolean fields persist and are editable	An email backend and a scheduler
`max_transactions` limit	PARTIAL	Declared in config, never read	One Pydantic validator
`invalidate_user()` · `ping()`	PARTIAL	Defined, called by nothing	Wire up or remove
Frontend tests	PLANNED	None	Vitest + Testing Library; Playwright for the auth flow
Celery / async analytics	PLANNED	Not installed	Broker already provisioned in Redis
CI/CD	PLANNED	No workflow files	GitHub Actions: both suites + build
Cloud deployment	PLANNED	No IaC of any kind	See Part 26 of the Q&A
Monitoring / alerting	PLANNED	Logs and `/health` only	Sentry, Prometheus, correlation ids
Bank import · OCR · budgets · goals · AI assistant	PLANNED	Nothing in the repository	See Part 48

PART 48 · CHAPTER X

Future Improvements

Rank	Improvement	Why it is ranked here
1	Token blacklist + httpOnly cookies	The two real security gaps. The first is a two-line change; the second is the correct long-term design
2	Frontend test suite	The largest coverage hole. The refresh mutex especially — it is subtle and untested
3	CI pipeline	70 tests that only run when someone remembers are worth much less than 70 that run on every push
4	Backups with a tested restore	Non-negotiable before real data. An untested backup is not a backup
5	Error tracking + metrics	Today a production failure is invisible unless someone reads stdout
6	Pre-aggregated analytics	Removes the full-history payload — the first thing to break at scale
7	Celery for async detection	The `analytics.services` seam already exists; the frontend already tolerates pending states
8	Feedback-driven detection	Review status is already stored and currently unused — the cheapest real accuracy win
9	CSV import	The highest-value user feature; manual entry is the main adoption barrier
10	Budgets and goals	Turns observation into action — the natural product extension
11	Merchant-level baselines	₹5,000 at a supermarket is not ₹5,000 at a jeweller
12	Receipt OCR · bank APIs · AI assistant	Genuinely interesting, genuinely far off — and all dependent on the data pipeline above

PART 49 · CHAPTER X

Glossary

ACID
Atomicity, Consistency, Isolation, Durability — the guarantees a database transaction provides.

Anomaly
A data point that does not fit the pattern around it. Here: a transaction unusual for *your* history.

API
A contract for programs to talk to each other. FinSight's are HTTP + JSON.

Caching
Storing a computed result so the next identical request skips the work.

Celery
Python distributed task queue. **Not installed here.**

Contamination
Isolation Forest parameter: the expected proportion of anomalies. It sets the decision threshold, so ~5% is always flagged.

Django
Batteries-included Python web framework — ORM, auth, admin, migrations.

Docker
Packages an app with its dependencies into a portable container.

DRF
Django REST Framework — serializers, viewsets, routers, permissions on top of Django.

Envelope
FinSight's uniform response shape: `{success, data, message, error_code}`.

FastAPI
Modern Python API framework built on type hints and Pydantic; free OpenAPI docs.

Index
A database structure making lookups fast without scanning every row.

IQR
Interquartile range, Q3 − Q1: the spread of the middle half of the data.

Isolation Forest
Unsupervised detector that isolates points with random splits; outliers need fewer splits.

JWT
JSON Web Token — a signed, self-describing token. Readable by anyone; trustworthy because of the signature.

MAD
Median absolute deviation — a spread measure resistant to outliers.

Microservice
An independently deployable service owning one capability.

Migration
A versioned, committed description of a schema change.

N+1 query
One query for a list plus one per row. Fixed with `select_related`.

NumPy
Fast typed arrays and vectorised maths.

ORM
Object-Relational Mapper — Python objects instead of SQL strings.

Outlier
A value far from the rest. Statistical term; "anomaly" is the business term.

pandas
DataFrames — tabular data with grouping, aggregation and filtering.

PostgreSQL
Production relational database: exact decimals, row locking, real constraints.

Pydantic
Runtime validation from type annotations; the Django↔FastAPI contract.

Redis
In-memory store. Here: Django's cache backend, analytics only.

REST
Resources at URLs, HTTP verbs for operations, stateless requests.

Row lock
`SELECT … FOR UPDATE` — blocks other writers on that row until commit.

SciPy
Scientific computing on NumPy. Used here for one linear regression.

scikit-learn
Python ML library. Supplies IsolationForest, StandardScaler, OrdinalEncoder.

Serialization
Converting objects to a transmissible format, and validating on the way back.

Service-oriented architecture
Independent services over a network, split along capability lines.

SQLite
File-based database. FinSight's zero-setup dev fallback.

StandardScaler
Rescales features to mean 0, variance 1 so no feature dominates by magnitude.

Stateless
Keeps no memory between requests; any replica can serve any request.

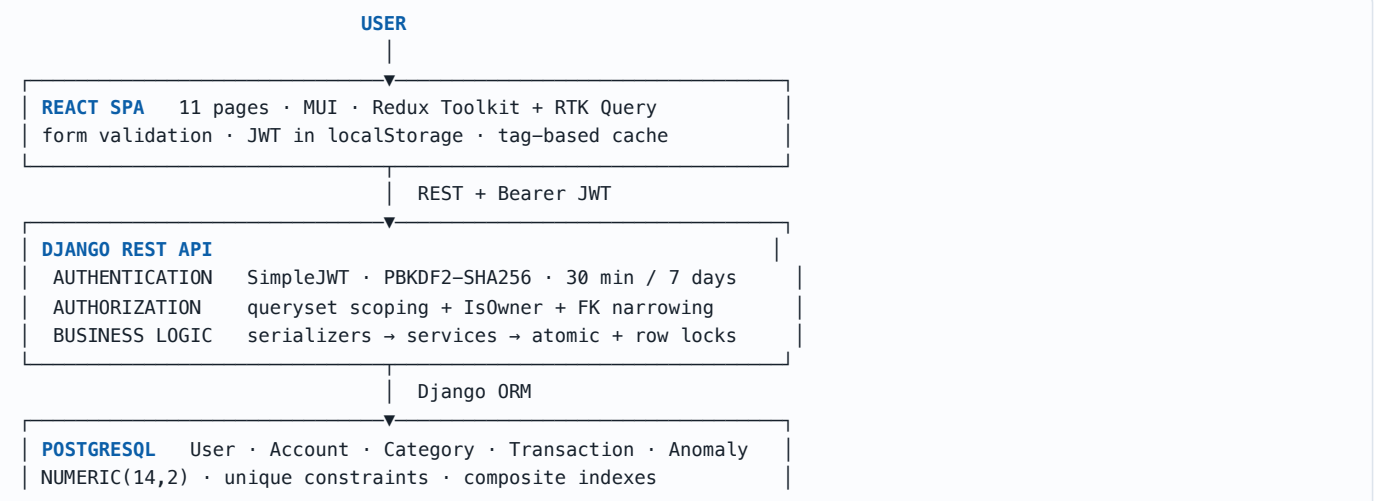
Transaction (DB)
A group of operations that all commit or all roll back.

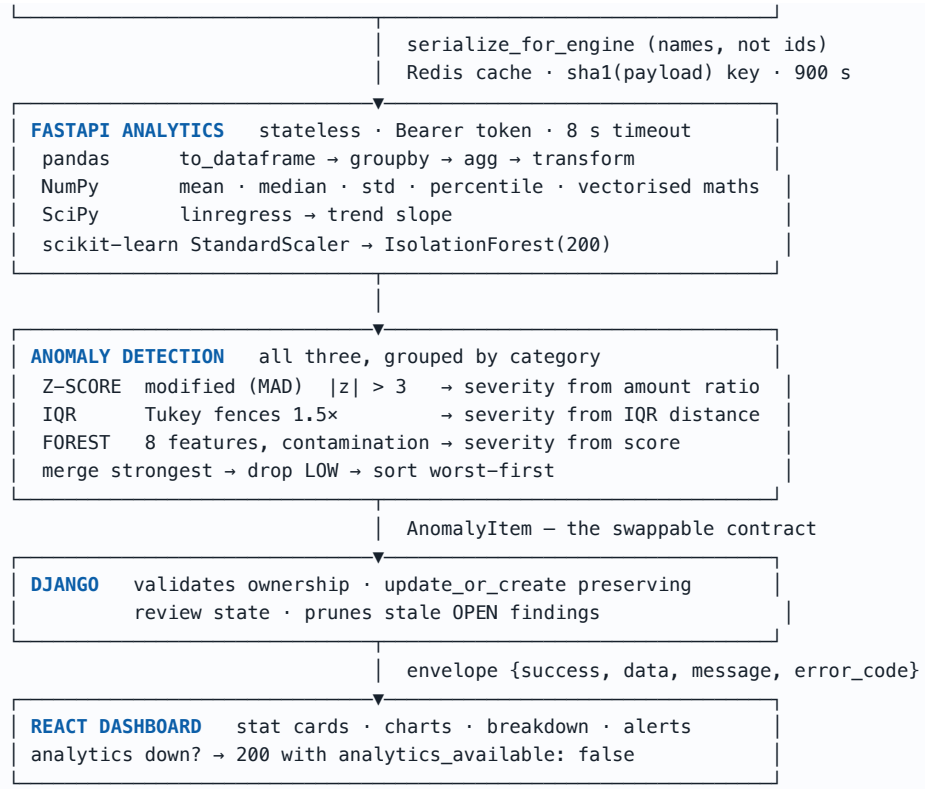
Transaction (finance)
One income or expense record. *Both meanings appear in this document — context distinguishes them.*

Z-score
How many standard deviations a value sits from the mean.

PART 50 · CHAPTER X

The Complete Picture





The architecture in one section

FinSight separates **state** from **computation**. Django owns everything that must be correct forever: identity, ownership, the ledger, and the balances derived from it. Every mutation runs inside a database transaction with row locks, so the ledger and the balances can never disagree. FastAPI owns everything derived: sums, medians, trends, category shares and anomaly findings. It has no database, no session and no memory, so it scales by replication, deploys by replacement, and can be rewritten entirely as long as it honours one JSON contract.

The boundary between them is the design. Django sends category *names* rather than primary keys, so the engine knows nothing of the schema. Django validates every transaction id it gets back, so a compromised engine cannot corrupt another user's data. And because analytics are *derived*, their absence is survivable: the dashboard computes locally and merely enriches, returning a complete 200 with a flag when the engine is unreachable — which is asserted by a test that severs the connection.

The anomaly detection is where the architecture pays off concretely. Three algorithms run behind one response shape, all computed within category groups so that rent is compared to rent and food to food. The Z-score is MAD-based specifically because the classic version hides the outliers it is meant to find. Every finding carries severity and a sentence in plain English, generated by the only service that knows why it was flagged. Because Django and React depend only on that shape, the mathematics can be replaced without touching either.

That is the whole system: a system of record that is conservative and transactional, an analytics engine that is stateless and disposable, a contract narrow enough that either side can change independently, and a frontend that degrades rather than breaks when the derived half is unavailable.

A closing note on accuracy. Every implementation claim in this document was verified against the source tree — file paths, function names, settings values, test names and counts. Where something is declared but unused, partially wired, or absent, it is labelled PARTIAL or PLANNED rather than described as working. The gaps in Parts 32, 38 and 47 are as much a part of understanding this project as the features are — and in an interview, they are usually the more persuasive half.